



Universidad
Zaragoza

Trabajo Fin de Grado

Predicción de precios de spot instances de AWS con
modelos de IA para su integración en sistemas de
orquestración de recursos en la nube

Prediction of AWS spot instance prices using AI
models for integration into cloud resource
orchestration systems

Autor

Álvaro de Francisco Nievas

Directores

Francisco Javier Fabra Caro

ESCUELA DE INGENIERÍA Y ARQUITECTURA
2025

AGRADECIMIENTOS

A mi tutor y amigo, Javier Fabra, por la confianza, las oportunidades y la guía que me has ofrecido durante todo este proceso. Tu dirección ha sido clave para convertir los desafíos de este proyecto en aprendizaje y crecimiento.

A mi familia y amigos, por vuestra motivación y apoyo constantes a lo largo de toda la carrera.

A mis compañeros de clase, por la ayuda y el compañerismo, que han hecho este camino más fácil.

RESUMEN

La optimización de costes en la computación en la nube mediante el uso de **instancias Spot de AWS** presenta un desafío fundamental debido a la alta volatilidad y opacidad de sus precios. La ingente cantidad de series temporales generadas por las distintas combinaciones de instancias y regiones hace inviable el uso de modelos predictivos tradicionales, dificultando la planificación económica de las cargas de trabajo.

Este trabajo aborda dicho reto mediante el desarrollo de un **sistema integral para la predicción de precios**. La solución se compone de un **módulo de captura** de datos históricos y un **modelo predictivo** generalista implementado en PyTorch. Este último emplea arquitecturas Sequence-to-Sequence que, en lugar de entrenarse para cada tipo de instancia, operan a nivel regional, permitiendo generalizar patrones a partir de miles de series temporales simultáneamente. Finalmente, se propone su **integración en sistemas de orquestación** para automatizar la toma de decisiones basada en las predicciones de coste.

Para la fase experimental, se diseñaron **métricas de negocio específicas** que, complementando a los indicadores de error tradicionales, permitieron extraer las conclusiones más relevantes del trabajo. Se demuestra empíricamente que un menor error de predicción no se traduce necesariamente en un mayor valor práctico; de hecho, un modelo entrenado con datos más diversos, aunque técnicamente menos preciso, duplicó la eficiencia en el ahorro de costes al capturar mejor las tendencias de precios. Además, se constata que un **preprocesamiento de datos fundamentado en el mercado**, como el ajuste de la granularidad temporal, genera mejoras de rendimiento más significativas que el aumento de la complejidad arquitectónica del modelo.

En conjunto, este TFG aporta un **framework de predicción**, una metodología de evaluación orientada al valor de negocio y un modelo robusto que valida la viabilidad de un enfoque predictivo regional para la optimización de costes en la nube.

Índice

1. Introducción y objetivos	1
1.1. Motivación	1
1.2. AWS y las instancias puntuales	1
1.3. Propuesta de mejora y contribuciones	2
1.4. Objetivos	2
1.5. Organización de la memoria	3
2. Fundamentos teóricos y trabajo relacionado	4
2.1. Computación en la nube y EC2	4
2.1.1. Modelos de servicio en la nube	4
2.1.2. Instancias EC2 y sus modelos de compra	5
2.1.3. Profundización en las instancias puntuales	6
2.2. Predicción de series temporales	8
2.2.1. Introducción a los modelos <i>Deep Learning</i>	8
2.2.2. Métricas de evaluación	8
2.3. Trabajo relacionado	9
2.3.1. Cambio de paradigma y obsolescencia de la investigación previa	9
2.3.2. Evolución de los modelos de predicción de precios	9
2.3.3. Gestión de interrupciones más allá del precio	10
2.3.4. El desafío de los datos propietarios	10
2.3.5. Análisis comparativo y posicionamiento de la solución	11
3. Diseño de la solución	12
3.1. Requisitos funcionales y no funcionales	12
3.2. Arquitectura del sistema	12
3.2.1. Sistema de captura y gestión de datos	13
3.2.2. Módulo predictivo de precios	13
3.2.3. API de servicios del modelo	14
3.3. Elección de la plataforma tecnológica	14

4. Sistema de captura y gestión de datos	16
4.1. Diseño del módulo de captura	16
4.1.1. Análisis de requisitos	16
4.1.2. Fuentes de datos y métodos de adquisición	16
4.1.3. Diseño de la base de datos	18
4.2. Caracterización de los datos	21
4.2.1. Series temporales de precios Spot	21
4.2.2. Dinámica y frecuencia de los cambios	23
4.2.3. Conclusiones del análisis de datos	23
5. Modelo predictivo de precios	25
5.1. Diseño basado en configuración	25
5.2. Preprocesamiento y preparación de datos	25
5.2.1. Generación de secuencias temporales	25
5.2.2. Normalización y escalado	26
5.2.3. Inclusión de características adicionales de la instancia	27
5.2.4. Características temporales	27
5.3. Diseño de las arquitecturas	27
5.3.1. Modelos base	27
5.3.2. Modelo Sequence-to-Sequence	28
5.3.3. Modelo Sequence-to-Sequence con características	28
5.4. Función de pérdida	29
5.5. Servitización del modelo	30
5.5.1. Generación de pronósticos (inferencia)	30
5.5.2. Diseño del servicio	31
6. Experimentación y resultados	33
6.1. Entorno de ejecución y configuración	33
6.2. Diseño de los experimentos	33
6.2.1. Selección y preparación del conjunto de datos	34
6.2.2. Estrategias y métricas de evaluación	35
6.2.3. Herramienta de visualización para la evaluación de resultados	36
6.3. Proceso de búsqueda de hiperparámetros	37
6.3.1. Parámetros de entrenamiento comunes	37
6.3.2. Fase exploratoria	37
6.3.3. Fase de refinamiento	40
6.3.4. Experimentos complementarios	41
6.4. Análisis y discusión de los resultados	42

6.4.1. Comparativa de arquitecturas	42
6.4.2. Impacto de la diversificación de datos	43
6.4.3. Selección y análisis del modelo finalista	44
6.4.4. Evaluación en otras regiones	44
7. Orquestación e integración en la nube	46
7.1. Oportunidad de mejora	46
7.1.1. Fundamentos de Apache Airflow	46
7.1.2. Diseño del pipeline de orquestación (DAGs)	47
7.2. Flujo de decisión lógica	48
7.3. Conclusiones sobre la integración	49
8. Conclusiones y trabajo futuro	51
8.1. Conclusiones Principales	51
8.2. Limitaciones del Trabajo	52
8.3. Líneas de Investigación Futura	52
9. Bibliografía	54
Lista de Figuras	57
Lista de Tablas	59
Anexos	60
A. Conceptos principales sobre <i>deep learning</i>	61
A.1. Redes neuronales recurrentes (RNNs)	61
A.2. Variantes de Redes Neuronales Recurrentes	62
A.2.1. Long Short-Term Memory (LSTM)	62
A.2.2. Gated Recurrent Unit (GRU)	63
A.3. Arquitectura Sequence-to-Sequence	65
A.4. Mecanismos de atención	65
A.5. Arquitecturas <i>Transformer</i>	66
A.6. Predicción Multi-Step en series temporales	67
A.6.1. Estrategia iterativa/autoregresiva	67
A.6.2. Estrategia directa (<i>Multi-output</i>)	67
B. Métricas de evaluación	69
B.1. Error Absoluto Medio (MAE)	69
B.2. Error Cuadrático Medio (MSE)	69
B.3. Raíz del Error Cuadrático Medio (RMSE)	69

B.4. Error Porcentual Absoluto Medio (MAPE)	70
C. Análisis detallado de la volatilidad y dinámica de precios	71
C.1. Índice de volatilidad regional	71
C.2. Análisis del comportamiento horario	72
D. Métricas de negocio	73
D.1. Precisión de tendencia (<i>Significant Trend Accuracy</i>)	73
D.2. Ahorro de coste (<i>Cost Savings</i>)	74
D.3. Ahorro con información perfecta (<i>Perfect Information Savings</i>)	74
D.4. Eficiencia del ahorro (<i>Savings Efficiency</i>)	74
E. Artefactos del experimento y enfoque dirigido por configuración	76
E.1. Diseño basado en configuración	76
E.2. Estructura de Artefactos Generados	76
E.2.1. Directorio de datos: <code>data/</code>	76
E.2.2. Directorio de entrenamiento: <code>training/</code>	77
E.2.3. Directorio de evaluación: <code>evaluation/</code>	77
E.2.4. Artefactos raíz	77
F. Resultados de los modelos entrenados	78
G. Herramienta de visualización para el análisis de resultados	81
G.1. Objetivos y funcionalidades	81
G.2. Visualizaciones implementadas	82
G.2.1. Comparativa global de modelos	82
G.2.2. Análisis del horizonte temporal y atributos de instancia	82
G.2.3. Distribución de errores y métricas de negocio	83
H. Traza de ejecución	85
I. Estimación del esfuerzo	88

Capítulo 1

Introducción y objetivos

La gestión eficiente de costes es un factor crítico para las organizaciones que operan en la nube. Este trabajo se centra en el potencial de optimización que ofrecen las instancias puntuales (Spot Instances) de AWS, cuyas particularidades y dinámicas de precios introducen tanto un reto como una oportunidad. En este capítulo se contextualiza el problema, se presenta una propuesta de mejora para abordar los desafíos clave y se definen los objetivos específicos de la investigación.

1.1. Motivación

Las empresas destinan aproximadamente el 60 % de su presupuesto de infraestructura de sistemas informáticos (TI) a servicios en la nube de proveedores como AWS, Microsoft Azure o Google Cloud, beneficiándose de su escalabilidad y eficiencia [23]. A diferencia del modelo tradicional, que exigía grandes inversiones y largos ciclos de planificación para la adquisición y mantenimiento de servidores físicos, el modelo de nube pública ofrece acceso bajo demanda a recursos informáticos, plataformas y aplicaciones a través de centros de datos externos distribuidos globalmente. Este enfoque permite un pago por uso real, reduciendo costes operativos. Frecuentemente, la virtualización permite que múltiples clientes compartan un mismo servidor físico mediante máquinas virtuales (VMs) independientes. Bajo un modelo de responsabilidad compartida, los proveedores se encargan de la infraestructura y la seguridad básica, mientras que los clientes gestionan la protección de sus datos y la configuración de sus aplicaciones.

Un uso eficiente de los recursos es clave para controlar los costes, ya que el sobreaprovisionamiento (instancias sobredimensionadas, almacenamiento no utilizado) genera gastos innecesarios al facturarse por consumo real.

1.2. AWS y las instancias puntuales

Los proveedores de nube ofrecen distintos modelos de servicio, distinguidos por el nivel de abstracción y el reparto de responsabilidades, para gestionar el consumo. Este trabajo se enfoca en el modelo de Infraestructura como Servicio (IaaS), que proporciona recursos de computación básicos como servidores virtuales. AWS, líder en este sector, ofrece Amazon EC2 (Elastic Compute Cloud)[6]

para el alquiler de VMs. Para optimizar el gasto, AWS presenta varias modalidades de compra, como las instancias Spot, también llamadas instancias puntuales. Estas aprovechan la capacidad sobrante de los centros de datos y pueden alcanzar hasta un 90 % de descuento sobre el precio bajo demanda [7]. Son ideales para cargas de trabajo tolerantes a fallos y no urgentes, como el procesamiento por lotes (batch processing)[11], ya que por su naturaleza pueden tolerar interrupciones. Las instancias Spot son fundamentales para la optimización de costes y para aprovecharlas eficazmente, es crucial entender sus características:

- **Interrupciones:** AWS puede reclamar una instancia Spot con solo dos minutos de preaviso, lo que requiere que las aplicaciones diseñadas para ellas sean tolerantes a interrupciones y capaces de guardar su estado.
- **Disponibilidad variable:** La oferta de capacidad spot fluctúa, ya que depende de otros factores como la demanda de instancias *bajo demanda*, lo que limita su disponibilidad.
- **Precios dinámicos:** Los precios varían constantemente a lo largo del día y entre regiones, lo que supone un reto, pero también una oportunidad para la optimización de costes.

1.3. Propuesta de mejora y contribuciones

La dinámica de precios de las instancias Spot presenta una clara oportunidad de optimización. Su coste varía, de media, tres veces al día, con diferencias notables que dependen del tipo de instancia y de su localización específica. Anticipar estas variaciones permite planificar la ejecución de tareas en los momentos más económicos, obteniendo así los mismos resultados a un coste inferior. No obstante, el desarrollo de un sistema que capitalice esta ventaja enfrenta retos significativos:

- **Opacidad del algoritmo:** El algoritmo de precios es propietario de AWS, por lo que no se conocen con certeza todos los factores que influyen en las variaciones.
- **Información incompleta:** Aunque se dispone de un historial de precios de 90 días, AWS no proporciona datos sobre la capacidad disponible, un factor que se presupone clave en la fijación del precio.
- **Escalabilidad del modelado:** El elevado número de series temporales únicas, producto de la combinación de regiones, zonas y tipos de instancia, hace inviable entrenar un modelo predictivo para cada una, lo que exige un enfoque de modelado generalista.

1.4. Objetivos

Este Trabajo Fin de Grado busca desarrollar un **sistema de predicción de precios de instancias puntuales de AWS** mediante modelos de **inteligencia artificial (IA)** para anticipar fluctuaciones. Se investigará su integración en sistemas de orquestación de recursos en la nube para

optimizar la asignación y gestión de costes. El resultado final será un **servicio web de predicción** accesible. El enfoque metodológico es computacional, utilizando Python y PyTorch para el desarrollo y entrenamiento. Se explorarán y compararán modelos de regresión basados en *Deep Learning*, enfocándose en redes neuronales recurrentes (RNNs). Para lograr estos objetivos, se definen las siguientes tareas:

1. Revisión bibliográfica y análisis del contexto de precios de instancias puntuales de AWS.
2. Recopilación y preprocesamiento de datos históricos de precios.
3. Diseño, entrenamiento y evaluación de modelos de predicción basados en IA.
4. Comparativa de resultados y selección del modelo óptimo.
5. Desarrollo e implementación de un servicio web para la exposición de predicciones.

1.5. Organización de la memoria

El Capítulo 2 establece el marco conceptual, profundizando en los fundamentos de la computación en la nube, las particularidades de las instancias Spot y las técnicas de predicción de series temporales. Además, se analiza el trabajo relacionado para posicionar esta propuesta en el estado del arte.

Sobre estas bases, el Capítulo 3 define el diseño de la solución, especificando sus requisitos y la arquitectura modular del sistema. La implementación se inicia en el Capítulo 4, que detalla el sistema de captura y gestión de datos, un componente esencial para construir el histórico de precios necesario para el entrenamiento de los modelos.

El núcleo técnico de la propuesta se desarrolla en el Capítulo 5, donde se describe el diseño y entrenamiento del modelo predictivo basado en arquitecturas de inteligencia artificial, así como su posterior servitización mediante una API. La validación de dicho modelo se lleva a cabo en el Capítulo 6, que presenta un diseño de experimentación riguroso junto a un análisis exhaustivo de los resultados.

Posteriormente, el Capítulo 7 explora una aplicación práctica de la solución, hipotetizando su integración en un orquestador de recursos como Apache Airflow para automatizar la toma de decisiones basada en costes. Finalmente, el Capítulo 8 sintetiza las principales contribuciones del trabajo, discute sus limitaciones y propone futuras líneas de investigación.

Capítulo 2

Fundamentos teóricos y trabajo relacionado

Este capítulo establece las bases técnicas y metodológicas del proyecto. Se introduce la computación en la nube, profundizando en el modelo IaaS, el servicio Amazon EC2 y las particularidades de las instancias *spot* para tareas tolerantes a interrupciones. Posteriormente, se exponen los fundamentos de la predicción de series temporales, abarcando desde los métodos estadísticos hasta las arquitecturas de *deep learning* y las métricas de evaluación empleadas. El capítulo concluye con una revisión del trabajo relacionado para posicionar esta propuesta en el estado del arte de la predicción de precios en entornos cloud.

2.1. Computación en la nube y EC2

Para comprender el contexto del presente trabajo, es fundamental establecer las bases de la computación en la nube y, en particular, el servicio de Amazon Web Services (AWS) centrado en instancias virtuales, Amazon EC2. La computación en la nube se ha consolidado como un paradigma fundamental en la infraestructura informática, ofreciendo flexibilidad, escalabilidad y eficiencia en la gestión de recursos.

2.1.1. Modelos de servicio en la nube

Los servicios en la nube se suelen clasificar en tres modelos principales, que se distinguen por el nivel de control que otorgan al usuario, como se puede ver en la Figura 2.1.

El modelo con mayor flexibilidad es la **Infraestructura como Servicio (IaaS)**, donde el proveedor se ocupa del hardware y la virtualización, pero el cliente tiene pleno control sobre el sistema operativo y las aplicaciones, beneficiándose de un esquema de pago por uso. Un paso más allá se encuentra la **Plataforma como Servicio (PaaS)**, en la que el proveedor gestiona también el sistema operativo y el middleware; esto permite al cliente olvidarse de la infraestructura subyacente y centrarse por completo en su código y sus datos. Por último, el **Software como Servicio (SaaS)** representa el nivel más alto de abstracción: el proveedor entrega una aplicación final, lista para ser utilizada, normalmente a través de un modelo de suscripción.

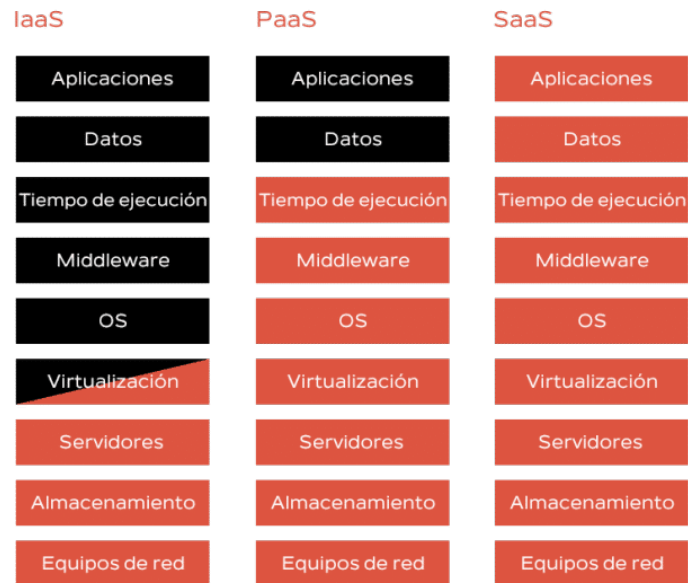


Figura 2.1: Modelos de Servicio en la Nube: IaaS, PaaS, SaaS. Fuente: [35]

Para este proyecto, el enfoque se centra en el modelo IaaS, ya que es el que sustenta los servicios de instancias virtuales como AWS EC2 y proporciona el control granular de los recursos necesario para los objetivos de este trabajo.

2.1.2. Instancias EC2 y sus modelos de compra

Elastic Compute Cloud (EC2) es un servicio fundamental de AWS que ofrece capacidad de computación escalable en la nube. Permite a los usuarios lanzar servidores virtuales (*instancias*) con configuraciones adaptables (CPU, memoria, almacenamiento, red), eliminando la inversión en hardware físico y facilitando el despliegue rápido de aplicaciones con capacidad ajustable a la demanda [6].

EC2 puede integrarse en arquitecturas complejas, como balanceadores de carga o clústeres de alto rendimiento. AWS ofrece múltiples opciones de compra para instancias EC2, optimizadas por coste/rendimiento o por requisitos específicos [13]. La selección del modelo de compra dependerá de la previsibilidad de la carga de trabajo, la tolerancia a interrupciones y otros factores.

Instancias Bajo Demanda (*On-Demand*) Son el modelo de pago por uso más flexible, facturando la capacidad de cómputo por horas o segundos sin compromisos a largo plazo. Son adecuadas para cargas de trabajo con picos irregulares o de corta duración que no admiten interrupciones. Su precio, el más elevado, sirve como referencia para calcular los ahorros que ofrecen otros modelos de compra.

Instancias puntuales (*Spot*) Permiten acceder a capacidad EC2 no utilizada con significativos descuentos sobre el precio bajo demanda. Constituyen una opción rentable para cargas de trabajo

tolerantes a interrupciones, como el procesamiento por lotes, ya que pueden ser recuperadas por AWS si la capacidad es requerida.

Savings Plans Ofrecen una reducción de costos a cambio de un compromiso de gasto constante (en USD por hora) durante un periodo de 1 o 3 años. Este modelo es flexible, ya que el descuento se aplica automáticamente a diferentes tipos de instancias y regiones, siendo adecuado para un volumen de uso predecible sin ligarse a una configuración concreta.

Instancias reservadas (*Reserved*) Proporcionan el descuento más elevado (hasta un 75 %) a cambio de un compromiso de uso de 1 o 3 años para una configuración de instancia, región y sistema operativo específicos. Son la opción óptima para cargas de trabajo con un uso predecible y sostenido.

2.1.3. Profundización en las instancias puntuales

Las **instancias puntuales** (*Spot Instances*) de Amazon EC2 son un modelo de compra clave para la optimización de costos en la nube. Permiten solicitar capacidad de cómputo EC2 no utilizada por AWS a un precio significativamente inferior al de las instancias bajo demanda. Este ahorro, que puede alcanzar hasta un 90 % [7], las hace atractivas para cargas de trabajo tolerantes a interrupciones como procesamiento por lotes, desarrollo o CI/CD. Su disponibilidad está sujeta a la capacidad excedente de AWS, lo que implica que pueden ser recuperadas si la capacidad es requerida por instancias bajo demanda o reservadas. Cuando Amazon EC2 recupera una instancia spot, este evento se denomina **interrupción**.

Evolución del algoritmo de precios

El mecanismo de precios de las instancias Spot ha evolucionado. Inicialmente, AWS usaba un modelo de **subasta**, donde el precio fluctuaba dinámicamente según la oferta máxima del usuario. Si la oferta era baja o superada, la instancia podía ser interrumpida. Esto generaba alta volatilidad e imprevisibilidad.

Entre finales de 2017 e inicios de 2018, AWS **modificó el algoritmo de precios** para mayor estabilidad. El precio actual de las instancias puntuales se determina por la oferta y la demanda de la capacidad no utilizada de AWS, no por las pujas de los usuarios. Los precios cambian gradualmente, simplificando la operativa y la predicción de estos.

Causas y factores de interrupción

La interrupción de una instancia Spot por parte de Amazon EC2 puede ocurrir por varias razones, las principales causas son:

- **Capacidad:** Es la causa más común y ocurre cuando AWS recupera los recursos para satisfacer la demanda de instancias *On-Demand* o Reservadas, o por mantenimiento del hardware subyacente.

- **Precio:** Si el usuario especificó un precio máximo en la solicitud de spot, la instancia se interrumpe si el precio actual supera ese máximo. Este escenario es menos frecuente con el nuevo modelo de precios.
- **Restricciones:** Si la solicitud incluye restricciones como grupos de lanzamiento o de Zona de Disponibilidad, las instancias se terminan si la restricción ya no puede ser satisfecha.

La acción que AWS toma al interrumpir una instancia (terminar, detener o hibernar) depende del comportamiento de interrupción especificado por el usuario al crear la solicitud.

Causas y factores de interrupción

Tras el cambio de paradigma del mercado Spot en 2017, la causa principal de interrupción ya no está ligada a un modelo de subasta, sino a la necesidad de AWS de reclamar la capacidad de cómputo. Esto convierte al precio en un indicador poco fiable de la estabilidad, dejando obsoleta la investigación previa que se centraba en predecir el precio para evitar interrupciones. Las principales causas son:

- **Capacidad:** Es el motivo más frecuente y ocurre cuando AWS necesita los recursos para satisfacer la demanda de instancias *On-Demand* o Reservadas, o por mantenimiento del hardware subyacente.
- **Precio máximo (definido por el usuario):** Se activa si el precio Spot supera el límite opcional fijado por el usuario en su solicitud. Es un factor secundario y no el mecanismo principal del mercado.
- **Restricciones:** Tienen lugar si la solicitud incluye restricciones, como grupos de lanzamiento o de Zona de Disponibilidad, que ya no pueden ser satisfechas.

La acción que AWS toma al interrumpir una instancia (terminar, detener o hibernar) depende del comportamiento especificado por el usuario al crear la solicitud.

Mecanismos de notificación de interrupción

Para permitir a los usuarios reaccionar a la interrupción inminente de una instancia spot, AWS proporciona dos mecanismos principales que se entregan a través de eventos de EventBridge y como elementos en los metadatos de la instancia (*Instance Metadata Service* - IMDS):

- **Aviso de interrupción (*eviction notice*):** Proporciona una notificación de **dos minutos** antes de que la instancia spot sea terminada. Este aviso es crucial para que las aplicaciones puedan realizar acciones como guardar el estado del trabajo en progreso, completar tareas pendientes, o transferir datos a almacenamiento persistente antes de la interrupción[20].
- **Recomendación de reequilibrio (*rebalance recommendation*):** Indica un **alto riesgo de interrupción** para una instancia. Se emite *antes* del aviso de dos minutos, ofreciendo una

oportunidad para tomar medidas preventivas como la migración de cargas de trabajo. Es una señal de mayor probabilidad de interrupción, pero no garantiza la interrupción ni su ausencia[12].

Estos mecanismos de notificación son cruciales para mitigar el impacto de las interrupciones, pero la anticipación de la volatilidad y la probabilidad de interrupción son fundamentales para una gestión eficiente.

2.2. Predicción de series temporales

La predicción de series temporales busca estimar valores futuros de una secuencia de datos basándose en sus observaciones pasadas. Dichas series se caracterizan por componentes como la tendencia, la estacionalidad y el ruido. Para su modelado, tradicionalmente se distinguen dos enfoques: los modelos estadísticos y los basados en *Deep Learning*. Los primeros, como los de la familia ARIMA (*Medias Móviles Integradas AutoRegresivas*)[16], se fundamentan en supuestos estadísticos sobre la estructura de los datos, como la linealidad o la estacionariedad (que la media y la varianza de la serie no cambien en el tiempo).

2.2.1. Introducción a los modelos *Deep Learning*

Aunque los métodos estadísticos son robustos para series individuales con patrones bien definidos, su aplicación en este proyecto presenta una limitación insalvable: su incapacidad para escalar. La necesidad de entrenar un modelo individual para cada una de las 109.264 series de precios de AWS consideradas ¹ lo convierte en una tarea inviable.

Por ello, se recurre a modelos de *Deep Learning*, que superan esta barrera al no requerir supuestos estrictos sobre la estructura de los datos. Su principal ventaja es la capacidad para construir **modelos multivariantes** que aprenden patrones complejos y no lineales directamente de grandes volúmenes de datos, mostrando una mayor robustez ante el ruido y las irregularidades. Esto permite que un único modelo no solo procese miles de series simultáneamente, sino que también incorpore características exógenas (tipo de instancia, región, etc.) para mejorar la precisión predictiva.

En el Anexo A se puede encontrar una introducción detallada a los conceptos de *Deep Learning* relevantes para este trabajo, incluyendo las redes neuronales recurrentes (RNNs) y sus variantes (LSTM, GRU), las arquitecturas *sequence-to-sequence*, los mecanismos de atención y la predicción *multi-step*.

2.2.2. Métricas de evaluación

La evaluación del modelo propuesto requiere un enfoque dual que combine la precisión estadística con el valor práctico. Por un lado, es necesario medir el error de las predicciones mediante métricas

¹El cálculo de 109.264 series temporales únicas se refiere a la combinación de tipo de instancia, sistema operativo y zona de disponibilidad para el 11 de marzo de 2025, abarcando las siguientes regiones de AWS: ap-northeast-1, ap-northeast-2, ap-northeast-3, ap-south-1, ap-southeast-1, ap-southeast-2, ca-central-1, eu-central-1, eu-north-1, eu-west-1, eu-west-2, eu-west-3, sa-east-1, us-east-1, us-east-2, y us-west-1, us-west-2.

estándar como el Error Absoluto Medio (MAE). Por otro, y dado que un menor error no garantiza un mayor ahorro, estas métricas son insuficientes. Por ello, este trabajo define e implementa también indicadores de negocio para cuantificar el impacto económico de las predicciones, como la Eficiencia del Ahorro. A continuación, se detallan las métricas de error estándar utilizadas, que se explican en detalle en el Anexo B.

- **MAE:** el **error absoluto medio** (*Mean Absolute Error*, MAE) calcula el promedio de los valores absolutos de las diferencias entre las predicciones y los valores reales.
- **MSE:** el **error cuadrático medio** (*Mean Squared Error*, MSE) mide el promedio de los cuadrados de los errores, es decir, la diferencia entre los valores predichos y los valores reales.
- **RMSE:** la **raíz del error cuadrático medio** (*Root Mean Squared Error*, RMSE) es la raíz cuadrada del MSE. Ofrece una métrica en las mismas unidades que la variable que se predice, lo que facilita su interpretación y la comparación directa con el MAE.
- **MAPE:** el **error porcentual absoluto medio** (*Mean Absolute Percentage Error*, MAPE) es una métrica de precisión que expresa el error como un porcentaje del valor real.

2.3. Trabajo relacionado

La optimización de costes en la computación en la nube, particularmente en el mercado de las instancias Spot de AWS, ha sido un área de intensa investigación. La capacidad de anticipar las fluctuaciones de precios y la disponibilidad de recursos permite a las organizaciones planificar la ejecución de sus cargas de trabajo de manera más económica y resiliente. Esta sección presenta una revisión del estado del arte, centrándose en la evolución de los modelos de predicción y las herramientas de orquestación inteligente, para posicionar las contribuciones de este trabajo.

2.3.1. Cambio de paradigma y obsolescencia de la investigación previa

La investigación sobre el comportamiento de las instancias Spot dio un giro radical a partir de 2017. Ese año, un cambio fundamental en el mecanismo de mercado de AWS dejó obsoleta buena parte del trabajo que se había realizado hasta la fecha. La clave de esta transformación fue que la interrupción de una instancia dejó de estar ligada al precio que el usuario estaba dispuesto a pagar. Desde entonces, las interrupciones se producen simplemente cuando AWS necesita recuperar la capacidad de cómputo, sin importar el precio spot en ese momento[27, 36]. Como consecuencia, los estudios enfocados en predecir el precio para evitar interrupciones[27] perdieron su relevancia práctica.

2.3.2. Evolución de los modelos de predicción de precios

Tras el cambio de paradigma, la investigación se reorientó hacia la predicción de precios para la optimización de costes, explorando desde modelos específicos por instancia hasta arquitecturas de aprendizaje profundo más generalizables.

Modelos locales y sus limitaciones

Un enfoque común ha sido el paradigma del **modelo local**, que entrena un predictor para cada tipo de instancia. Un trabajo representativo es el de Malik and Bagmar (2021), quienes proponen una técnica para predecir los precios de las instancias Spot de Amazon EC2 utilizando un modelo de **Random Forest** [28]. Su metodología se basa en entrenar el modelo con un historial de 90 días de precios para una combinación específica de instancia y zona de disponibilidad, con el fin de ofrecer un rango de precios estimado para optimizar las pujas [28]. Sin embargo, este enfoque, al igual que otros modelos locales, demuestra ser una tarea de ingeniería frágil y que no escala, dada la enorme cantidad de series temporales existentes en el mercado de AWS.

Hacia modelos generales con aprendizaje profundo

Para superar las limitaciones de los modelos locales, la investigación ha avanzado hacia arquitecturas más potentes. Un ejemplo de esta transición es el trabajo de Song, Lin, and Zou (2022), quienes proponen el uso de una **Red Convolutiva Temporal (TCN)** para la predicción de precios de las instancias EC2 Spot [34]. Las TCN son una arquitectura de red neuronal diseñada para datos secuenciales que captura eficientemente dependencias a largo plazo, representando un paso hacia modelos más sofisticados y generalizables que pueden aprender patrones complejos en los datos de precios [34].

2.3.3. Gestión de interrupciones más allá del precio

Dado que el precio ya no es un indicador fiable de la estabilidad, la gestión de interrupciones requiere un enfoque multidimensional. El trabajo de Lee, Hwang, and Lee (2022) y su sistema **SpotLake** aborda esta cuestión de frente [27]. SpotLake es un servicio de archivo de datos que recolecta y sirve no solo el historial de precios, sino también datasets más recientes y menos estudiados como el *Spot Placement Score* (SPS), que indica la probabilidad de que una solicitud sea satisfecha, y la frecuencia de interrupciones, que los autores convierten en un *Interrupt-free Score* (IFS) [27]. Un hallazgo clave de su análisis es la baja correlación entre precio, disponibilidad y estabilidad, lo que subraya la necesidad de modelos que no dependan únicamente del precio para tomar decisiones informadas [27].

2.3.4. El desafío de los datos propietarios

El desafío de predecir interrupciones se ve agravado por la asimetría de información. Los modelos más precisos suelen ser desarrollados por los propios proveedores de la nube, que tienen acceso a telemetría interna. Un caso paradigmático es el trabajo de Yang et al. (2022), quienes desarrollaron un modelo de alta precisión para la predicción de desalojos de VMs Spot en Microsoft Azure [36]. Sin embargo, su enfoque depende de un **dataset interno de Microsoft** que contiene registros históricos de despliegues y desalojos, lo que lo convierte en una solución de “caja negra”, opaca e irreproducible

para la comunidad investigadora externa [36].

2.3.5. Análisis comparativo y posicionamiento de la solución

El análisis del estado del arte revela una brecha de oportunidad. Las herramientas modernas como Karpenter han evolucionado hacia el uso de estrategias de asignación de AWS que funcionan como una **caja negra**. Por otro lado, las plataformas comerciales, como **nOps** o **Cast AI**, a menudo se centran en la predicción de interrupciones y la gestión de costes, pero operan como servicios propietarios sin ofrecer transparencia en sus modelos predictivos [29, 25].

Este trabajo se posiciona para llenar ese vacío. En lugar de competir con la estrategia de aprovisionamiento de Karpenter, este proyecto se centra en desarrollar un marco de trabajo, transparente y de propósito general para la predicción avanzada de precios, basado en la metodología de Modelos de Predicción Globales (GFM). La contribución principal es la creación de un sistema que pueda ser integrado en orquestadores para informar decisiones de optimización de costes de manera proactiva, ofreciendo una alternativa a las estrategias nativas de AWS y una mayor flexibilidad para una gama más amplia de cargas de trabajo.

Capítulo 3

Diseño de la solución

El diseño de la solución se fundamenta en los requisitos funcionales y no funcionales definidos para el sistema. En este capítulo se presenta una arquitectura modular, detallando sus componentes principales y las interacciones entre ellos. Asimismo, se justifica la elección de Amazon Web Services (AWS) como plataforma tecnológica, argumentando cómo sus servicios contribuyen a los objetivos del proyecto.

3.1. Requisitos funcionales y no funcionales

Los requisitos funcionales y no funcionales que definen las capacidades y calidad del sistema se detallan a continuación en las Tablas 3.1 y 3.2.

Tabla 3.1: Requisitos funcionales del sistema

ID	Descripción del requisito funcional
RF01	El sistema debe generar pronósticos de precios para las instancias Spot de AWS con un horizonte mínimo de 10 días.
RF02	El sistema debe recopilar y almacenar continuamente el historial de precios de instancias Spot así como otros datos contextuales.
RF03	El sistema debe facilitar la integración de las predicciones de precios en sistemas de orquestación de recursos en la nube para la toma de decisiones automatizada.
RF04	El sistema debe ofrecer una API RESTful para el acceso programático a las predicciones generadas y a los datos históricos de precios.
RF05	El sistema debe proporcionar mecanismos para la evaluación automatizada del rendimiento del modelo predictivo, incluyendo métricas de error y de negocio.

3.2. Arquitectura del sistema

Se optó por una arquitectura modular que separa el sistema de captura de datos del módulo predictivo. Esta decisión responde a la naturaleza distinta de ambos componentes: la captura es una tarea de ingeniería de datos continua y estable, mientras que el modelado predictivo es un proceso experimental e iterativo. Separarlos permite desarrollar, desplegar y escalar cada componente de forma independiente. Por ejemplo, es posible actualizar el modelo de predicción sin interrumpir la recolección de datos históricos, lo que simplifica y agiliza el ciclo de experimentación.

Tabla 3.2: Requisitos no funcionales del sistema

ID	Descripción del requisito no funcional
RNF01	Debe alcanzar un MAPE (Error Porcentual Absoluto Medio) inferior al 25 %.
RNF02	La latencia en la generación de predicciones y en la respuesta de la API debe ser mínima para la toma de decisiones en tiempo real.
RNF03	La arquitectura debe escalar para manejar un número creciente de series temporales, mayor volumen de datos históricos y elevado número de solicitudes de predicción.
RNF04	El sistema debe ser robusto ante fluctuaciones inesperadas o cambios abruptos en los patrones de precios, adaptándose sin degradación crítica de su rendimiento.
RNF05	El diseño y la implementación de la solución deben optimizar el gasto en recursos para su despliegue y operación.
RNF06	El código y la infraestructura deben ser fáciles de mantener, actualizar y extender, promoviendo la modularidad y la claridad.
RNF07	El sistema debe proteger el acceso a los datos, las credenciales de AWS y las operaciones del sistema, adhiriéndose al principio de mínimo privilegio.

La solución propuesta se fundamenta en la interacción coordinada de tres componentes, cada uno responsable de problemáticas específicas. La Figura 3.1 ilustra la arquitectura general.

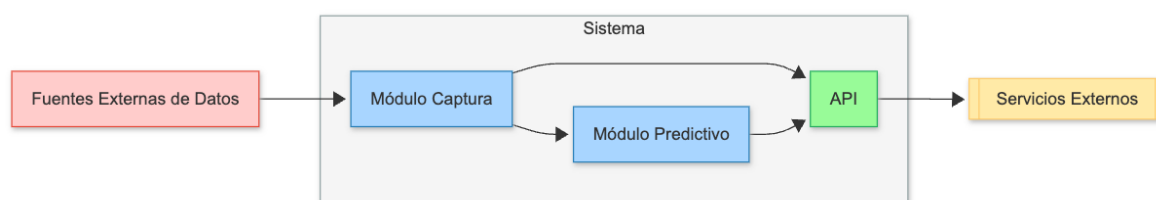


Figura 3.1: Arquitectura general del sistema de predicción

3.2.1. Sistema de captura y gestión de datos

Este módulo es el responsable de la adquisición, procesamiento y almacenamiento de datos de precios y metadatos de instancias. Su objetivo principal es superar la limitación de 90 días de historial, construyendo una base de datos persistente con información completa.

El **Capítulo 4: Sistema de captura y gestión de datos** se dedicará a describir en detalle el diseño e implementación de este módulo, incluyendo las fuentes de datos, el esquema de la base de datos y los procesos de ingestión. Para una representación más detallada de su funcionamiento interno, véase la Figura 3.2.

3.2.2. Módulo predictivo de precios

El componente central de la solución es el modelo predictivo de precios, cuyo objetivo es anticipar las fluctuaciones de las instancias Spot. Para lograr esto, se entrenarán y evaluarán diversos modelos de inteligencia artificial, seleccionando el de mejor rendimiento para la fase de inferencia.

El **Capítulo 5: Modelo predictivo de precios** detallará el proceso de preprocesamiento de datos, la selección y configuración de los modelos predictivos, así como el *pipeline* completo de entrenamiento, evaluación e inferencia. La Figura 3.3 presenta la arquitectura de este módulo, ilustrando sus

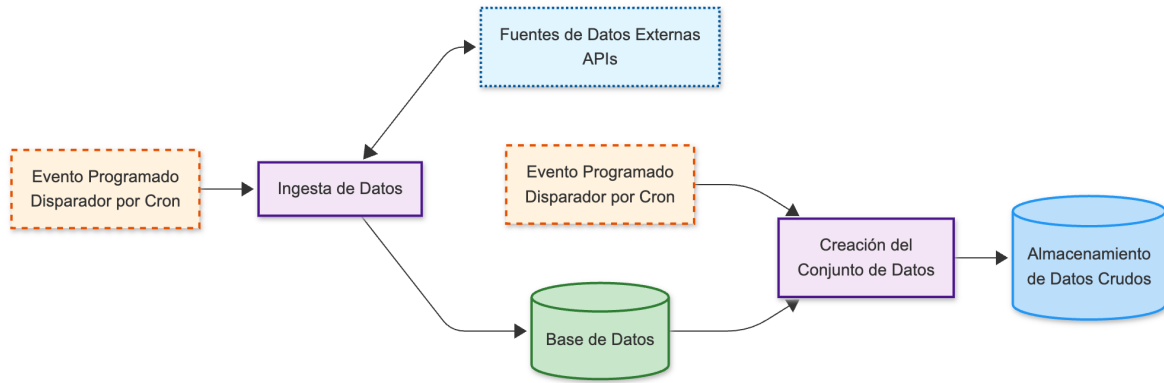


Figura 3.2: Arquitectura del módulo de captura y gestión de datos

interacciones y componentes principales.

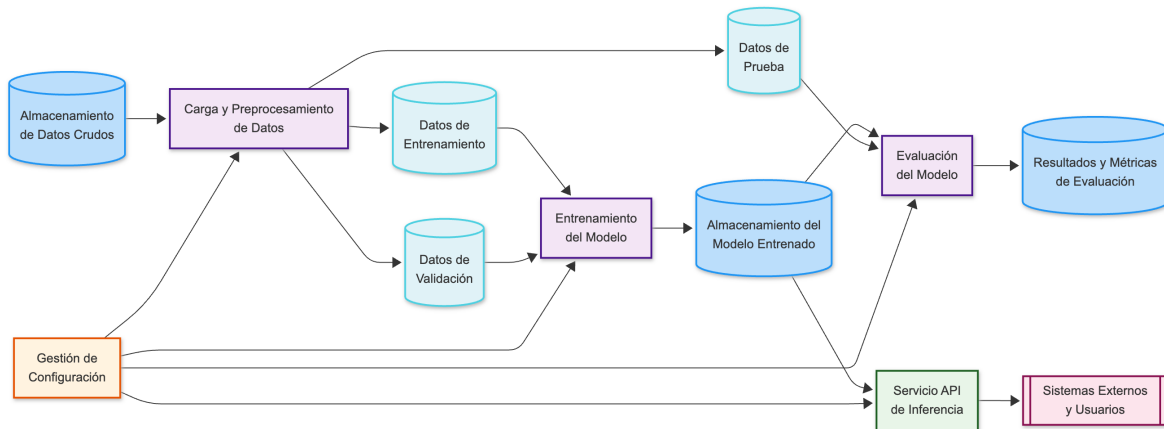


Figura 3.3: Arquitectura del módulo predictivo

3.2.3. API de servicios del modelo

Para que las predicciones sean utilizables, deben ser accesibles por otros sistemas y aplicaciones. La API permitirá la consulta de estas y, adicionalmente, de datos históricos de precios y otras características. Aunque este componente forma parte del diseño general, su implementación detallada y su integración con los sistemas de orquestación se abordarán en el **Capítulo 5: Modelo predictivo de precios** y **Capítulo 7: Orquestación e integración en la nube**.

3.3. Elección de la plataforma tecnológica

La selección de Amazon Web Services (AWS) como plataforma para el desarrollo y despliegue se justifica por los siguientes motivos:

- **Acceso directo a APIs:** AWS proporciona acceso de baja latencia a sus APIs de servicios, esencial para la eficiente recolección de datos de precios de instancias.
- **Amplia oferta de servicios:** Su catálogo de servicios es extenso y competitivo, abarcando

funciones *serverless*, bases de datos gestionadas y herramientas de *Machine Learning*, entre otros.

Para la implementación de la solución, se integrarán los siguientes servicios clave del ecosistema AWS:

- **AWS Lambda:** Se emplearán funciones *Lambda* para la ejecución programada de las tareas de recolección de datos.
- **Amazon RDS (*Relational Database Service*):** Para la persistencia y gestión del historial de precios *Spot* y *On-Demand*, así como de los metadatos de las instancias.
- **Amazon S3 (*Simple Storage Service*):** Se utilizará como almacenamiento de objetos para los conjuntos de datos preprocesados y otros artefactos.
- **Amazon EC2 (*Elastic Compute Cloud*):** Se hará uso de instancias *EC2* con capacidades de GPU para el entrenamiento de los modelos.

Capítulo 4

Sistema de captura y gestión de datos

Este capítulo se centra en la fase inicial del proyecto: la adquisición, gestión y caracterización de los datos de precios¹. Se establecen los requisitos funcionales y no funcionales del módulo de captura, detallando las fuentes y métodos de adquisición de la información. Asimismo, se explica el diseño de la base de datos para su almacenamiento y se analiza el comportamiento de los datos, elementos clave para el modelado predictivo.

4.1. Diseño del módulo de captura

4.1.1. Análisis de requisitos

Los requisitos específicos del módulo de captura se detallan a continuación. La Tabla 4.1 presenta los requisitos funcionales y la Tabla 4.2 los no funcionales.

Tabla 4.1: Requisitos funcionales del módulo de captura

ID	Descripción del Requisito Funcional
RF-REC01	El módulo debe obtener precios históricos de instancias Spot de todas las regiones y zonas de disponibilidad relevantes.
RF-REC02	El módulo debe obtener precios bajo demanda (On-Demand) de las instancias para referencia.
RF-REC03	El módulo debe recopilar metadatos y características técnicas de los tipos de instancias (ej., vCPUs, RAM, arquitectura).
RF-REC04	El módulo debe almacenar los datos sin procesamiento previo significativo, manteniendo su granularidad y formato original.
RF-REC05	El módulo debe almacenar la información recopilada de forma persistente y estructurada.
RF-REC08	El módulo debe asegurar la coherencia y validez de los datos almacenados mediante el uso de Foreign Keys.

4.1.2. Fuentes de datos y métodos de adquisición

La estrategia de captura de información se basa en un enfoque puramente *serverless*, empleando funciones **Lambda** como unidad de ejecución. Esta elección es costo-eficiente, ya que las tareas de

¹La implementación del sistema de captura de datos descrito en este capítulo está disponible en el siguiente repositorio: <https://github.com/jfabra/spot>

Tabla 4.2: Requisitos no funcionales del módulo de captura

ID	Descripción del Requisito No Funcional
RNF-REC01	La captura de datos debe ser continua y sin interrupciones, minimizando la pérdida de información.
RNF-REC02	La arquitectura debe permitir la adición de nuevas regiones, zonas de disponibilidad o tipos de instancia sin reestructuraciones mayores.
RNF-REC03	La latencia en la captura y procesamiento de datos debe ser baja para mantener la base de datos actualizada.
RNF-REC04	El módulo debe garantizar la integridad y exactitud de los datos almacenados.
RNF-REC05	El módulo debe proteger el acceso a los datos y las credenciales de AWS.
RNF-REC06	El código y la infraestructura deben ser fáciles de mantener y actualizar.

extracción de datos no son intensivas y su duración está dentro de los límites de Lambda (15 minutos).

EventBridge, un servicio de orquestación sin servidor, gestiona la activación de cada función Lambda. Sus reglas cron permiten una ejecución periódica según la naturaleza de los datos. Las funciones Lambda, escritas en Python, usan el SDK **Boto3** para interactuar con las APIs de AWS, y se despliegan como imágenes de contenedor vía Elastic Container Registry. Para seguridad y mínimo privilegio, cada función Lambda tiene un rol de ejecución con permisos específicos. Tras la obtención, los datos se procesan y almacenan en una base de datos.

Precios Spot

Los precios Spot son dinámicos, variando según la demanda y oferta de AWS. Cada serie de precios Spot se define por la combinación única de **región**, **zona de disponibilidad** (*Availability Zone*), **tipo de instancia** (*Instance Type*) y **sistema operativo** (*Product Description*). La función `describe_spot_price_history` del SDK de AWS se usa para la recolección horaria. Permite filtrar por `StartTime` y `EndTime` (con un límite de 90 días), así como por `InstanceTypes` y `ProductDescriptions` para obtener datos históricos granularmente. Un ejemplo de la traza de los datos de precios Spot obtenidos se muestra a continuación:

```
"SpotPriceHistory": [
  {
    "AvailabilityZone": "us-east-1b",
    "InstanceType": "c6g.medium",
    "ProductDescription": "Linux/UNIX",
    "SpotPrice": "0.011500",
    "Timestamp": "2025-06-20T20:03:06+00:00"
  },
  {
    "AvailabilityZone": "us-east-1d",
    "InstanceType": "c6g.medium",
    "ProductDescription": "Linux/UNIX",
    "SpotPrice": "0.013900",
    "Timestamp": "2025-06-20T17:47:48+00:00"
  },
  ...
]
```

Figura 4.1: Traza de ejemplo de la respuesta de la API `describe_spot_price_history` de AWS.

AWS categoriza precios Spot por Sistema Operativo. Sin embargo, todas las descripciones de producto Linux (UNIX/Linux, Red Hat Enterprise Linux, SUSE Linux, Ubuntu Pro Linux) comparten el mismo precio. Las instancias Windows suelen tener un precio Spot más alto.

Precios bajo demanda (On-Demand)

La recopilación de detalles de instancias se ejecuta condicionalmente tras la obtención de precios Spot. Su objetivo es identificar y complementar metadatos de instancias con precios registrados pero sin detalles en la base de datos. Si se detectan, se consulta la API de AWS para obtener el número de vCPUs, la cantidad de memoria RAM, las arquitecturas de procesador soportadas y el fabricante del procesador. Estos datos enriquecen las características del modelo predictivo, aportando contexto técnico. La modularidad permite añadir campos según las necesidades futuras del modelo.

Evaluación del rendimiento (Benchmarks)

Para complementar la información de las instancias y evaluar el rendimiento por coste, se integran **benchmarks** de Sparecores², una fuente externa que prueba el rendimiento de instancias *cloud*. Esta información es útil para crear planes de aprovisionamiento optimizados, seleccionando instancias con la mejor relación rendimiento/coste. Sparecores usa **stress-ng** con el método **div16** como benchmark principal. Esto cuantifica el rendimiento multi-core, ofreciendo una métrica fiable del rendimiento bruto de la CPU.

Riesgo de expulsión

Se recopila el riesgo de expulsión de instancias de una fuente semi-oficial de AWS³ llamada Spot Advisor. Esta fuente, detalla el riesgo de interrupción de instancias Spot y el ahorro frente al precio bajo demanda, segmentado por región, tipo de instancia y producto, pero sin granularidad por zona de disponibilidad. La fiabilidad de esta fuente es limitada, no se actualiza en tiempo real sino que agrupa variaciones en periodos de varios días, imposibilitando asociar cambios de interrupción a variaciones de precio instantáneas. Pese a estas limitaciones, la información se recopila porque no hay registro histórico oficial y su adquisición es sencilla.

4.1.3. Diseño de la base de datos

Para la persistencia de los datos, se seleccionó una base de datos relacional, **PostgreSQL**, gestionada a través de **Amazon RDS**. Aunque se consideraron alternativas NoSQL, la naturaleza estructurada y fuertemente relacional de los datos (precios, metadatos de instancias, ubicaciones) y la necesidad de realizar consultas analíticas complejas que combinan estas fuentes, hicieron de un modelo relacional la opción más adecuada. Se descartaron servicios especializados en series temporales como Amazon Timestream, ya que el esquema debía almacenar no solo los precios, sino también metadatos de diversa naturaleza, como los resultados de *benchmarks* o las tasas de interrupción, que no se ajustan a dicho modelo.

Amazon RDS se prefirió sobre una instalación autogestionada de PostgreSQL en EC2 por su simplicidad. RDS automatiza tareas operativas (instalación, parches, copias de seguridad, monitoreo,

²<https://sparecores.com/servers?vendor=aws>

³<https://spot-bid-advisor.s3.amazonaws.com/spot-advisor-data.json>

escalabilidad), reduciendo la administración. La instancia específica fue una `t4g.medium` (4GB de RAM), equilibrando coste y rendimiento. No se configuró con Multi-AZ, ya que la alta disponibilidad no es crítica para este proyecto.

Conectividad y Seguridad

La conectividad entre AWS Lambda y RDS se maneja con **Amazon RDS Proxy**. Este *proxy* es clave para reutilizar y multiplexar conexiones, evitando el colapso de la base de datos bajo la carga concurrente de múltiples funciones Lambda. La red utiliza la VPC por defecto de AWS, configurada con un NAT Gateway para conexiones salientes de Lambda a internet. El acceso a la base de datos se controla estrictamente con grupos de seguridad, que permiten tráfico solo desde los grupos de seguridad de las funciones Lambda hacia RDS o RDS Proxy. Las credenciales de la base de datos se gestionan mediante variables de entorno en las funciones Lambda.

Esquema de la base de datos

El esquema de la base de datos organiza la información recopilada. Se compone de las siguientes tablas principales con relaciones definidas:

- **instance_locations**: Tabla de dimensiones que normaliza la información geográfica y técnica de las instancias, evitando redundancia.
- **instance_details**: Contiene metadatos técnicos específicos de cada tipo de instancia, complementando otras tablas.
- **spot_prices**: Tabla central para precios Spot históricos. Almacena precios con granularidad de segundos (`price_timestamp`) y se relaciona con **instance_locations**. Utiliza particionamiento por `price_timestamp` (extensión `pg-partman`) para optimizar el rendimiento.
- **on_demand_prices**: Almacena precios bajo demanda. Su baja volatilidad y ausencia de variación por zona de disponibilidad la convierten en un punto de referencia para el análisis de precios Spot.
- **eviction_notices**: Almacena tasas de expulsión (*eviction risk*) de una fuente semi-oficial. Se mantiene por la ausencia de una fuente oficial persistente y bajo coste de adquisición, aunque su fiabilidad es limitada.

La Figura 4.2 muestra el esquema relacional de la base de datos, detallando las entidades y sus relaciones.

Optimización de consultas

Para mejorar el rendimiento de las consultas, especialmente en series temporales con grandes volúmenes de datos, se han utilizado índices. En la tabla **spot_prices**, se implementó un índice BRIN (*Block Range Index*) sobre la columna `price_timestamp`.

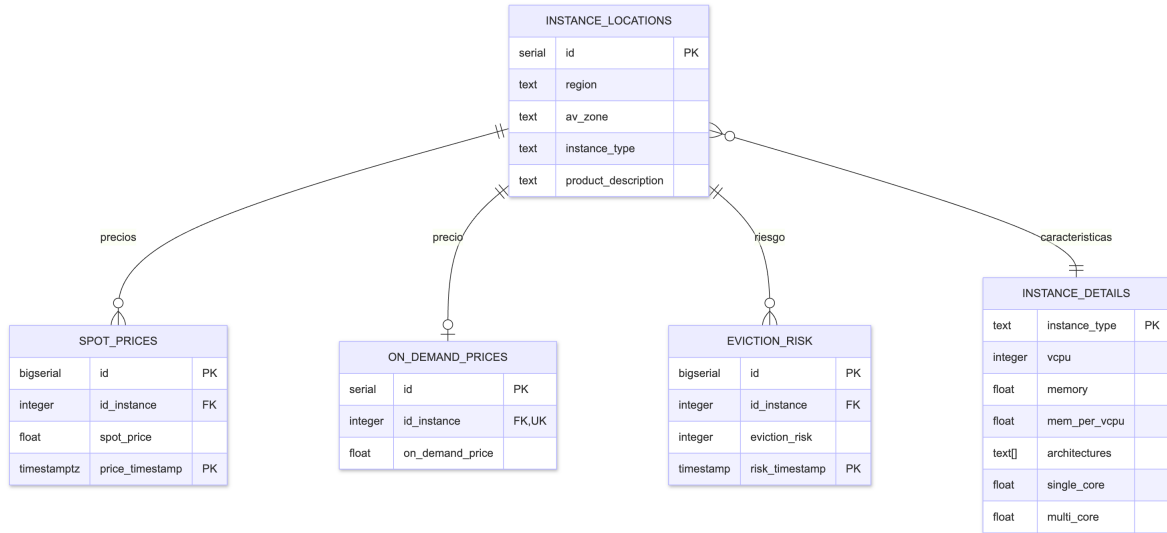


Figura 4.2: Esquema Relacional de la Base de Datos

Los índices BRIN son eficientes para datos ordenados o de series temporales. Almacenan un resumen de los valores mínimos y máximos por bloque de páginas de datos, permitiendo al motor de la base de datos acceder rápidamente solo a los bloques relevantes para una consulta de rango [30].

A diferencia de los índices B-tree, que son de propósito general, los BRIN ofrecen ventajas en tamaño y eficiencia para rangos amplios en series temporales densas, donde los B-tree pueden ser grandes y menos eficientes para escaneos extensos [31]. La elección de BRIN en `price_timestamp` complementa el particionamiento de la tabla, mejorando la respuesta del sistema a las consultas históricas.

Vistas y vistas materializadas

El esquema se complementa con vistas normales y materializadas para facilitar el análisis y optimizar el rendimiento de consultas sobre grandes volúmenes de datos. Estas estructuras pre-calculan o simplifican el acceso a datos agregados y analíticos, reduciendo la carga de procesamiento en el momento de la consulta.

Vistas Consultas guardadas que actúan como tablas virtuales, simplificando el acceso a los datos. Se crearon las siguientes:

- **instance_price_change_velocity**: Calcula la frecuencia y magnitud de los cambios de precios por instancia.
- **region_price_volatility**: Proporciona métricas de volatilidad de precios a nivel de región.
- **last_price_no_av_zone**: Muestra el último precio promedio de instancias por región, tipo y descripción de producto, sin distinguir por zona de disponibilidad, e incluye metadatos técnicos.

Vistas materializadas Almacenan el resultado de una consulta como una tabla física para acelerar el acceso a datos precalculados. Se refrescan automáticamente mediante **trabajos cron** programados en la base de datos con la extensión **pg_cron** para mantener los datos actualizados. Se crearon las siguientes:

- **region_av_zone**: Contiene las combinaciones únicas de región y zona de disponibilidad. Se refresca diariamente.
- **latest_spot_prices**: Guarda el último precio Spot registrado para cada combinación de instancia, región y zona de disponibilidad. Se refresca cada hora.

4.2. Caracterización de los datos

En esta sección se caracterizan los precios Spot, resaltando su heterogeneidad y dinamismo. A partir del conjunto de datos analizado, se observa un promedio de 2366.39 cambios de precio por hora a nivel global. La Figura 4.3 muestra la distribución de estos precios, evidenciando una amplia dispersión de valores que subraya la necesidad de un modelo predictivo capaz de **generalizar entre múltiples series temporales**. Dicha capacidad es clave para optimizar el ajuste de hiperparámetros, reducir la complejidad de mantenimiento y facilitar la escalabilidad del sistema.

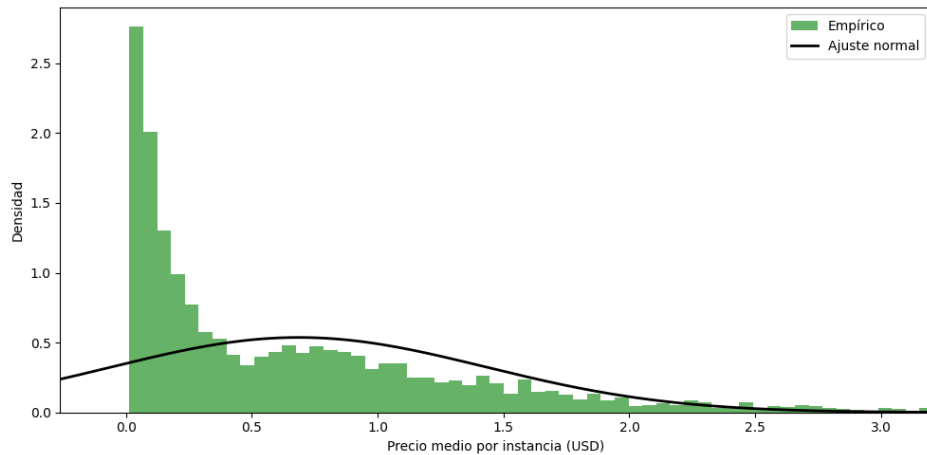


Figura 4.3: Distribución del precio medio de spot AWS por instancia.

4.2.1. Series temporales de precios Spot

Naturaleza y composición

Como ya se ha detallado en la Sección 4.1.2, cada serie temporal corresponde a una combinación única de región, zona de disponibilidad, tipo de instancia y sistema operativo. Cada punto de dato en estas series es el registro de una variación del precio, donde la **marca temporal** indica el momento exacto del cambio con una granularidad de segundos. Aunque cada serie es única, existen patrones subyacentes que vinculan series con características similares, una observación clave para la búsqueda de un modelo generalista.

Desafío de la variabilidad

Variabilidad por instancia/AZ Cada serie temporal exhibe una variabilidad constante, con fluctuaciones cuyos rangos pueden desplazarse significativamente en el tiempo. La Figura 4.4 ilustra este comportamiento, mostrando cómo la predicción de estas dinámicas es fundamental para la optimización de costes, permitiendo la adquisición estratégica de recursos en momentos de precios bajos.

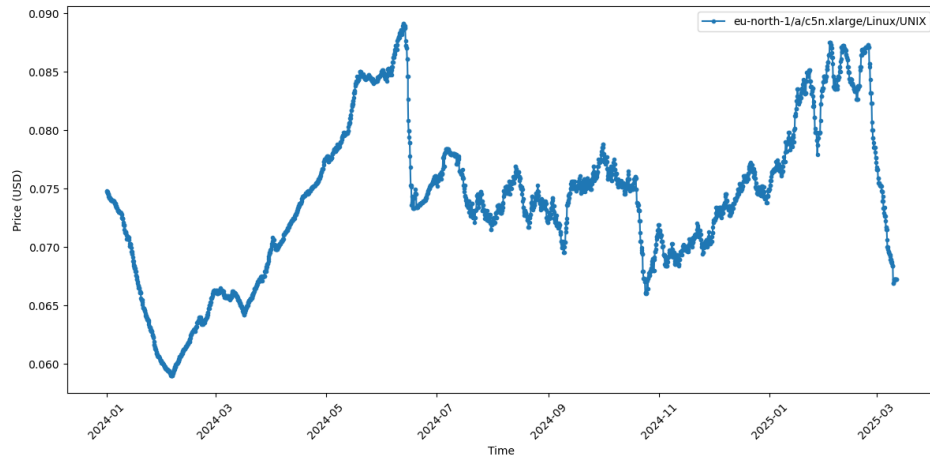


Figura 4.4: Precio de `c5n.xlarge` en `eu-north-1a` desde 2024-01-01 hasta 2025-03-12.

Heterogeneidad por región La complejidad de los precios Spot se acentúa por la heterogeneidad entre regiones, que demuestran una **independencia notable**. La Figura 4.5 ilustra este fenómeno, mostrando cómo instancias idénticas en regiones distintas (`eu-west-1` vs. `us-west-1a`) exhiben dinámicas completamente diferentes. Este hecho refuerza la necesidad de un enfoque de modelado que trate cada región como un mercado independiente.

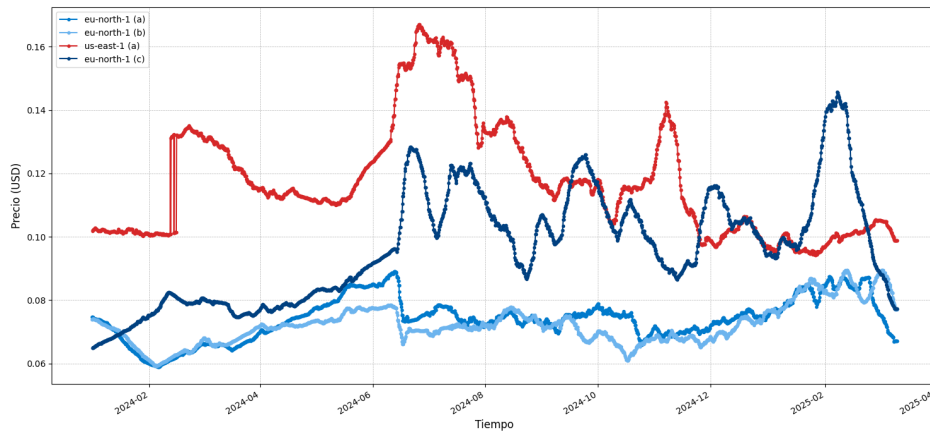


Figura 4.5: Precio de `c5n.xlarge` en distintas regiones y AZ desde 2024-01-01 hasta 2025-03-12.

4.2.2. Dinámica y frecuencia de los cambios

Distribución de la frecuencia

El análisis del tiempo transcurrido entre actualizaciones de precios revela una media de 6.6 horas. La Figura 4.6 ilustra esta distribución, donde se observa un pico principal entre 5 y 8 horas, sugiriendo un ciclo de actualización regular. Esta observación condujo a la decisión de ajustar la granularidad de la predicción, pasando de un pronóstico horario a agrupar los precios en intervalos de varias horas para alinear el modelo con la frecuencia real de los datos.

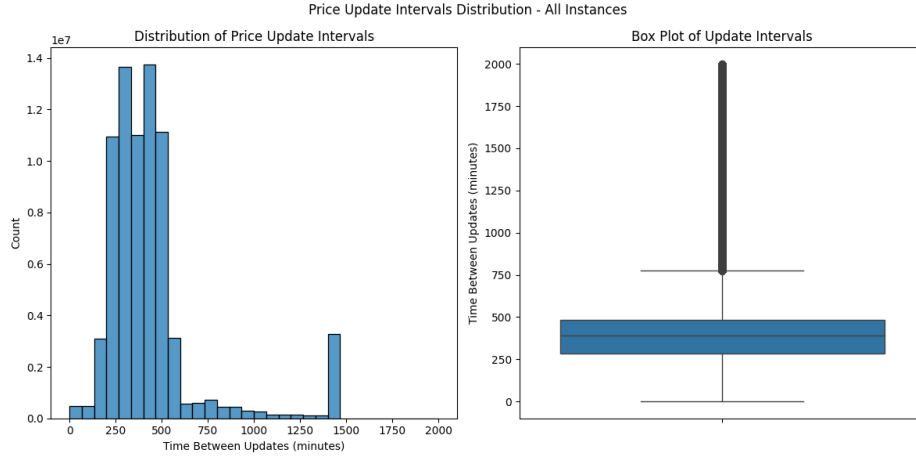


Figura 4.6: Distribución de la frecuencia en la variación de precio.

Volatilidad por región

Para cuantificar de forma objetiva las diferencias de comportamiento entre regiones, se desarrolló un **índice de volatilidad** que consolida indicadores clave como la dispersión y la frecuencia de cambios. El análisis de dicho índice reveló una disparidad significativa entre los mercados de AWS.

El estudio detallado, presentado en el **Anexo C**, muestra que regiones como **ap-southeast-2** presentan la mayor volatilidad, mientras que **us-west-1** es de las más estables. Este análisis cuantitativo fue fundamental para la selección de regiones en la fase de experimentación.

4.2.3. Conclusiones del análisis de datos

El análisis exploratorio ha guiado de manera decisiva el diseño del sistema. Las siguientes conclusiones sintetizan los hallazgos y su impacto en el trabajo:

Heterogeneidad y generalización intra-regional La variabilidad entre regiones confirma la inviabilidad de un modelo global único, fundamentando la estrategia de entrenar un modelo generalista por cada región. Esta aproximación, validada en el Capítulo 6, equilibra especialización y escalabilidad.

Granularidad de predicción y su impacto en el rendimiento El análisis de la frecuencia de cambios de precios motivó la decisión estratégica de adoptar un paso temporal de 12 horas. Como

se demuestra en el Capítulo 6, este ajuste, basado directamente en la dinámica de los datos, mejoró el rendimiento del modelo en un 22.2 %, redujo el tiempo de entrenamiento en un 67 % y mitigó el sobreajuste.

Diseño de los experimentos El análisis de volatilidad, detallado en el Anexo C, identificó a `eu-north-1` como una de las regiones más complejas. Basado en este dato, se seleccionó deliberadamente dicha región como el campo de pruebas principal para asegurar la robustez del modelo. La validación posterior en mercados más estables, como `us-east-1` y `ap-northeast-1`, permitió confirmar la solidez de la metodología.

Capítulo 5

Modelo predictivo de precios

Partiendo de los datos históricos capturados, este capítulo detalla el desarrollo del núcleo técnico del proyecto: el modelo predictivo de precios¹. Se describe el proceso integral de construcción del modelo, que abarca desde el preprocesamiento de los datos, con la generación de secuencias y su normalización, hasta el diseño y la implementación de diversas arquitecturas de Deep Learning. Asimismo, el capítulo cubre el entrenamiento de estos modelos mediante una función de pérdida personalizada y su posterior servitización a través de una API RESTful.

5.1. Diseño basado en configuración

El sistema se fundamenta en un diseño altamente configurable que emplea un archivo `config.yaml` como la fuente de verdad para cada experimento. Este archivo centraliza todos los parámetros del ciclo de vida del modelo, desde la selección y preparación de los datos hasta la definición de la arquitectura y los hiperparámetros de entrenamiento. Este enfoque no solo garantiza la **reproducibilidad** de cada ejecución, sino que también acelera drásticamente el ciclo de experimentación al permitir la modificación de cualquier parámetro sin alterar el código fuente. En el Anexo E se detalla en profundidad tanto este enfoque de diseño como la estructura de los artefactos generados en cada experimento.

5.2. Preprocesamiento y preparación de datos

La preparación de los datos de entrada es crítica para el rendimiento de los modelos predictivos de series temporales. Para este estudio, los datos de precios se estructuran en secuencias temporales y se complementan con características de instancia.

5.2.1. Generación de secuencias temporales

Los modelos de *Deep Learning* para series temporales requieren que los datos se estructuren en secuencias de entrada y salida. La secuencia de entrada comprende un historial de precios, y la de salida los precios futuros a predecir, siendo ambas longitudes configurables. Estas secuencias se generan

¹El código fuente del predictor está disponible en el repositorio: https://github.com/alvdef/spot_predictor

mediante una ventana deslizante para crear múltiples ejemplos de entrenamiento. Las instancias con historial insuficiente se excluyen.

La división de los datos en conjuntos de entrenamiento, validación y prueba se realiza **estrictamente de forma temporal**. Esta separación cronológica previene la fuga de datos, asegurando que la evaluación del modelo refleje su rendimiento en datos futuros no vistos, lo cual es esencial para una estimación realista en producción. Los datos se cargan utilizando **DataLoader** de PyTorch, aplicando aleatorización únicamente en el conjunto de entrenamiento.

5.2.2. Normalización y escalado

La heterogeneidad de los precios entre instancias impide una normalización global efectiva. Por ello, se aplica una **normalización por instancia** para mantener la relevancia de las fluctuaciones en cada serie. La Figura 5.1 ilustra dos instancias de la misma familia y generación que divergen un 694.32 % en la media de sus precios.

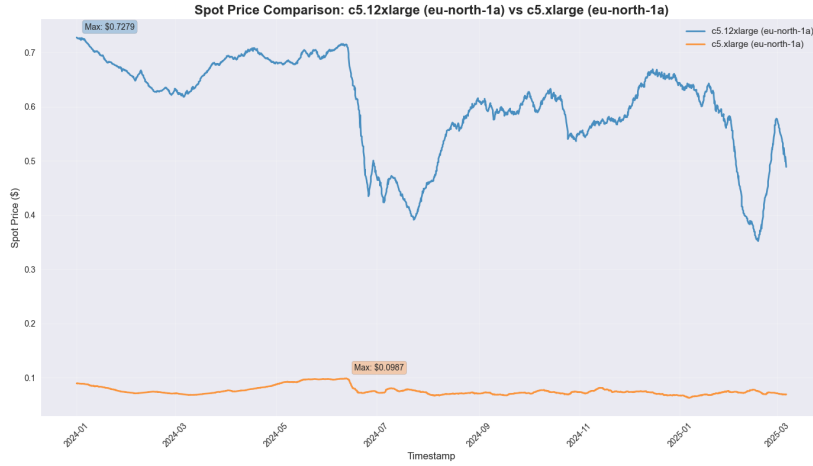


Figura 5.1: Precios de las instancias c5 xlarge y 12xlarge en eu-north-1a

Para cada `id_instance`, se calculan la media (μ) y la desviación estándar (σ) de su serie temporal de precios históricos. Estos parámetros se emplean para normalizar la secuencia de precios de esa instancia, garantizando que las fluctuaciones relativas y los patrones de tendencia mantengan su importancia. La fórmula aplicada es:

$$X_{\text{normalizado}} = \frac{X - \mu}{\sigma}$$

En la inferencia, las predicciones se desnormalizan utilizando los mismos parámetros (μ y σ) de la instancia correspondiente para retornar los precios a su escala original. Para instancias no vistas durante el entrenamiento, se utiliza una estrategia de respaldo: se aplican la media de las medias y la media de las desviaciones estándar de todas las instancias conocidas para la normalización, es decir:

$$X_{\text{normalizado}} = \frac{X - \bar{\mu}}{\bar{\sigma}}$$

5.2.3. Inclusión de características adicionales de la instancia

Además de los precios históricos, el modelo incorpora características estáticas de la instancia objetivo. Estas incluyen la generación, el tamaño, la familia y la arquitectura del procesador. Estas características son fundamentales para contextualizar la predicción, ya que diferentes tipos de instancias exhiben comportamientos de precios variados.

5.2.4. Características temporales

Se ha considerado la inclusión de características temporales adicionales (ej., día de la semana, día del mes), codificadas con funciones sinusoidales para mantener su naturaleza cíclica. Sin embargo, esta funcionalidad no se integra en la implementación actual del modelo predictivo. Su inclusión se pospone como trabajo futuro, con la expectativa de que mejore la precisión al capturar patrones temporales recurrentes.

5.3. Diseño de las arquitecturas

Se exploraron diversas arquitecturas especializadas en series temporales, cuyas bases teóricas y funcionamiento general fueron establecidos en las Secciones A.2 y A.3 del Anexo A. La selección y el diseño de estas arquitecturas se realizaron con el objetivo de capturar tanto las dependencias temporales intrínsecas de los precios como la influencia de características exógenas. A continuación, se detalla la configuración específica de los modelos implementados, justificando las decisiones de ingeniería adoptadas.

5.3.1. Modelos base

Se implementaron arquitecturas basadas en Redes Neuronales Recurrentes (RNN), específicamente con unidades LSTM y GRU, detalladas en el Anexo A.2, como enfoques *baseline*.

Arquitectura La arquitectura consiste en una o varias capas recurrentes (`nn.LSTM` o `nn.GRU`). La predicción de un horizonte fijo se obtiene mediante una capa lineal (`nn.Linear`) que proyecta directamente el último estado oculto de la capa recurrente a dicho número de pasos futuros. La Figura 5.2 ilustra la arquitectura del enfoque *baseline*.



Figura 5.2: Arquitectura del enfoque básico

Decisiones de Ingeniería Estos modelos se eligieron por su relativa simplicidad y eficiencia computacional, sirviendo como puntos de comparación. La predicción directa multi-paso simplifica la inferencia al obtener parte del horizonte de pronóstico en una única operación.

5.3.2. Modelo Sequence-to-Sequence

Esta arquitectura se presenta como una mejora del modelo anterior, cambiando la arquitectura de las RNNs y añadiendo un mecanismo de atención.

Arquitectura El modelo se construye con un codificador, un decodificador y un mecanismo de **MultiheadAttention**. El codificador procesa la secuencia para generar una representación contextual, sobre la cual el mecanismo de atención destaca las partes más relevantes. Posteriormente, el decodificador, inicializado con la salida del mecanismo de atención, genera la secuencia de predicciones utilizando la salida en un instante como parte de la entrada para el siguiente. En la Figura 5.3 se observa lo descrito.

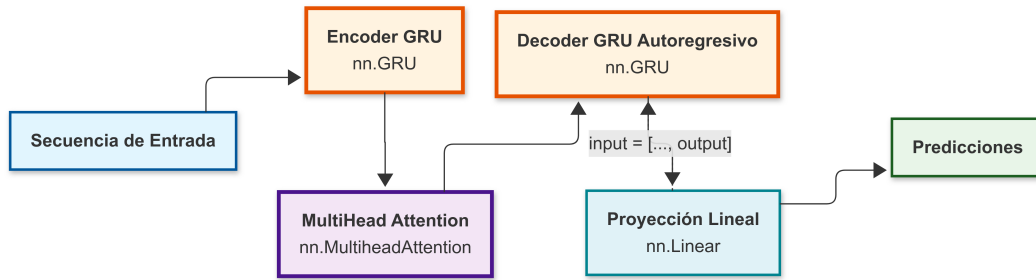


Figura 5.3: Arquitectura del modelo Seq2Seq

Decisiones de ingeniería En cuanto a las decisiones de ingeniería, se optó por emplear redes GRU tanto en el codificador como en el decodificador, en lugar de LSTM. Esta elección se fundamenta en la menor complejidad de las GRU, detallada en el Anexo A.2.2, lo que permite mantener una carga computacional comparable al enfoque *baseline* en los modelos Seq2Seq, a pesar del incremento de complejidad que estos conllevan. Por otro lado, la inclusión de *MultiheadAttention* se inspiró en su aplicación en tareas de Procesamiento del Lenguaje Natural [?], permitiendo al modelo focalizarse en la información más relevante.

5.3.3. Modelo Sequence-to-Sequence con características

Esta arquitectura evoluciona el modelo Seq2Seq al integrar atributos estáticos de las instancias. El objetivo es que el modelo aprenda patrones particulares de las series a predecir.

Arquitectura El modelo FeatureSeq2Seq mantiene la arquitectura de Seq2Seq y añade una subred de características de instancia que procesa los atributos estáticos de la instancia. Posteriormente, una subred de integración de contexto combina la salida del codificador de precios con una representación expandida de estas características de instancia. La salida de esta integración sirve como la entrada inicial para la primera predicción del decodificador, permitiendo que las propiedades de la instancia influyan directamente en la generación de toda la secuencia de precios futura. El estado oculto inicial

del decodificador se basa directamente en el estado final del codificador. En la Figura 5.4 se observa lo descrito.

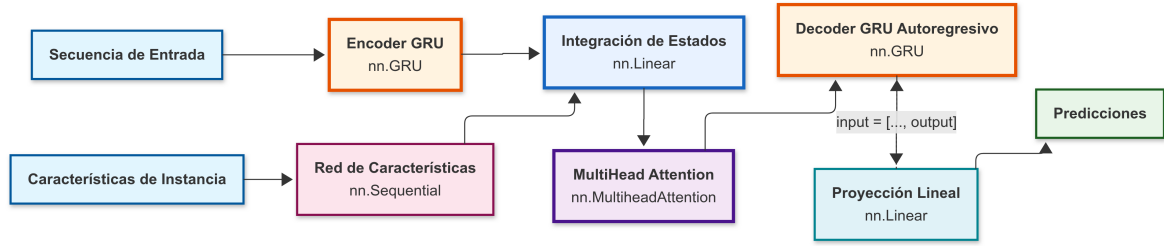


Figura 5.4: Arquitectura del modelo FeatureSeq2Seq

Decisiones de ingeniería Una subred fusiona la salida del codificador con las características contextuales de la instancia, previamente procesadas. El resultado de esta fusión actúa como la *query* para el mecanismo de atención, que refina el contexto proporcionado al decodificador. Esta arquitectura asegura que tanto la información de la secuencia de entrada como las características adicionales influyan en cada paso de la predicción de la secuencia de salida. Una alternativa evaluada habría sido inicializar el estado oculto del decodificador directamente con estas características, pero tras obtener peores resultados en pruebas preliminares, se escogió la primera opción.

5.4. Función de pérdida

La función de pérdida guía el proceso de aprendizaje del modelo, indicándole cómo ajustar sus pesos para minimizar el error. En el contexto de la predicción de precios de instancias spot, es fundamental no solo minimizar la diferencia en la magnitud de la predicción, sino también capturar con precisión la dirección de los cambios de precio. Para lograr este doble objetivo, se implementó **TrendFocusLoss**, una función de pérdida personalizada.

La inclusión de un término de pérdida centrado en la tendencia alinea el entrenamiento del modelo con un objetivo de negocio crítico: identificar si los precios van a subir o bajar. Esto es prioritario, ya que la ejecución de una tarea en el momento presente es siempre preferible a la demora debido a la incertidumbre asociada a las futuras fluctuaciones de precios. La función de pérdida asigna un coste específico a la predicción errónea de la dirección del cambio. Para ello, combina dos componentes principales mediante una ponderación configurable, permitiendo ajustar el balance entre la precisión de la magnitud y la dirección:

- **Error Cuadrático Medio (MSE):** Definida en el Anexo A, penaliza la diferencia en la magnitud entre las predicciones y los valores reales. Su uso asegura que el modelo se esfuerce por reducir el error absoluto de la predicción de precios.
- **Pérdida de tendencia:** Definida en el Anexo B, penaliza los errores en la dirección del cambio de precios, si estos son suficientemente significativos. Se emplea una versión diferenciable de

esta pérdida para permitir el cálculo de gradientes y su integración efectiva en el proceso de optimización del modelo.

5.5. Servitización del modelo

Para hacer accesible el modelo de predicción de precios y facilitar la integración se ha diseñado una API RESTful. Esta sección detalla la estrategia de inferencia del modelo y el diseño de la API para su consumo.

5.5.1. Generación de pronósticos (inferencia)

La predicción de precios requiere pronosticar múltiples valores a lo largo de un horizonte temporal, conocido como **predicción multi-step**. El sistema aborda esto combinando la capacidad de predicción directa de los modelos entrenados con un proceso autorregresivo a nivel de sistema.

Como se introdujo en el Anexo A, se distinguen enfoques directos e iterativos. En este trabajo, el modelo de *Deep Learning* realiza una predicción multi-step directa para un número fijo de pasos, mientras que el sistema usa un proceso autorregresivo para extender el pronóstico al horizonte deseado.

Capacidad de predicción directa del modelo

Cada modelo está entrenado para generar una predicción directa para una ventana temporal fija configurada durante el entrenamiento. Por ejemplo, si `model_pred_days` fuera 2 días y `timestep_hours` 12h, el modelo predice los precios para los siguientes 4 *timesteps*. El modelo utiliza su estado interno para producir este conjunto inicial de pronósticos en una única llamada a su método `forward`. El modelo no es consciente de horizontes de predicción más allá.

Extensión del pronóstico mediante autorregresión

Dado que el horizonte es limitado y no cubre todos los valores deseados (ej., 20 días para RF01), se implementa un proceso autorregresivo a través del método `forecast`, para extender el pronóstico recursivamente:

1. Se inicia con la secuencia de datos históricos reales.
2. Se invoca repetidamente el método `forward` del modelo para obtener bloques de predicciones.
3. Las predicciones generadas se reintroducen como parte de la secuencia de entrada para la siguiente predicción, autoalimentando el modelo con sus propios pronósticos.
4. Se calculan las características temporales futuras correspondientes a los nuevos timesteps.
5. Este proceso se repite hasta alcanzar el número total de pasos (`n_steps`) deseado.

Esta estrategia permite generar pronósticos para cualquier horizonte temporal sin reentrenar el modelo. No obstante, la naturaleza autorregresiva acumula errores a medida que el horizonte de predicción se extiende. La Figura 5.5 ilustra el proceso explicado en esta Sección.

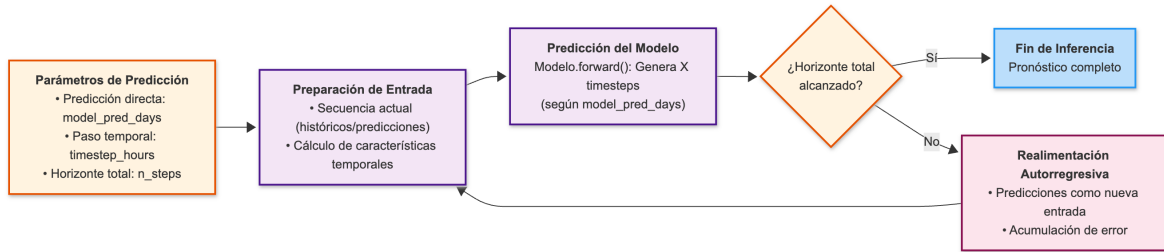


Figura 5.5: Enfoque híbrido

5.5.2. Diseño del servicio

Utilizando el *framework* **FastAPI**, se ha desarrollado un servicio web que expone las capacidades predictivas. El diseño de la API se centra en un *endpoint* principal, que recibe los parámetros necesarios para realizar una predicción, como la región, la zona de disponibilidad, el tipo de instancia, el sistema operativo y el número de días de historial a considerar para la predicción. La API se encarga internamente de:

1. **Obtener los datos de entrada** desde la base de datos y AWS.
2. **Preprocesar y normalizar** los datos según los mismos criterios utilizados durante el entrenamiento del modelo.
3. **Cargar el modelo** de entrenado en memoria.
4. **Inferir los valores** futuros mediante el proceso descrito en la Sección 5.5.1.
5. **Devolver las predicciones** junto con los parámetros de entrada y otros metadatos.

Los servicios de la API fueron definidos con Swagger, lo que facilita la documentación interactiva y el testeo. La Figura 5.6 muestra una captura de la interfaz de Swagger.

prediction

POST /predict/ Predict

Fetch recent spot price history, process features, and forecast future prices.

Parameters Try it out

No parameters

Request body required application/json

Example Value Schema

```

{
  "region": "eu-north-1",
  "av_zone": "a",
  "instance_type": "c5.xlarge",
  "os": "Linux/UNIX",
  "days": 1
}

```

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema <pre> { "predictions": [0], "input_parameters": {}, "processing_time": 0, "min_spot_price": 0, "min_spot_timestamp": "2025-06-27T05:29:03.978Z" } </pre>	No links
422	Validation Error Media type application/json Example Value Schema <pre> { "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string" }] } </pre>	No links

GET /predict/health Health Check

Figura 5.6: Interfaz del metodo de predicci3n en Swagger de la API

Capítulo 6

Experimentación y resultados

La validación empírica del sistema es el eje central de este capítulo, donde se evalúa el rendimiento de los modelos predictivos para seleccionar la arquitectura más eficaz. Se presenta el diseño sistemático de los experimentos, que incluye la configuración del entorno de ejecución, la preparación de los conjuntos de datos y la definición de las métricas de evaluación, con un énfasis particular en los indicadores de negocio. El capítulo también detalla el proceso iterativo de búsqueda de hiperparámetros y concluye con un análisis exhaustivo de los resultados obtenidos, que fundamenta la selección y justificación del modelo finalista.

6.1. Entorno de ejecución y configuración

La fase de experimentación y entrenamiento de los modelos se llevó a cabo en una instancia `g4dn.2xlarge`, configurada para cargas de trabajo de *Deep Learning*. Esta instancia cuenta con 8 vCPU, 32 GiB de RAM y una GPU NVIDIA Tesla T4 con 14.56 GB de VRAM. La instancia también dispuso de un SSD NVMe de 225 GB para almacenar los artefactos del modelo, como archivos de configuración, puntos de control del entrenamiento (*checkpoints*), registros de ejecución y resultados de evaluación.

Se utilizó la Amazon Machine Image (AMI) preconfigurada `Deep Learning OSS Nvidia Driver AMI GPU PyTorch 2.6.0 (Amazon Linux 2023) 20250406` para simplificar la gestión de dependencias. El entorno de *software* incluyó `Python 3.12`, el *framework* de aprendizaje automático `PyTorch 2.6.0`. Las dependencias se gestionaron mediante `pip`, instalándose directamente sobre el entorno base de la AMI.

6.2. Diseño de los experimentos

Se adoptó un enfoque sistemático de experimentación y toma de decisiones, progresando desde una fase exploratoria inicial hasta el entrenamiento de modelos definitivos con parámetros optimizados.

6.2.1. Selección y preparación del conjunto de datos

Estrategia de división temporal

La división de los datos se realizó de manera estrictamente **cronológica** para simular un escenario de producción y evitar el *data leakage*. Los períodos de los conjuntos fueron:

- **Entrenamiento:** Del 1 de enero de 2024 al 29 de diciembre de 2024 (363 días).
- **Validación:** Del 29 de diciembre de 2024 al 6 de marzo de 2025 (67 días).
- **Prueba:** Del 6 de marzo de 2025 al 21 de abril de 2025 (45 días).

Selección de la región de experimentación principal

Los modelos predictivos fueron diseñados para ser **entrenados de forma regional**. Esta decisión se debe a que las series de precios en una misma región suelen compartir patrones comunes de comportamiento, influenciados por factores geográficos, de demanda local y de capacidad de infraestructura. Esta aproximación de entrenamiento regional también permite agrupar un número considerable de series temporales dentro de un mismo modelo, lo que resulta en un entrenamiento más robusto y una mejor generalización para las instancias dentro de esa región.

Se seleccionó la región **eu-north-1 (Estocolmo)** como el campo de pruebas principal. La elección de esta región se fundamenta en los hallazgos del análisis cuantitativo presentado previamente en este trabajo. Como se estableció en la caracterización de datos de la Sección 4.2 y se detalla en el Anexo C, el análisis de volatilidad identificó a **eu-north-1** como una de las regiones más complejas y con mayor dinámica de precios. La Tabla 6.1 corrobora esta evaluación, posicionándola con un alto índice de volatilidad (Vol. Rank 5).

La decisión de centrarse en un **entorno de alta volatilidad** es deliberada: desarrollar y validar el modelo en un escenario adverso asegura su robustez y capacidad de adaptación. Un modelo que demuestra un rendimiento sólido en condiciones exigentes garantiza una mayor fiabilidad y precisión en regiones con dinámicas de mercado más estables. Adicionalmente, y desde una perspectiva práctica, la proximidad geográfica de **eu-north-1** a la ubicación de desarrollo (Zaragoza) la convierte en una opción pragmática para la aplicación de la solución.

Tabla 6.1: Resumen de volatilidad de precios por región

Región	Vol. Rank	Vol. Score	Índice de Variación	Cambios/Instancia
ap-southeast-2	1	0.901	2.070	263.90
eu-north-1	5	0.734	1.619	245.71
ap-northeast-1	10	0.668	1.430	317.34
us-east-1	14	0.649	1.535	195.45
us-west-1	17	0.563	1.446	148.11

Nota: Los datos completos y las fórmulas se encuentran en el Anexo C

Criterios de filtrado y normalización de datos

Para estandarizar el conjunto de datos y concentrar la experimentación en tipos de instancias relevantes, se aplicaron los siguientes filtros:

- `product_description`: ['Linux/UNIX']
- `architectures`: ['x86_64']
- `instance_family`: ['c'] (instancias optimizadas para cómputo)
- `metal`: False

La elección de estos filtros se alinea directamente con el dominio de aplicación del proyecto. Como se ha establecido a lo largo de este trabajo, el objetivo principal es la optimización de costes en cargas de trabajo de procesamiento por lotes (*batch processing*). Este escenario, caracterizado por su tolerancia a interrupciones, convierte a las instancias Spot en una alternativa idónea. En este contexto, las instancias de la familia `c`, optimizadas para cómputo, son una elección natural y eficiente para este tipo de tareas. La selección de la arquitectura `x86_64` y el sistema operativo `Linux/UNIX` responde a que representan una de las configuraciones más estandarizadas y ampliamente utilizadas para estas cargas de trabajo de propósito general. Finalmente, se excluyeron las instancias `metal` porque sus patrones de precios son muy distintos a los de las máquinas virtuales convencionales, y su inclusión habría desviado significativamente las predicciones del modelo base.

La media (μ) y la desviación estándar (σ) necesarias para la **normalización de datos** se implementaron **por instancia**, obteniéndose del conjunto de entrenamiento. La media global resultante fue 0.5032 y la desviación estándar global 0.1037. En cuanto a las características de instancia, los modelos utilizaron `generation`, `modifiers` y `size`.

6.2.2. Estrategias y métricas de evaluación

Para la evaluación del modelo se hace **validación cruzada cronológica con ventana deslizante**. Este enfoque consiste en entrenar el modelo con datos históricos y evaluarlo sobre un conjunto de datos futuros y no vistos. Para obtener una medida de rendimiento fiable, la ventana de evaluación se desliza progresivamente a través del conjunto de test (por ejemplo, avanzando un día cada vez). Este método genera múltiples escenarios de prueba, lo que reduce la dependencia de un único punto de inicio y ofrece una estimación más robusta y generalizable del rendimiento del modelo. El resultado final de cada métrica se obtiene promediando los valores calculados en cada una de las ventanas de evaluación.

Para medir el rendimiento, se emplearon tanto métricas de precisión estándar (detalladas en la Sección 2.2.2), como un conjunto de indicadores clave de negocio (KPIs) diseñados para cuantificar el valor práctico y financiero de las predicciones del modelo. Las métricas diseñadas son:

- **Precisión de tendencia:** Mide la capacidad del modelo para predecir correctamente la dirección (subida o bajada) de los cambios de precio que superan un umbral de relevancia.
- **Ahorro de coste:** Cuantifica el beneficio financiero porcentual que se obtendría al utilizar las predicciones del modelo para decidir el momento óptimo de aprovisionamiento de una instancia, en comparación con una ejecución inmediata.
- **Ahorro con información perfecta:** Funciona como un límite superior teórico, calculando el máximo ahorro posible si se conocieran de antemano todos los precios futuros en una ventana de decisión.
- **Eficiencia del ahorro:** Evalúa la efectividad del modelo midiendo la proporción del ahorro potencial que es capaz de capturar, comparando el ahorro logrado por el modelo con el ahorro de información perfecta.

En el Anexo D se presenta una descripción detallada de cada uno de estos indicadores, incluyendo sus fórmulas matemáticas y la lógica subyacente de su cálculo.

6.2.3. Herramienta de visualización para la evaluación de resultados

Para facilitar el análisis y la evaluación, se desarrolló un *dashboard* interactivo. Esta herramienta, implementada con `React`[32] y la librería `Recharts`[33], es un componente fundamental para la evaluación y selección de modelos, ya que transforma métricas complejas en visualizaciones que permiten una validación rigurosa del rendimiento. Su diseño se centra en tres capacidades clave para el análisis experimental:

- **Análisis comparativo y granular:** La herramienta permite la carga y comparación simultánea de múltiples modelos. Mediante un sistema de filtrado dinámico, es posible segmentar los resultados por dimensiones como el tipo de instancia, la región o el horizonte temporal; de gran importancia para validar los modelos en escenarios específicos e identificar nichos de rendimiento.
- **Correlación entre configuración y rendimiento:** Muestra una comparativa de las configuraciones (`config.yaml`) de dos modelos, resaltando las diferencias. Esto permite correlacionar directamente cambios en los hiperparámetros con las variaciones observadas en el rendimiento, agilizando el proceso iterativo de ajuste.
- **Análisis multidimensional de resultados:** El *dashboard* ofrece múltiples perspectivas para evaluar el comportamiento del modelo, entre otras:
 - **Evolución del error en el horizonte temporal:** Gráficos que muestran la degradación del MAPE a medida que se avanza en el horizonte de predicción.
 - **Distribución de errores:** Histogramas que clasifican las predicciones en rangos de precisión, detallando más allá del error promedio.

- **Métricas de impacto de negocio:** Visualizaciones como la eficiencia de ahorro, que compara el ahorro de costes real obtenido por el modelo frente al ahorro teórico máximo posible.

En el Anexo G se presenta una descripción de la herramienta, junto con un catálogo completo de todas las funcionalidades y visualizaciones implementadas.

6.3. Proceso de búsqueda de hiperparámetros

Durante la experimentación, se evaluaron diversas arquitecturas de modelos de *Deep Learning* especializadas en series temporales. Una descripción detallada de los modelos evaluados se encuentra en el Anexo A y su implementación específica en la Sección 5.3. La optimización de los modelos se realizó mediante un proceso iterativo de *grid search*, con el objetivo principal de minimizar el MAPE global. En el Anexo I se encuentra una traza de ejecución del entrenamiento de un modelo.

6.3.1. Parámetros de entrenamiento comunes

Los parámetros de entrenamiento comunes a todas las ejecuciones de los modelos incluyeron el optimizador `AdamW`, una tasa de aprendizaje inicial de 0.0001. Se utilizó un programador de tasas de aprendizaje que ajustaba este valor hasta un mínimo de `2e-06`, `pct_start=0.2` de y un `div_factor=10`. Adicionalmente, se aplicó un `weight_decay=5e-05` para la regularización L2. El tamaño del *batch* se configuró en 128 para el entrenamiento y 32 para la evaluación. Respecto a la arquitectura, todos los modelos definitivos emplearon 2 capas en sus RNNs y una tasa de dropout de 0.2.

6.3.2. Fase exploratoria

La fase inicial de experimentación consistió en una búsqueda exploratoria rápida, entrenando modelos hasta un máximo de **20 epochs** con una paciencia de 3 épocas. El tamaño de la capa oculta en estos experimentos fue 64. El objetivo fue identificar configuraciones de hiperparámetros prometedoras en rendimiento y eficiencia. Para esta exploración, se evaluaron **dos arquitecturas** principales: un modelo *baseline* recurrente y arquitecturas Sequence-to-Sequence (Seq2Seq) con atención. Para el **modelo baseline** se decidió usar celdas LSTM ya que mostraron un rendimiento similar, en ocasiones superior, a las GRUs y se esperaba que su mayor complejidad y capacidad teórica permitiera capturar de manera más efectiva las dependencias a largo plazo. Por otro lado, en el **modelo Seq2Seq** se justificó el uso de celdas GRU en la Sección 5.3.2.

Para estas celdas y configuraciones, se exploraron distintas longitudes de secuencia de entrada, horizontes de predicción y granos horarios con el fin de optimizar la representación temporal de los datos y la capacidad predictiva de los modelos. En los análisis presentados, `MAPE_0.5` se refiere al Error Porcentual Absoluto Medio (MAPE) promedio calculado sobre los primeros 5 días del horizonte de

predicción, mientras que `MAPE_5_20` corresponde al MAPE promedio para el rango que abarca desde el día 5 hasta el día 20 de la predicción.

Longitud de la secuencia de entrada (`sequence_length`)

Se evaluaron longitudes de secuencia de 10, 20 y 30 días. La **longitud de secuencia de 20 días** (equivalente a 40 timesteps con un `timestep_hours` de 12 horas) fue la opción seleccionada. Se determinó que un aumento a 30 días no ofrecía mejoras significativas en el rendimiento, mientras que incrementaba la complejidad computacional. Esta longitud también se consideró coherente con el horizonte de predicción esperado para la aplicación.

Grano horario (`timestep_hours`)

La elección del grano horario es fundamental para equilibrar el detalle temporal y la eficiencia. Basado en el análisis de datos previo (Sección 4.2), se hipotetizó que una granularidad demasiado fina podría ser contraproducente dado que la frecuencia media de cambio de precios se sitúa en torno a las 6.6 horas. Para validar esto, se evaluaron configuraciones de 4, 6 y 12 horas. La Tabla 6.2 resume los resultados agregados.

Tabla 6.2: Comparativa de rendimiento por grano horario (`timestep_hours`).

<code>timestep_hours</code>	MAPE 0-4 (%)	MAPE 5-20 (%)	Tiempo Entr. (h)	Overfitting Ratio
4h	29.65	35.22	3.30	2.29
6h	27.96	32.18	1.48	1.94
12h	22.00	28.79	1.14	1.45

Los datos confirman que un grano horario de 12 horas es superior. El MAPE a corto plazo mejora en un **25.8 %** frente a la configuración de 4h, ya que se filtra el ruido de alta frecuencia que dificulta el aprendizaje. Adicionalmente, el sobreajuste se reduce notablemente (ratio de 2.29 a 1.45), indicando una mejor generalización, y el tiempo de entrenamiento disminuye en un **65.5 %**, optimizando drásticamente el coste computacional. Por tanto, se seleccionó un **grano horario de 12 horas** para las fases posteriores de experimentación por ofrecer el mejor compromiso entre precisión, generalización y eficiencia.

Longitud de la predicción en el entrenamiento (`model_pred_days`)

Otro hiperparámetro crucial es la longitud del horizonte (`tr_prediction_length`) que el modelo aprende a predecir directamente. Se calcula en base a `model_pred_days`, multiplicándolo por `timestep_hours` para obtener el número de puntos a predecir (`tr_prediction_length`). Un horizonte más corto simplifica la tarea de aprendizaje, pero podría limitar la capacidad del modelo para capturar dinámicas a más largo plazo. Por el contrario, un horizonte más largo podría forzar al modelo a aprender dependencias más complejas, pero a riesgo de un mayor error y coste computacional. Se evaluaron horizontes de 2, 3, 4 y 5 días. La Tabla 6.3 muestra los resultados.

Tabla 6.3: Comparativa de rendimiento por longitud de predicción en el entrenamiento.

model_pred_days	MAPE 0-4 (%)	MAPE 5-20 (%)	Tiempo Entr. (h)	Overfitting Ratio
2	28.69	32.83	1.20	1.66
3	26.52	30.32	1.49	1.62
4	24.12	30.78	2.66	1.99
5	24.67	33.55	3.33	2.54

El análisis de los datos indica que la relación no es lineal. Mientras que un horizonte de 2 días es rápido de entrenar, su error es el más alto. Aumentar el horizonte a 3 y 4 días mejora progresivamente la precisión a corto plazo (MAPE_0.4). La configuración de **4 días de predicción** emerge como el punto óptimo. Alcanza el MAPE a corto plazo más bajo (24.12 %) sin sacrificar el rendimiento a largo plazo, manteniendo un MAPE_5.20 (30.78 %) competitivo y mejor que el de horizontes más cortos o más largos. Incrementar la longitud a 5 días degrada el rendimiento a largo plazo e incrementa el sobreajuste, sugiriendo que la tarea de aprendizaje se vuelve demasiado compleja.

La Figura 6.1 ilustra de forma contundente el impacto de esta decisión, comparando dos modelos idénticos salvo por este hiperparámetro (2 y 4 días). El modelo entrenado para predecir 4 días demuestra una precisión notablemente superior. En la ventana inicial, su MAPE se mantiene consistentemente por debajo del 17.5 %, mientras que el modelo de 2 días sufre una degradación abrupta y muy pronunciada tras sus 4 pasos iniciales, superando el 30 % de error.

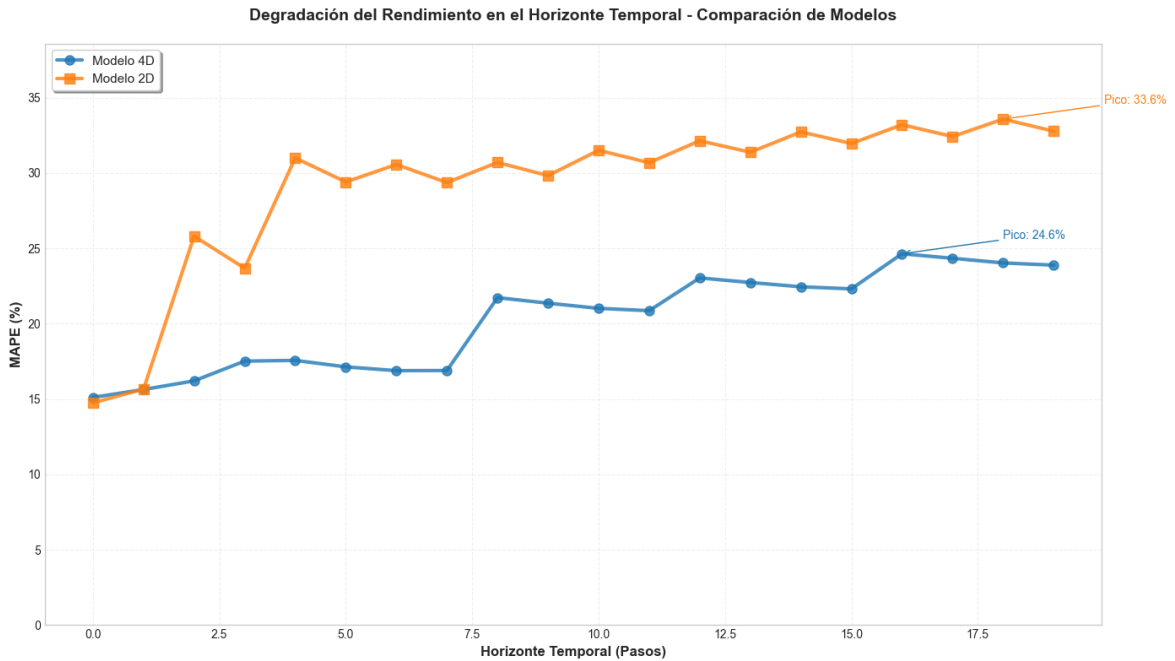


Figura 6.1: Degradación del rendimiento para modelos con `tr_prediction_length` de 2 días y 4 días

Es crucial observar el salto en el error que experimenta el modelo de 4 días a partir del cuarto día. Este punto marca el final de su capacidad de predicción directa, momento en el que el sistema comienza a emplear el **proceso autorregresivo** descrito en la Sección 5.5.1 para extender el pronóstico.

La acumulación de errores inherente a esta técnica explica el incremento del MAPE en los pasos posteriores. En consecuencia, se seleccionó una longitud de predicción de 4 días para la fase de refinamiento.

6.3.3. Fase de refinamiento

Tras la fase exploratoria, se procedió a una fase de refinamiento y entrenamiento de los modelos definitivos. Los modelos seleccionados, particularmente la arquitectura **FeatureSeq2Seq**, se entrenaron con un máximo de **40 *epochs*** y un *early stopping patience* que se incrementó a **5 épocas** para permitir una convergencia más estable. Este proceso implicó un *grid search* más focalizado, utilizando las combinaciones de hiperparámetros que se mostraron más prometedoras, específicamente un paso de tiempo de 12 horas y una longitud de predicción del modelo de 4 días.

Impacto del tamaño de la capa oculta (`hidden_size`)

El tamaño de la capa oculta define la capacidad del modelo para aprender patrones complejos. En esta fase, se evaluó si un aumento en la capacidad del modelo de 128 a 256 neuronas se traduciría en una mejora del rendimiento. El objetivo era encontrar el punto óptimo entre la complejidad del modelo y su precisión. Los resultados de esta comparativa se resumen en la Tabla 6.6.

Tabla 6.4: Comparativa de rendimiento por tamaño de capa oculta (`hidden_size`).

<code>hidden_size</code>	MAPE 0-4 (%)	MAPE 5-20 (%)	Tiempo (h)	Entr.	Overfitting Ratio
128	19.58	24.76	1.02		0.62
256	16.23	21.56	1.04		0.64

Observamos que duplicar el tamaño de la capa oculta de 128 a 256 neuronas produce una **mejora sustancial y generalizada del rendimiento**. El error a corto plazo (`MAPE_0_4`) se reduce de 19.58 % a 16.23 %, lo que representa una **mejora relativa del 17.1 %**. De manera aún más significativa, el rendimiento a largo plazo (`MAPE_5_20`) también mejora, descendiendo de 24.76 % a 21.56 %. Es importante destacar que esta ganancia en precisión se consigue sin un impacto negativo relevante en otros indicadores. El tiempo de entrenamiento apenas se incrementa y el ratio de sobreajuste se mantiene en un nivel excelente (0.64), lo que demuestra que el modelo más grande generaliza de forma muy eficaz. Por lo tanto, se seleccionó un **tamaño de capa oculta de 256 neuronas** como la configuración superior, ya que proporciona una reducción significativa del error de predicción en todos los horizontes evaluados sin introducir sobrecostes computacionales o problemas de sobreajuste.

Impacto de los pesos de la función de pérdida (`mse_weight`)

La función de pérdida personalizada **TrendFocusLoss**, detallada en la Sección 5.4, fue diseñada para balancear dos objetivos: la precisión en la magnitud del precio (componente MSE) y la correcta predicción de la dirección de su cambio (componente de tendencia). El hiperparámetro `mse_weight` controla esta ponderación, donde un valor de 1.0 equivale a un MSE puro y un valor menor da más

importancia al componente de tendencia. En la fase de refinamiento se evaluaron dos configuraciones, `mse_weight=0.5` y `mse_weight=0.7`, para determinar si un mayor enfoque en la tendencia podría mejorar los resultados de negocio. Los resultados se presentan en la Tabla 6.5.

Tabla 6.5: Comparativa de rendimiento por peso de la función de pérdida (`mse_weight`).

<code>mse_weight</code>	MAPE 0-4 (%)	MAPE 5-20 (%)	Eficiencia Ahorro (%)	Precisión Tendencia (%)
0.5	17.97	23.40	9.7	51.4
0.7	17.84	22.92	14.5	52.9

Los resultados muestran un impacto casi nulo sobre el MAPE. Aunque el peso de 0.7 ofrece una ligera ventaja en las métricas de negocio, la escasa sensibilidad del modelo a este hiperparámetro sugiere que la implementación de la pérdida de tendencia no fue efectiva. Ante esta limitada influencia y para asegurar la máxima robustez, se decidió simplificar y utilizar exclusivamente el Error Cuadrático Medio (MSE), estableciendo `mse_weight=1.0` en el modelo final. La mejora de la función de pérdida orientada a la tendencia se plantea como **trabajo futuro**.

6.3.4. Experimentos complementarios

Para consolidar los hallazgos, se llevaron a cabo experimentos adicionales con las configuraciones más prometedoras, explorando los límites de la arquitectura y el impacto de la diversificación de los datos.

Exploración de capas ocultas de mayor tamaño

Habiendo identificado en la fase anterior que un tamaño de capa oculta de 256 neuronas superaba al de 128, se planteó si un modelo de mayor capacidad podría mejorar el rendimiento. Para ello, se evaluaron tamaños de capa oculta de 512 y 1024 neuronas, comparándolos con los resultados previos. La Tabla 6.6 consolida todos los hallazgos.

Tabla 6.6: Comparativa de rendimiento por tamaño de capa oculta (`hidden_size`).

<code>hidden_size</code>	MAPE 0-4 (%)	MAPE 5-20 (%)	Tiempo Entr. (h)	Overfitting Ratio
256	16.23	21.56	1.04	0.64
512	19.54	36.33	2.27	0.76
1024	18.72	48.88	3.62	1.01

Los resultados muestran que incrementar la capacidad más allá de 256 neuronas es perjudicial. El error a largo plazo se dispara, lo que apunta a un claro **sobreajuste**: el modelo memoriza el ruido del conjunto de entrenamiento en lugar de aprender patrones generalizables, tal y como confirma el aumento del `Overfitting_Ratio`. Es posible que la señal en los datos de precios no sea lo suficientemente compleja como para justificar un modelo tan grande. Estos experimentos confirman que el **punto dulce es 256 neuronas**, ofreciendo el mejor rendimiento predictivo sin incurrir en los problemas de sobreajuste o el sobre coste computacional de los modelos de mayor tamaño.

Impacto de datos diversificados

Se realizó un experimento para evaluar si diversificar el conjunto de entrenamiento con instancias de arquitectura ARM mejoraba la generalización del modelo. Se comparó el modelo finalista entrenado solo con datos `x86_64` (Línea base) frente a otro entrenado con el conjunto ampliado. Los resultados, presentados en la Tabla 6.7, fueron reveladores y contraintuitivos.

Tabla 6.7: Comparativa de rendimiento al incluir datos de arquitecturas ARM.

Conjunto de Datos	MAPE 0-4 (%)	MAPE 5-20 (%)	Eficiencia Ahorro (%)	Ahorro de Coste (%)
<code>x86_64</code>	16.41	21.55	7.8	29.5
<code>x86_64 + ARM</code>	20.62	25.21	16.3	65.1

Contrario a la hipótesis inicial, la diversificación de datos incrementó el error de predicción (MAPE). Sin embargo, este mismo modelo **duplicó las métricas de negocio**, tanto el Ahorro de Coste como la Eficiencia del Ahorro. Este fenómeno sugiere que el modelo entrenado con datos más heterogéneos, aunque **menos preciso** en el valor exacto del precio, aprende **patrones de comportamiento más generales y útiles**. Su capacidad para identificar la dirección y magnitud de los cambios de precio es superior, lo que conduce a decisiones estratégicas de aprovisionamiento mucho más rentables. El hallazgo es crucial, pues demuestra que la optimización exclusiva de métricas de error estadístico (MAPE) puede ser subóptima para un problema con un objetivo de negocio claro. Un modelo con un MAPE más alto puede ser, objetivamente, más valioso.

6.4. Análisis y discusión de los resultados

En esta sección se presentan y analizan los resultados obtenidos durante la fase de experimentación. El objetivo es realizar una evaluación cuantitativa y cualitativa de las arquitecturas de modelos finalistas para seleccionar la solución más óptima, así como discutir las limitaciones y desafíos encontrados durante el proceso.

6.4.1. Comparativa de arquitecturas

Se evaluaron las cuatro arquitecturas principales bajo condiciones idénticas, utilizando la configuración de hiperparámetros óptima (grano horario de 12 horas, predicción de 4 días y capa oculta de 256 neuronas) sobre el conjunto de datos de instancias `x86_64`. La Tabla 6.8 resume los resultados.

El análisis revela un claro balance de fortalezas. No hay un único ganador absoluto, y la elección del mejor modelo depende del caso de uso específico. La arquitectura **FeatureSeq2Seq** se posiciona como la más equilibrada, con el mejor MAPE global y a largo plazo. La hipótesis es que la incorporación de características de instancia le proporciona el contexto necesario para capturar tendencias extendidas. Por otro lado, el modelo **Seq2Seq** simple sorprende al lograr el mejor rendimiento a corto plazo y, de manera aún más significativa, las mejores métricas de negocio. Esto sugiere que para predicciones

Tabla 6.8: Resultados comparativos de las arquitecturas finalistas (Datos: solo x86_64).

Arquitectura	MAPE (%)	MAPE 0-4 (%)	MAPE 5-20d (%)	Overfitting	Eficiencia Ahorro (%)	Precisión Tendencia (%)
LSTM	23.48	19.47	24.82	1.95	13.7	55.1
GRU	20.33	12.62	22.90	1.93	8.3	51.4
Seq2Seq	22.92	11.57	26.71	0.56	28.4	62.4
FeatureSeq2Seq	20.08	13.03	22.44	0.57	20.9	56.4

inmediatas, la especialización en la tarea de regresión paso a paso es más importante que el contexto de la instancia.

El hallazgo más inesperado es la **superioridad del modelo GRU sobre LSTM** en casi todas las métricas de precisión, contradiciendo la premisa teórica de que LSTM sería superior por su mayor capacidad teórica para capturar dependencias a largo plazo. Esto indica que la menor complejidad de GRU es suficiente y menos propensa a dificultades de optimización, por lo que se adapta mejor a la naturaleza de este problema.

6.4.2. Impacto de la diversificación de datos

A continuación, se evaluó el impacto de entrenar los modelos con un conjunto de datos más diverso. Se compararon los resultados de los modelos entrenados exclusivamente con datos de la arquitectura x86_64 frente a los mismos modelos entrenados con un conjunto de datos ampliado que también incluía instancias con arquitectura ARM. La Tabla 6.9 compara los resultados.

Tabla 6.9: Impacto de la diversificación de datos en el rendimiento de los modelos.

Arquitectura	Conjunto de Datos	MAPE 0-20 (%)	Eficiencia Ahorro (%)	Precisión Tendencia (%)	Overfitting
	x86_64 + ARM	27.20	40.2	68.1	1.34
Seq2Seq	x86_64	22.92	28.4	62.4	0.56
	x86_64 + ARM	28.42	40.2	68.1	1.34
FeatureSeq2Seq	x86_64	20.08	20.9	56.4	0.57
	x86_64 + ARM	25.64	31.5	60.1	0.70

Los resultados confirman una tendencia paradójica: para todas las arquitecturas, la inclusión de datos diversificados aumenta el error de predicción (MAPE), pero, a la vez, mejora drásticamente las métricas de negocio. Este hallazgo sugiere que, aunque el modelo pierde precisión en el valor exacto, aprende patrones de comportamiento de precios más generales y abstractos. El modelo **Seq2Seq**, por ejemplo, ve su Eficiencia Ahorro aumentar de 28.4 % a 40.2 % y su Precisión Tendencia de 62.4 % a 68.1 % al ser entrenado con el conjunto de datos más rico. Esta capacidad de abstracción le permite predecir mejor la dirección y relevancia de los cambios, que es la información que realmente aporta valor para tomar decisiones de aprovisionamiento. El modelo se vuelve menos preciso, pero estratégicamente más inteligente; la optimización del modelo no debe centrarse exclusivamente en minimizar el error de predicción, sino en maximizar las métricas de negocio.

6.4.3. Selección y análisis del modelo finalista

La evaluación exhaustiva de las distintas arquitecturas y estrategias de datos revela una dicotomía fundamental entre la precisión puramente estadística y el valor práctico para el negocio. Si bien el modelo `FeatureSeq2Seq` entrenado exclusivamente con datos `x86_64` logró el Error Porcentual Absoluto Medio (MAPE) más bajo, demostrando una alta precisión en la predicción del valor del precio, no es el modelo que mejor responde a los objetivos de este trabajo.

6.4.4. Evaluación en otras regiones

Para concluir la fase de experimentación, se evaluó la capacidad de generalización del modelo con mejor rendimiento promedio en la región de `eu-north-1`. El objetivo era validar su robustez y adaptabilidad en mercados con dinámicas de precios menos volátiles. Para ello, se seleccionaron dos regiones adicionales, `us-east-1` y `ap-northeast-1`, que, según el análisis de volatilidad del Anexo C, presentan un comportamiento más estable.

El modelo `featureseq2seq`, que había demostrado el mape más bajo en promedio en un horizonte de 20 días, fue entrenado desde cero para cada una de estas nuevas regiones, manteniendo la misma configuración de hiperparámetros que en los experimentos originales. Los resultados consolidados, que se comparan con el rendimiento del modelo en la región de alta volatilidad, se presentan en la Tabla 6.10.

Tabla 6.10: Resultados Comparativos del Modelo en Regiones con Diferente Volatilidad

Métrica	<code>eu-north-1</code> (Alta Volatilidad)	<code>us-east-1</code> (Menor Volatilidad)	<code>ap-northeast-1</code> (Menor Volatilidad)
MAPE (0-4 días)	13.03 %	5.80 %	5.40 %
MAPE (5-20 días)	22.44 %	13.17 %	20.87 %
MAPE (0-20 días)	20.08 %	11.33 %	17.00 %
Overfitting Ratio	0.57	0.42	0.43
Trend Accuracy	57.48 %	71.77 %	59.53 %
Cost Savings	0.727	0.301	0.024
Savings Efficiency	20.87 %	16.67 %	-1.03 %

Los resultados confirman la hipótesis de que el modelo no solo es robusto, sino que su rendimiento mejora sustancialmente en entornos de menor volatilidad. En las regiones `us-east-1` y `ap-northeast-1`, el MAPE a corto plazo (0-4 días) se redujo drásticamente a un 5.80 % y 5.40 % respectivamente, frente al 13.03 % observado en `eu-north-1`. Esta mejora demuestra la capacidad del modelo para capturar con mayor precisión las tendencias de precios en mercados más predecibles.

El rendimiento a largo plazo también mostró mejoras notables. En particular, la región `us-east-1` obtuvo un MAPE global del 11.33 %, casi la mitad del error registrado en la región de alta volatilidad. Adicionalmente, los tiempos de entrenamiento fueron significativamente menores, destacando el caso de `ap-northeast-1`, que requirió tan solo 0.4 horas, lo que subraya la eficiencia del modelo en mercados más estables.

En cuanto a las métricas de negocio, los resultados son más matizados. La región **us-east-1** mantuvo una eficiencia de ahorro (*Savings Efficiency*) sólida del 16.67%. Sin embargo, en **ap-northeast-1**, la eficiencia fue negativa (-1.03 %), lo que sugiere que en mercados con fluctuaciones de precios muy leves, las oportunidades para optimizar costes mediante la postergación de tareas son limitadas, y el modelo puede no generar un beneficio económico tangible a pesar de su precisión predictiva.

Para un análisis de los resultados de cada una de las configuraciones evaluadas en los distintos escenarios, se presentan tablas comparativas en el Anexo F, que recogen de forma las métricas obtenidas en toda la fase de experimentación.

Capítulo 7

Orquestación e integración en la nube

Este capítulo detalla la arquitectura de una solución que dota de inteligencia económica a Apache Airflow, transformando su comportamiento de planificación. Se propone una extensión modular que consulta el modelo predictivo para determinar el momento de ejecución más rentable para cargas de trabajo flexibles, lanzándolas cuando el coste sea óptimo.

7.1. Oportunidad de mejora

Surge una potencial optimización ya que los orquestadores de flujos de trabajo estándar, como Airflow, Prefect o Dagster, operan con planificaciones estáticas, ejecutando tareas según un calendario fijo o dependencias predefinidas. Este paradigma, presenta un potencial de optimización significativo en el contexto de las instancias Spot. Podríamos incorporar conocimiento sobre las fluctuaciones de precios y así planificar las ejecuciones en los momentos más económicos.

7.1.1. Fundamentos de Apache Airflow

Para comprender cómo se integra la solución propuesta, es fundamental describir primero el funcionamiento de Apache Airflow, la plataforma de orquestación seleccionada. En Airflow, un flujo de trabajo se representa como un **Grafo Acíclico Dirigido** o **DAG**, que es un script de Python que define una secuencia de tareas y sus dependencias [2]. Cada DAG está compuesto por tareas individuales, y cada tarea es una instancia de un **Operador**. El Operador es la unidad de trabajo fundamental que encapsula la lógica de una acción específica, como ejecutar un script o una consulta. La arquitectura de Airflow, ilustrada en la Figura 7.1, está diseñada para orquestar la ejecución de estos operadores.

Los componentes principales que gestionan este proceso son: el **Planificador** (*Scheduler*), que es el corazón del sistema y decide qué tareas ejecutar; el **Ejecutor** (*Executor*), que gestiona cómo y dónde se ejecutan dichas tareas [3]; y los **Trabajadores** (*Workers*), que son los procesos que finalmente ejecutan la lógica de cada operador. El estado de todo el sistema se almacena de forma

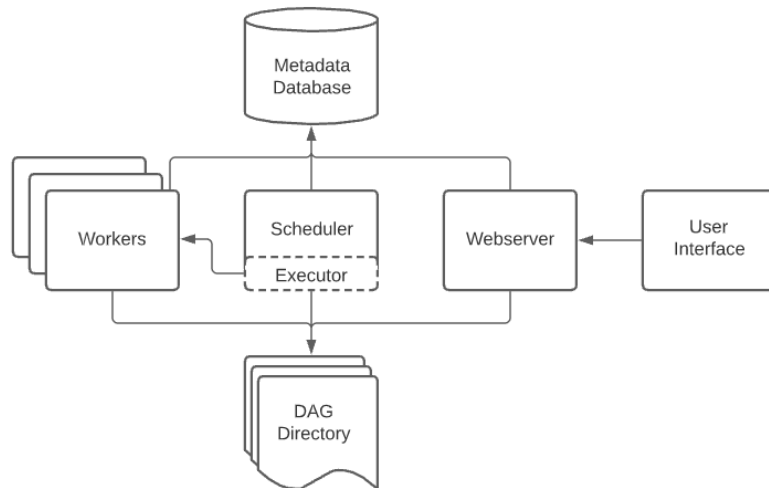


Figura 7.1: Arquitectura de alto nivel de Apache Airflow.

persistente en una **Base de Datos de Metadatos**, mientras que una **Interfaz Web (Webserver)** permite a los usuarios monitorizar y gestionar los flujos de trabajo.

El ciclo de vida de una tarea comienza cuando el **Scheduler** la identifica como lista para ejecutar dentro de un DAG. La envía al **Executor**, que a su vez la asigna a un **Worker**. Mientras tanto, todos los componentes se coordinan a través de la **Metadata Database**, y el **Webserver** ofrece una vista en tiempo real del proceso.

7.1.2. Diseño del pipeline de orquestación (DAGs)

La solución de orquestación se materializa como un DAG de Airflow que integra la inteligencia de predicción de precios para optimizar la ejecución de cargas de trabajo. Este DAG se compone de operadores especializados y estándar que trabajan de forma coordinada. La Figura 7.2 ilustra cómo interactúan los componentes diseñados para este fin:

1. **API de Predicción de Costes (PredictionAPI):** Como se detalló en la Sección 5.5.1, este servicio web expone las capacidades del modelo de IA. Aunque es un componente externo al DAG, es el cerebro al que consultan nuestros operadores para la toma de decisiones. Su responsabilidad es devolver un pronóstico de precios para una carga de trabajo específica.
2. **Sensor de Optimización de Coste (CostOptimizationSensor):** Para integrar la lógica de decisión en Airflow, se desarrollará un sensor personalizado (CostOptimizationSensor). Se eligió este tipo de operador frente a un **PythonOperator** estándar porque el patrón de *esperar una condición externa* es la función principal de los sensores en Airflow. Este enfoque es más idiomático y eficiente dentro del ecosistema, ya que permite que el sensor ocupe una ranura de trabajo (worker slot) solo durante los breves momentos de sondeo (*poke*), liberándola en los intervalos de espera.

3. **Operador de Ejecución:** Esta tarea, que se ejecuta inmediatamente después del sensor, corresponde a un operador estándar de Airflow (ej. `PythonOperator` o `BashOperator`). Su función es ejecutar la carga de trabajo principal del usuario (ej., un proceso de *batch*). Este operador no requiere ninguna modificación, ya que se beneficia directamente de la decisión de temporización inteligente tomada por el `CostOptimizationSensor` que le precede.

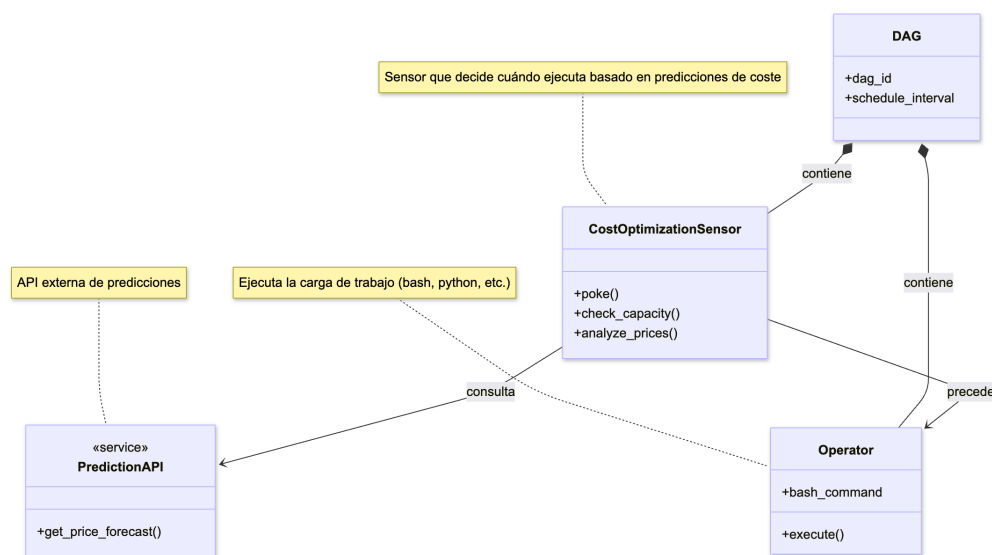


Figura 7.2: Diagrama de los elementos a modificar en Apache Airflow

7.2. Flujo de decisión lógica

La interacción coordinada de estos componentes transforma un DAG estándar en un proceso económicamente inteligente. El método `poke` del `CostOptimizationSensor` se ejecuta a intervalos regulares y sigue un algoritmo de decisión claro:

1. **Activación:** El DAG se inicia y el `CostOptimizationSensor` se activa, pausando el flujo.
2. **Prioridad de capacidad provisionada:** Como primera comprobación, el sensor puede verificar si existe capacidad de cómputo ya disponible. Si es así, no hay beneficio económico en esperar, por lo que autoriza la ejecución inmediata.
3. **Consulta a la API Predictiva:** El sensor realiza una llamada a la `PredictionAPI`, enviando los parámetros de la tarea. La API responde con el precio actual y una serie temporal de predicciones de precios para la ventana de decisión configurada.
4. **Análisis de Opciones Viables:** El sensor itera sobre la serie de precios pronosticada para identificar la mejor oportunidad de ejecución. Para cada punto futuro en el pronóstico, evalúa dos condiciones:

- **Viabilidad temporal:** Verifica que la ejecución en ese punto futuro, más su duración estimada y un margen de seguridad, no sobrepase el plazo límite.
 - **Viabilidad económica:** Calcula el ahorro potencial respecto al precio actual y comprueba si supera un umbral de ahorro predefinido (ej., 2%).
5. **Selección de la Estrategia Óptima:** Tras analizar todos los puntos futuros, el sensor identifica la opción más rentable (`timestamp_óptimo_viable`) de entre todas aquellas que son viables tanto temporal como económicamente.
6. **Decisión final:**
- Si se ha encontrado un `timestamp_óptimo_viable`, significa que existe una oportunidad futura de ahorro que respeta el plazo. La decisión es **ESPERAR**. El sensor retorna **False** y volverá a sondear en el siguiente intervalo.
 - Si no existe ninguna oportunidad futura que cumpla ambas condiciones de viabilidad, el beneficio de la espera es nulo o arriesgado. La decisión es **EJECUTAR AHORA**. El sensor finaliza con éxito, retornando **True**.
7. **Ejecución Óptima:** Una vez que el sensor finaliza, el DAG continúa y el siguiente operador lanza la carga de trabajo.

La Figura 7.3 representa este proceso, donde el `CostOptimizationSensor` se convierte en el centro de decisión que, apoyado por la `PredictionAPI`, decide el ciclo de vida de la ejecución.

7.3. Conclusiones sobre la integración

La integración de un sensor de optimización de costes en Apache Airflow podría ser un método eficaz y no invasivo para transformar un planificador estático en un sistema dinámico y económicamente consciente. Al desacoplar la lógica de decisión en un componente modular y consumir un servicio de predicción, se dota a los flujos de trabajo de la capacidad de adaptarse a las condiciones del mercado en tiempo real, generando ahorros directos en la factura de computación. No obstante, esta solución presenta oportunidades de mejora y futuras líneas de trabajo:

- **Predicción del riesgo de interrupción:** El modelo actual se centra exclusivamente en el precio. Las instancias Spot, sin embargo, pueden ser interrumpidas por AWS. Un modelo más avanzado debería predecir no solo el precio, sino también la probabilidad de interrupción, permitiendo una decisión que equilibre coste y riesgo.
- **Optimización a nivel de DAG:** La optimización actual se realiza tarea por tarea. Una mejora significativa sería analizar el grafo de dependencias completo (el DAG) y optimizar la ruta crítica en su conjunto, tomando decisiones de planificación que consideren la duración y los plazos de todo el flujo de trabajo.

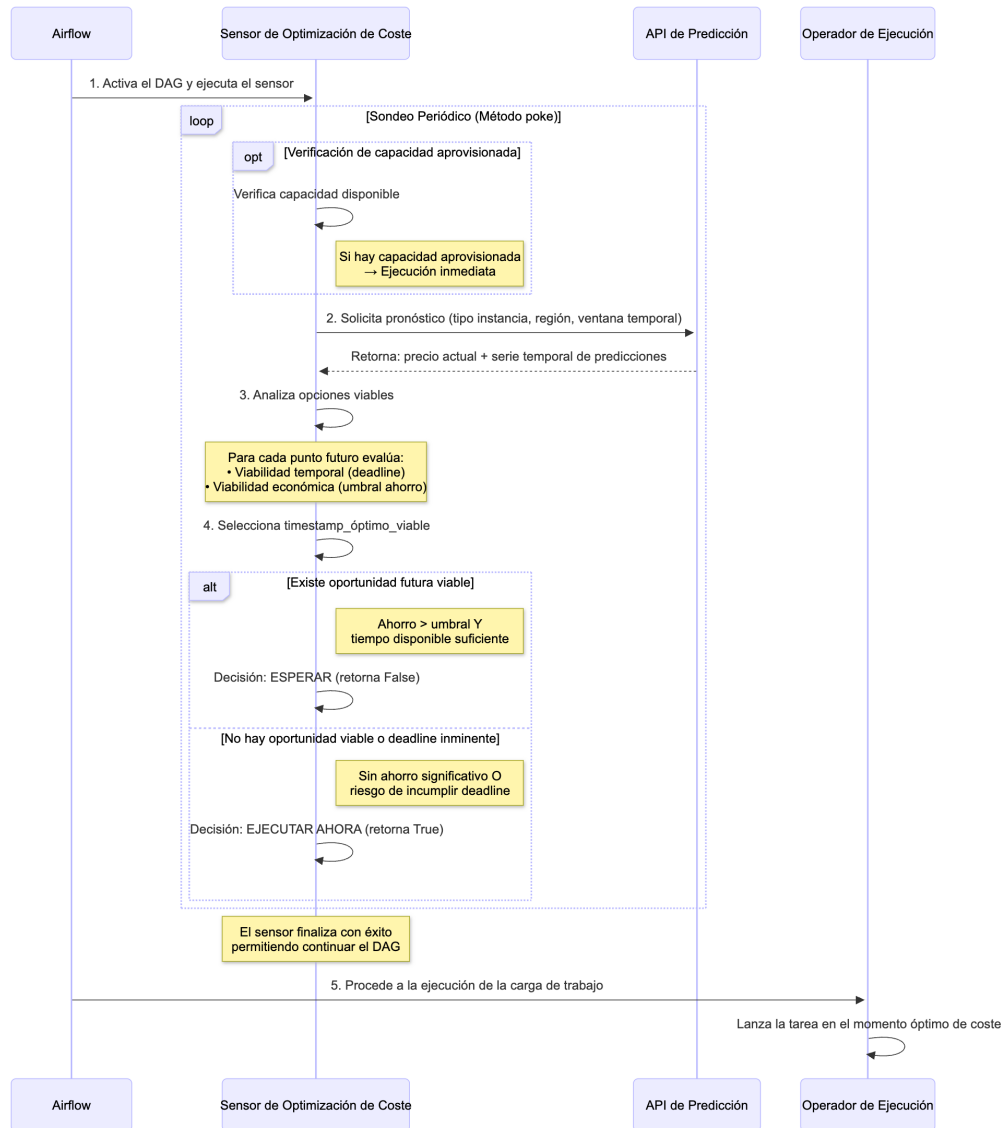


Figura 7.3: Diagrama de secuencia del flujo de decisión inteligente en Airflow

La arquitectura propuesta establece una base sólida para la operación de cargas de trabajo flexibles de manera más inteligente y económica, abriendo la puerta a optimizaciones que refuercen la eficiencia en la nube.

Capítulo 8

Conclusiones y trabajo futuro

En este capítulo se presentan las conclusiones extraídas del diseño y la evaluación experimental del sistema de predicción. Se analizan los resultados principales del proyecto, se exponen las limitaciones identificadas en el trabajo y, finalmente, se proponen futuras líneas de investigación para dar continuidad a la solución desarrollada.

8.1. Conclusiones Principales

El desarrollo de este proyecto ha resultado en un framework completo y reproducible para la predicción de precios de instancias spot de AWS. Este sistema, que abarca desde la captura de datos hasta la servitización de un modelo predictivo, ha sido validado mediante una metodología de evaluación orientada al valor de negocio. El análisis experimental arroja las siguientes conclusiones principales:

La optimización basada en métricas de negocio es más efectiva que la minimización del error estadístico. Se ha demostrado empíricamente que un menor error de predicción (MAPE) no se traduce necesariamente en un mayor valor práctico. Los experimentos revelaron que un modelo entrenado con datos más diversos, aunque obtuvo un MAPE superior (20.62 % vs. 16.41 %), duplicó la eficiencia en el ahorro de costes (16.3 % vs. 7.8 %). Este resultado valida la contribución metodológica del trabajo: la necesidad de optimizar los modelos utilizando KPIs que midan el impacto financiero real, como la Eficiencia del Ahorro.

Una adecuada ingeniería de características supera en impacto a la complejidad del modelo. El análisis de los datos de origen condujo a mejoras de rendimiento más significativas que la propia elección de arquitecturas. La decisión de ajustar la granularidad temporal de las series a 12 horas, fundamentada en la frecuencia media de cambio de precios observada (6.6 horas), mejoró el rendimiento del modelo en un 22.2 % y redujo el tiempo de entrenamiento en un 67 %, demostrando ser una decisión de diseño más crítica que la complejidad arquitectónica.

La simplicidad arquitectónica resulta ventajosa para este problema. Las arquitecturas computacionalmente más sencillas mostraron un rendimiento superior o comparable a las más complejas. El modelo Seq2Seq simple obtuvo mejores métricas de negocio que su variante con características adicionales, y los modelos basados en celdas GRU, más ligeros, superaron en precisión a los basados en LSTM. Esto indica que una menor complejidad se adapta mejor a la naturaleza de los datos de precios Spot y es menos propensa a dificultades de optimización.

8.2. Limitaciones del Trabajo

La solución desarrollada y el enfoque adoptado presentan las siguientes limitaciones:

La viabilidad práctica está condicionada por el beneficio económico real. La utilidad del predictor depende directamente de la volatilidad del mercado. Como se demostró en la región `ap-northeast-1`, a pesar de obtener una alta precisión predictiva, el beneficio económico del sistema fue negativo (-1.03 %), ya que las fluctuaciones de precios eran marginales. Su aplicabilidad queda, por tanto, acotada a mercados donde las variaciones de precio justifiquen la complejidad de la solución.

El precio no es un indicador fiable de la disponibilidad. El sistema optimiza las decisiones basándose exclusivamente en el precio. Sin embargo, el mercado Spot presenta una desconexión entre coste y capacidad, ya que una instancia puede ser interrumpida por necesidad de recursos de AWS, independientemente de su precio. Por tanto, el sistema se enfrenta a la limitación de poder recomendar un momento de ejecución óptimo en coste en el que, en la práctica, no haya capacidad disponible.

8.3. Líneas de Investigación Futura

Basándose en los resultados y limitaciones, se proponen las siguientes líneas de investigación para dar continuidad al proyecto:

Incorporar características temporales en el modelo. Durante el análisis exploratorio se identificaron patrones horarios recurrentes en los cambios de precio. Se propone integrar estas características temporales en la arquitectura del modelo, codificadas mediante funciones cíclicas, para evaluar si mejoran su precisión predictiva.

Diseñar funciones de pérdida que optimicen métricas de negocio. Se plantea investigar y desarrollar una función de pérdida ponderada que incorpore directamente los KPIs de negocio, como el ahorro de coste, en el cálculo del gradiente. Esto alinearía el proceso de entrenamiento con la maximización de su valor práctico, en lugar de centrarse únicamente en la minimización del error estadístico.

Evolucionar el sistema hacia un recomendador de instancias multi-objetivo. El siguiente paso lógico es transformar el predictor en un recomendador que considere múltiples variables. Esto implicaría integrar nuevas fuentes de datos, como el Spot Placement Score (SPS), para predecir simultáneamente el precio y el riesgo de interrupción, ofreciendo así recomendaciones que equilibren coste, rendimiento y estabilidad.

Implementar y validar la integración en un orquestador. Resulta fundamental llevar a la práctica la arquitectura teórica de integración con Apache Airflow. La implementación del CostOptimizationSensor en un entorno real permitiría validar empíricamente los beneficios de la planificación dinámica y medir el ahorro de costes en un escenario de producción.

Capítulo 9

Bibliografía

- [1] URL: <https://spot-bid-advisor.s3.amazonaws.com/spot-advisor-data.json>.
- [2] Airflow dags. Accedido: 27-06-2025. URL: <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html>.
- [3] Airflow executor. Accedido: 27-06-2025. URL: <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/executor/index.html>.
- [4] Airflow operators. Accedido: 27-06-2025. URL: <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/operators.html>.
- [5] Amazon ec2. Accedido: 2025-06-26. URL: <https://aws.amazon.com/ec2/>.
- [6] Amazon ec2 features. Accedido: 2025-06-23. URL: <https://aws.amazon.com/ec2/features/>.
- [7] Amazon ec2 spot. Accedido: 09-06-2025. URL: <https://aws.amazon.com/es/ec2/spot>.
- [8] Attention (machine learning). URL: [https://en.wikipedia.org/wiki/Attention_\(machine_learning\)](https://en.wikipedia.org/wiki/Attention_(machine_learning)).
- [9] Autoencoder. URL: <https://en.wikipedia.org/wiki/Autoencoder>.
- [10] Backpropagation through time. URL: https://en.wikipedia.org/wiki/Backpropagation_through_time.
- [11] Batch processing. Accedido: Accedido: 26-06-2025. URL: https://en.wikipedia.org/wiki/Batch_processing/.
- [12] Ec2 instance rebalance recommendations. Accedido: 27-06-2025. URL: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/rebalance-recommendations.html>.
- [13] Formas de compra ec2. Accedido: 26-06-2025. URL: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/instance-purchasing-options.html>.
- [14] Gated recurrent unit. URL: https://en.wikipedia.org/wiki/Gated_recurrent_unit.

- [15] Long short-term memory. URL: https://en.wikipedia.org/wiki/Long_short-term_memory.
- [16] Modelo autorregresivo integrado de media móvil. Accedido: 27-06-2025. URL: https://es.wikipedia.org/wiki/Modelo_autorregresivo_integrado_de_media_m%C3%B3vil.
- [17] Recurrent neural network. URL: https://en.wikipedia.org/wiki/Recurrent_neural_network.
- [18] Red neuronal recurrente. Accedido: 27-06-2025. URL: https://www.tensorflow.org/tutorials/structured_data/time_series?hl=es-419#recurrent_neural_network.
- [19] Seq2seq. URL: <https://en.wikipedia.org/wiki/Seq2seq>.
- [20] Spot instance interruptions. Accedido: 27-06-2025. URL: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-interruptions.html>.
- [21] Transformer (deep learning architecture). URL: [https://en.wikipedia.org/wiki/Transformer_\(deep_learning_architecture\)](https://en.wikipedia.org/wiki/Transformer_(deep_learning_architecture)).
- [22] Vanishing gradient problem. URL: https://en.wikipedia.org/wiki/Vanishing_gradient_problem.
- [23] Worldwide public cloud services spending to reach \$597 billion in 2023, according to idc, 2023. Accedido: 2025-05-23. URL: <https://my.idc.com/getdoc.jsp?containerId=prUS53284225>.
- [24] A. Baldominos Gómez, Y. Saez, D. Quintana, and P. Isasi. AWS PredSpot: Machine Learning for Predicting the Price of Spot Instances in AWS Cloud. *International Journal of Interactive Multimedia and Artificial Intelligence*, 2022. URL: https://www.researchgate.net/publication/358954877_AWS_PredSpot_Machine_Learning_for_Predicting_the_Price_of_Spot_Instances_in_AWS_Cloud, doi:10.9781/ijimai.2022.03.005.
- [25] Cast AI. Spot instance pricing: How to maximize cost savings. URL: <https://cast.ai/blog/spot-instance-pricing/>.
- [26] E. L. Kvinge. General Global Models for Time Series Forecasting. Master's thesis, Norwegian University of Science and Technology, 2021. URL: https://www.researchgate.net/publication/356421279_Global_Models_for_Time_Series_Forecasting_A_Simulation_Study.
- [27] S. Lee, J. Hwang, and K. Lee. Spotlake: Diverse spot instance dataset archive service, 2022. In 2022 IEEE International Symposium on Workload Characterization (IISWC). URL: <https://leeky.me/publications/spotlake.pdf>.
- [28] M. Malik and N. Bagmar. Forecasting price of amazon spot instances using machine learning, 2021. *International Journal of Artificial Intelligence and Machine*

- Learning (IJAIML), vol. 11, no. 2. URL: <https://www.igi-global.com/article/forecasting-price-of-amazon-spot-instances-using-machine-learning/277435>.
- [29] nOps. Ec2 spot instances: How to use them to save on aws costs. URL: <https://www.nops.io/blog/ec2-spot-instances/>.
- [30] PostgreSQL Documentation. 64.5. brin indexes. URL: <https://www.postgresql.org/docs/current/brin.html>.
- [31] PostgreSQL Documentation. Chapter 11. indexes. URL: <https://www.postgresql.org/docs/current/indexes.html>.
- [32] React Documentation. URL: <https://react.dev/reference/react>.
- [33] Recharts Documentation. URL: <https://recharts.org/en-US>.
- [34] X. Song, R. Lin, and H. Zou. Amazon ec2 spot price prediction using temporal convolution network, 2022. In ICETIS 2022; 7th International Conference on Electronic Technology and Information Science. URL: <https://www.vde-verlag.de/proceedings-en/565778124.html>.
- [35] StackScale. Migración segura al cloud: on-premise, híbrida y multicloud. Accedido: 26-06-2025. URL: <https://www.stackscale.com/es/blog/migracion-segura-al-cloud/>.
- [36] F. Yang, B. Pang, J. Zhang, B. Qiao, L. Wang, C. Couturier, C. Bansal, S. Ram, S. Qin, and Z. Ma et al. Spot virtual machine eviction prediction in microsoft cloud, 2022. In Companion Proceedings of the Web Conference 2022. URL: https://www.researchgate.net/publication/359106510_Spot_Virtual_Machine_Eviction_Prediction_in_Microsoft_Cloud.

Lista de Figuras

2.1. Modelos de Servicio en la Nube: IaaS, PaaS, SaaS. Fuente: [35]	5
3.1. Arquitectura general del sistema de predicción	13
3.2. Arquitectura del módulo de captura y gestión de datos	14
3.3. Arquitectura del módulo predictivo	14
4.1. Traza de ejemplo de la respuesta de la API <code>describe_spot_price_history</code> de AWS. .	17
4.2. Esquema Relacional de la Base de Datos	20
4.3. Distribución del precio medio de spot AWS por instancia.	21
4.4. Precio de <code>c5n.xlarge</code> en <code>eu-north-1a</code> desde 2024-01-01 hasta 2025-03-12.	22
4.5. Precio de <code>c5n.xlarge</code> en distintas regiones y AZ desde 2024-01-01 hasta 2025-03-12. .	22
4.6. Distribución de la frecuencia en la variación de precio.	23
5.1. Precios de las instancias <code>c5 xlarge</code> y <code>12xlarge</code> en <code>eu-north-1a</code>	26
5.2. Arquitectura del enfoque básico	27
5.3. Arquitectura del modelo Seq2Seq	28
5.4. Arquitectura del modelo FeatureSeq2Seq	29
5.5. Enfoque híbrido	31
5.6. Interfaz del metodo de predicción en Swagger de la API	32
6.1. Degradación del rendimiento para modelos con <code>tr_prediction_length</code> de 2 días y 4 días	39
7.1. Arquitectura de alto nivel de Apache Airflow.	47
7.2. Diagrama de los elementos a modificar en Apache Airflow	48
7.3. Diagrama de secuencia del flujo de decisión inteligente en Airflow	50
A.1. Arquitectura de una Red Neuronal Recurrente desenrollada. Fuente: [17]	62
A.2. Arquitectura de una celda Long Short-Term Memory. Fuente [15]	64
A.3. Arquitectura de una celda Gated Recurrent Unit. Fuente [14]	65
A.4. Arquitectura básica Sequence-to-Sequence (Encoder-Decoder)	65
A.5. Esquema de un Mecanismo de Atención en una Arquitectura Seq2Seq. Fuente [9] . . .	66

C.1. Distribución horaria (UTC) de la frecuencia de cambios de precio.	72
G.1. Sistema de filtrado dinámico de las instancias evaluadas	82
G.2. Tabla comparativa de las diferencias de configuración entre dos modelos	82
G.3. Tabla de comparación de métricas clave para todos los modelos evaluados.	83
G.4. Evolución del error (MAPE) en función del horizonte de predicción	83
G.5. Rendimiento del modelo (MAPE) agrupado por generación de instancia	83
G.6. Matriz de progresión del MAPE por tipo de instancia y horizonte temporal.	84
G.7. Distribución de errores de predicción (MAPE) en rangos de precisión	84
G.8. Análisis de la eficiencia de costes, comparando el ahorro real y el teórico.	84

Lista de Tablas

3.1. Requisitos funcionales del sistema	12
3.2. Requisitos no funcionales del sistema	13
4.1. Requisitos funcionales del módulo de captura	16
4.2. Requisitos no funcionales del módulo de captura	17
6.1. Resumen de volatilidad de precios por región	34
6.2. Comparativa de rendimiento por grano horario (<code>timestep_hours</code>).	38
6.3. Comparativa de rendimiento por longitud de predicción en el entrenamiento.	39
6.4. Comparativa de rendimiento por tamaño de capa oculta (<code>hidden_size</code>).	40
6.5. Comparativa de rendimiento por peso de la función de pérdida (<code>mse_weight</code>).	41
6.6. Comparativa de rendimiento por tamaño de capa oculta (<code>hidden_size</code>).	41
6.7. Comparativa de rendimiento al incluir datos de arquitecturas ARM.	42
6.8. Resultados comparativos de las arquitecturas finalistas (Datos: solo x86_64).	43
6.9. Impacto de la diversificación de datos en el rendimiento de los modelos.	43
6.10. Resultados Comparativos del Modelo en Regiones con Diferente Volatilidad	44
C.1. Resultados del análisis de volatilidad en las regiones	72
F.1. Tabla comparativa de los resultados de todos los modelos evaluados en la fase de experimentación.	78
I.1. Tabla de estimación de esfuerzo por fases y tareas.	88

Anexos

Anexo A

Conceptos principales sobre *deep learning*

A.1. Redes neuronales recurrentes (RNNs)

Las *redes neuronales recurrentes* (RNNs)[17] están intrínsecamente diseñadas para procesar secuencias de datos, donde la información de un paso temporal anterior es crucial para comprender el paso actual. A diferencia de las redes neuronales *feed-forward* tradicionales, que asumen la independencia entre entradas, las RNNs poseen una memoria interna que les permite persistir información a través del tiempo. Esta capacidad se materializa a través de un **estado oculto** (h_t) que se actualiza en cada paso temporal (t).

Formalmente, la activación del estado oculto en el instante t se calcula a partir de la entrada actual (x_t) y el estado oculto previo (h_{t-1}). La ecuación fundamental que describe esta actualización es:

$$h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

donde x_t es el vector de entrada en el paso temporal t , h_t es el estado oculto en el paso t , h_{t-1} es el estado oculto previo, W_{hh} y W_{xh} son matrices de pesos para las conexiones recurrentes y de entrada respectivamente, b_h es el vector de sesgo para la capa oculta, y f es una función de activación no lineal (ej., tanh o ReLU).

La salida en el instante t , denotada como y_t , se calcula a partir del estado oculto actual:

$$y_t = g(W_{hy}h_t + b_y)$$

donde W_{hy} es la matriz de pesos para la capa de salida, b_y es el vector de sesgo de salida, y g es una función de activación apropiada para la tarea (ej., lineal para regresión).

El concepto de memoria en las RNNs se visualiza al desenrollar la red en el tiempo, como se muestra en la Figura A.1. Cada bloque en el diagrama representa una unidad recurrente en un paso temporal específico, procesando una entrada x_t y propagando su estado oculto h_t al siguiente paso. Este mecanismo permite a las RNNs capturar dependencias temporales, siendo inherentemente adecuadas para modelar series temporales.

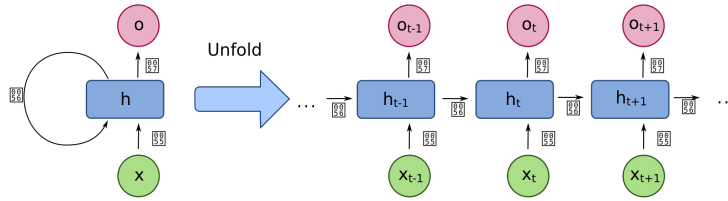


Figura A.1: Arquitectura de una Red Neuronal Recurrente desenrollada. Fuente: [17]

Problema del desvanecimiento/explosión del gradiente

A pesar de su capacidad para modelar secuencias, las RNNs tradicionales presentan un desafío significativo durante el entrenamiento conocido como el problema del desvanecimiento o la explosión del gradiente[22]. Este fenómeno ocurre durante el proceso de **retropropagación** a través del tiempo (*BPTT*)[10], donde los gradientes se propagan desde la capa de salida hacia las capas temporales anteriores.

El **desvanecimiento del gradiente** se produce cuando los gradientes se vuelven exponencialmente pequeños a medida que se retropropagan. Las capas anteriores de la red, que corresponden a pasos temporales distantes, no actualizan sus pesos de manera efectiva por lo que la red no tiene en cuenta las dependencias a largo plazo y no aprende patrones en secuencias extensas. Conceptualmente, esto es similar a multiplicar repetidamente números menores que uno, lo que lleva a un valor final extremadamente pequeño.

Por el contrario, la **explosión del gradiente** ocurre cuando los gradientes se vuelven exponencialmente grandes, lo que provoca actualizaciones de pesos excesivamente grandes y hace que el modelo diverja, impidiendo un entrenamiento estable. Esto es análogo a multiplicar repetidamente números mayores que uno. Para mitigar estos problemas, se desarrollaron arquitecturas de RNN más avanzadas.

A.2. Variantes de Redes Neuronales Recurrentes

Para superar las limitaciones de las RNNs básicas, especialmente el problema del gradiente y la dificultad para capturar dependencias a largo plazo, se han desarrollado variantes especializadas. Las más destacadas son las redes Long Short-Term Memory (LSTM) y las Gated Recurrent Units (GRU).

A.2.1. Long Short-Term Memory (LSTM)

Las redes Long Short-Term Memory (LSTM)[15] son una evolución de las RNNs diseñadas específicamente para resolver los problemas de desvanecimiento y explosión del gradiente, permitiendo a la red aprender dependencias a largo plazo. Lo logran mediante la introducción de un componente clave: la **celda de memoria** (C_t), que actúa como un carril separado para el estado de la celda, y un conjunto de **compuertas** que regulan el flujo de información hacia y desde esta celda. Estas

compuertas son capas de redes neuronales *sigmoid* que producen valores entre 0 y 1, indicando cuánta información debe pasar.

Una celda LSTM consta de tres compuertas principales:

1. **Compuerta de Olvido (*Forget Gate*, f_t):** Decide qué información de la celda de memoria anterior (C_{t-1}) debe ser descartada. Su activación se calcula como:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

donde σ es la función sigmoide.

2. **Compuerta de Entrada (*Input Gate*, i_t):** Decide qué nueva información se va a almacenar en la celda de memoria. Consiste en dos partes: una compuerta sigmoide que decide qué valores actualizar y una capa $\tanh$ que crea un vector de nuevos valores candidatos ($\tilde{C}_t$) para añadir al estado de la celda.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

El nuevo estado de la celda de memoria (C_t) se actualiza combinando la información olvidada y la nueva información de entrada:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

donde $\odot$ denota el producto elemento a elemento (Hadamard).

3. **Compuerta de Salida (*Output Gate*, o_t):** Decide qué parte del estado de la celda de memoria (C_t) se va a exponer como estado oculto actual (h_t) para el siguiente paso temporal y la predicción.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

Gracias a estas compuertas, las LSTMs pueden regular explícitamente el flujo de información, permitiendo retener datos relevantes a largo plazo y olvidar los irrelevantes. Esto las hace excepcionalmente aptas para capturar dependencias temporales extendidas, una característica crucial para predecir series de precios volátiles como las Spot Instances. Su arquitectura detallada se ilustra en la Figura A.2.

A.2.2. Gated Recurrent Unit (GRU)

Las Gated Recurrent Units (GRUs) son una variante de las RNNs introducida como una alternativa más simple y computacionalmente más eficiente a las LSTMs, manteniendo al mismo tiempo una capacidad similar para abordar el problema del gradiente y capturar dependencias a largo plazo[14]. A diferencia de las LSTMs, las GRUs combinan la compuerta de olvido y la compuerta de entrada



Una celda GRU consta de dos compuertas principales:

- $$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

- $$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

$$\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t] + b_h)$$
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

64

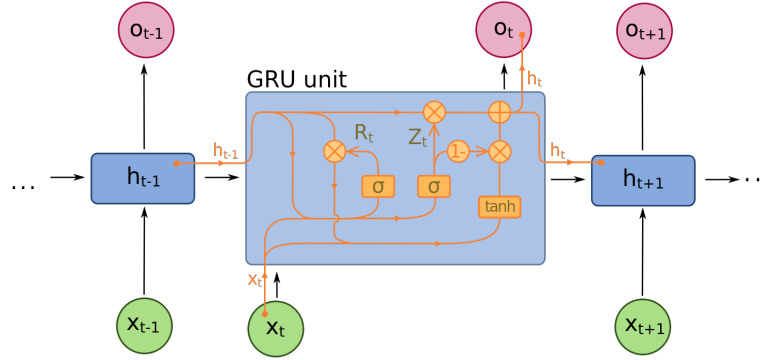


Figura A.3: Arquitectura de una celda Gated Recurrent Unit. Fuente [14]

A.3. Arquitectura Sequence-to-Sequence

La arquitectura **Sequence-to-Sequence (Seq2Seq)** transforma una secuencia de entrada en una de salida, útil para predicción multi-step en series temporales. A diferencia de las RNNs simples que predicen un único valor o requieren un bucle explícito, esta puede generar toda la secuencia de predicciones futuras de una sola vez, aprovechando el contexto de la secuencia de entrada. Este se compone de dos módulos principales, que a su vez son RNNs. El **Encoder** procesa la secuencia de entrada (ej., precios históricos), lee la secuencia y la comprime en un vector de contexto que resume la información clave. El **Decoder** usa el vector de contexto del Encoder para generar la secuencia de salida (ej., precios predichos)[19].

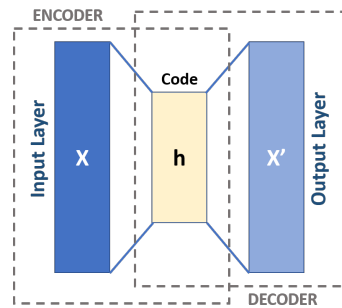


Figura A.4: Arquitectura básica Sequence-to-Sequence (Encoder-Decoder)

A.4. Mecanismos de atención

En el procesamiento de secuencias, especialmente con dependencias a largo plazo, los modelos RNN tradicionales (como LSTMs o GRUs) enfrentan limitaciones debido al desvanecimiento del gradiente. Los **mecanismos de atención** surgen para superar esto. Estos mecanismos permiten al modelo enfocarse selectivamente en partes relevantes de la secuencia de entrada al generar cada elemento de salida. Esto facilita la captura de dependencias a largo plazo, mejora la interpretabilidad al visualizar la importancia de cada entrada, y maneja secuencias largas de manera más eficiente[?].

El proceso de atención se resume en cuatro pasos:

1. **Cálculo de puntuaciones de alineación:** Se puntúa la relevancia de cada elemento de la secuencia de entrada para la salida actual, comparando el estado del decodificador con los estados ocultos del codificador.
2. **Asignación de importancia (Pesos de Atención):** Las puntuaciones se transforman en pesos de probabilidad (que suman 1) usando Softmax, indicando la importancia relativa de cada entrada.
3. **Creación de un vector de contexto:** Los estados ocultos de la entrada se ponderan con los pesos de atención calculados y se suman. El resultado es un "vector de contexto" que resume la información más relevante.
4. **Combinación y generación de la Salida:** Finalmente, el vector de contexto se combina con el estado actual del decodificador para producir la salida final.

En la Figura A.5 se describe el proceso de atención en una arquitectura Sequence-to-Sequence.

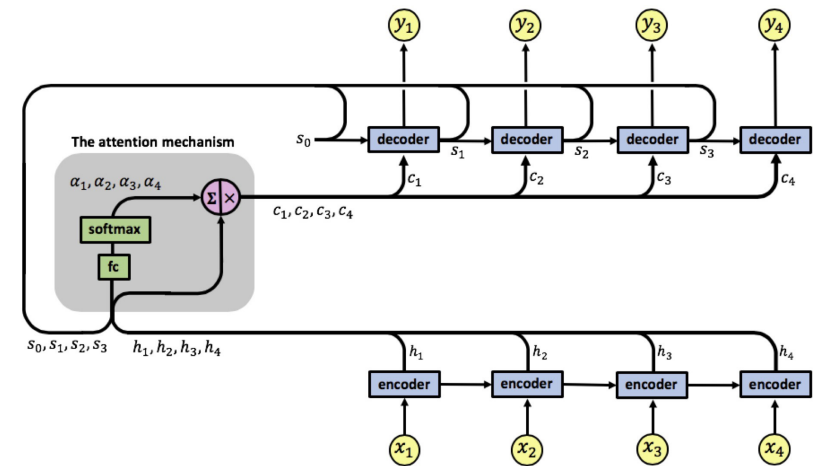


Figura A.5: Esquema de un Mecanismo de Atención en una Arquitectura Seq2Seq. Fuente [9]

A.5. Arquitecturas *Transformer*

Las **arquitecturas Transformer**, introducidas por Vaswani et al. (2017) en "Attention Is All You Need", representan una clase de modelos de *Deep Learning* que han revolucionado el procesamiento de secuencias. A diferencia de las RNNs, los Transformers prescinden completamente de la recurrencia y basan su funcionamiento exclusivamente en mecanismos de auto-atención (*self-attention*) y redes neuronales *feed-forward*. Esta concepción les permite:

- **Procesamiento Paralelo:** Al no depender de operaciones secuenciales, los Transformers pueden procesar todos los elementos de una secuencia de entrada simultáneamente, lo que acelera significativamente el entrenamiento.

- **Captura de Dependencias a Largo Plazo:** El mecanismo de auto-atención permite que cada elemento de la secuencia interactúe directamente con todos los demás, sin importar su distancia, facilitando la identificación de relaciones de largo alcance.
- **Riqueza Representacional:** Mediante múltiples "cabezas" de atención, pueden aprender diversas perspectivas de la información, generando representaciones contextuales más ricas.

Si bien los Transformers han demostrado un rendimiento excepcional en tareas complejas como el procesamiento de lenguaje natural, donde las dependencias a muy largo plazo son comunes y la cantidad de datos es vasta, su aplicación presenta desafíos:

- **Exigencia Computacional:** La complejidad de la auto-atención escala cuadráticamente con la longitud de la secuencia de entrada ($O(n^2)$), lo que implica altos requerimientos de computación y memoria para secuencias largas.
- **Requerimientos de Datos:** Suelen necesitar un gran volumen de datos para un entrenamiento eficaz, ya que su capacidad para modelar patrones complejos se manifiesta plenamente con conjuntos de datos extensos.

Dadas las características específicas de los datos que vamos a trabajar —volátiles pero sin dependencias semánticas o de muy largo alcance típicas del lenguaje natural— y considerando la eficiencia computacional, una arquitectura Transformer completa podría ser un **enfoque excesivamente complejo** para este problema.

A.6. Predicción Multi-Step en series temporales

La predicción de series temporales a menudo requiere pronosticar múltiples valores futuros en lugar de solo el siguiente. Este escenario se conoce como **predicción multi-step**. Existen varios enfoques para abordarlo:

A.6.1. Estrategia iterativa/autoregresiva

Es un método común, especialmente con modelos como LSTM o GRU que naturalmente producen una única salida por paso. Aquí, el modelo predice el siguiente valor ($t + 1$), y esta predicción se reintroduce como entrada para predecir el valor subsiguiente ($t + 2$), y así sucesivamente hasta cubrir el horizonte deseado. Sus ventajas son la simplicidad y la reutilización de un modelo de un solo paso. Su limitación es la acumulación de errores, que pueden magnificarse en pasos futuros, y un mayor coste computacional para horizontes muy largos.

A.6.2. Estrategia directa (*Multi-output*)

Otro enfoque es entrenar un modelo para predecir directamente todos los pasos futuros deseados en una única pasada. Esto requiere que el modelo tenga múltiples salidas, una para cada paso temporal

a predecir. Evita la propagación de errores entre pasos de predicción, pero puede ser más complejo y costoso de entrenar y puede tener dificultades para capturar las dependencias temporales entre las propias predicciones si el horizonte es muy largo.

Anexo B

Métricas de evaluación

En este anexo se detallan las métricas más comúnmente empleadas en el ámbito de la predicción de series temporales.

B.1. Error Absoluto Medio (MAE)

El **Error Absoluto Medio (Mean Absolute Error, MAE)** calcula el promedio de los valores absolutos de las diferencias entre las predicciones y los valores reales. La fórmula para el MAE es:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Donde N es el número de observaciones, y_i es el valor real y $\hat{y}_i$ es el valor predicho. El MAE se expresa en las mismas unidades que la variable que se está prediciendo, lo que facilita su interpretación directa.

B.2. Error Cuadrático Medio (MSE)

El **Error Cuadrático Medio (Mean Squared Error, MSE)** mide el promedio de los cuadrados de los errores, es decir, la diferencia entre los valores predichos y los valores reales. La fórmula para el MSE es:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

El MSE penaliza fuertemente los errores grandes debido al término cuadrático, lo que lo hace sensible a valores atípicos. Su principal desventaja es que su unidad es el cuadrado de la unidad de la variable predicha, lo que dificulta su interpretabilidad directa en el contexto original de los datos.

B.3. Raíz del Error Cuadrático Medio (RMSE)

La **Raíz del Error Cuadrático Medio (Root Mean Squared Error, RMSE)** es la raíz cuadrada del MSE. Ofrece una métrica en las mismas unidades que la variable que se predice, lo que facilita su interpretación y la comparación directa con el MAE. La fórmula para el RMSE es:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

El RMSE es una de las métricas más utilizadas para evaluar la precisión de los modelos de regresión y predicción, ya que proporciona una buena indicación del error promedio de las predicciones en la escala original de los datos, con una mayor penalización para errores grandes en comparación con el MAE.

B.4. Error Porcentual Absoluto Medio (MAPE)

El **Error Porcentual Absoluto Medio (Mean Absolute Percentage Error, MAPE)** es una métrica de precisión que expresa el error como un porcentaje del valor real. Es intuitiva y fácil de entender, lo que la hace popular en contextos de negocio. La fórmula para el MAPE es:

$$\text{MAPE} = \frac{1}{N} \sum_{i=1}^N \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100 \%$$

Una ventaja clave del MAPE es que el error se presenta como un porcentaje, lo que permite comparar el rendimiento del modelo entre diferentes escalas de series temporales. Sin embargo, presenta limitaciones: se vuelve indefinido si el valor real (y_i) es 0 y puede dar resultados muy grandes si y_i es muy pequeño. Para precios de *spot instances*, donde los precios pueden ser bajos pero rara vez cero, esta limitación es menos crítica, pero debe tenerse en cuenta.

Anexo C

Análisis detallado de la volatilidad y dinámica de precios

Este anexo complementa el Capítulo 4.2 proporcionando un análisis de dos aspectos clave de los datos de precios Spot: la volatilidad por región y el comportamiento horario de los cambios de precio.

C.1. Índice de volatilidad regional

Para cuantificar y comparar la inestabilidad de los precios entre las distintas regiones de AWS, se diseñó un índice de volatilidad (IV). Este índice es una métrica ponderada que agrega cuatro componentes fundamentales:

- **Coefficiente de Variación (CV):** Mide la dispersión de los precios respecto a su media.
- **Rango Normalizado (RN):** Indica la magnitud máxima de las fluctuaciones en relación con el precio promedio (μ). Se calcula como: $RN = \frac{P_{max} - P_{min}}{\mu}$.
- **Cambio Porcentual Promedio (CPM):** Cuantifica la magnitud promedio de las variaciones de precio entre actualizaciones consecutivas.
- **Cambios por Instancia (CPI):** Representa la frecuencia promedio de cambios por instancia, un indicador directo de inestabilidad.

Matemáticamente, el índice de volatilidad se define como una combinación lineal ponderada de estos componentes normalizados, para asegurar que cada métrica contribuye de forma equitativa:

$$IV = (0,3 \times \frac{CV}{\max(CV)}) + (0,2 \times \frac{RN}{\max(RN)}) + (0,2 \times \frac{CPM}{\max(CPM)}) + (0,3 \times \frac{CPI}{\max(CPI)})$$

Donde $\max(\cdot)$ denota el valor máximo de cada métrica en el conjunto de datos, utilizado para normalizar los valores entre 0 y 1. La Tabla ?? presenta el resumen completo de estas métricas para todas las regiones analizadas, ordenadas de mayor a menor índice de volatilidad.

Tabla C.1: Resultados del análisis de volatilidad en las regiones

Region	IV	CV	RN	CPM	CPI	Num. Tipos Instancia
ap-southeast-2	0.9007	2.0701	70.4403	0.5203	263.9033	3525
ca-central-1	0.7716	1.5048	54.7970	0.3485	316.9869	2519
sa-east-1	0.7648	1.6619	34.2703	0.6628	245.3933	2446
ap-southeast-1	0.7598	2.0056	30.0602	0.5626	231.6961	3300
eu-north-1	0.7338	1.6192	29.5400	0.6242	245.7113	2425
ap-south-1	0.7274	1.9419	25.9106	0.5687	217.4443	2800
eu-west-2	0.6923	1.5501	23.6212	0.3671	313.8346	2696
eu-central-1	0.6800	1.5684	26.2606	0.5582	227.0501	3976
ap-northeast-3	0.6684	1.4882	22.4135	0.3923	293.1173	1262
ap-northeast-1	0.6682	1.4299	19.9488	0.3688	317.3414	3875
ap-northeast-2	0.6626	1.5439	27.5725	0.4824	232.8320	2792
us-west-2	0.6568	1.4222	18.1117	0.3715	310.9811	5503
us-east-2	0.6515	1.5312	22.8685	0.5387	218.8090	4130
us-east-1	0.6485	1.5348	33.6667	0.4972	195.4500	7366
eu-west-1	0.6476	1.3650	15.7854	0.3478	324.8386	4121
eu-west-3	0.6462	1.3942	12.3013	0.4041	311.0803	2168
us-west-1	0.5627	1.4460	20.7658	0.5215	148.1079	2753

C.2. Análisis del comportamiento horario

Para complementar el análisis, se examinó la distribución de los cambios de precio a lo largo de las horas del día (en UTC). En la Figura C.1 se observa un pico de actividad muy claro a las **15:00 horas UTC**, indicando una concentración de cambios a nivel global. A pesar de que las regiones de AWS se extienden por diferentes zonas horarias, este patrón horario muestra una notable similitud entre ellas al alinearse a la hora UTC.

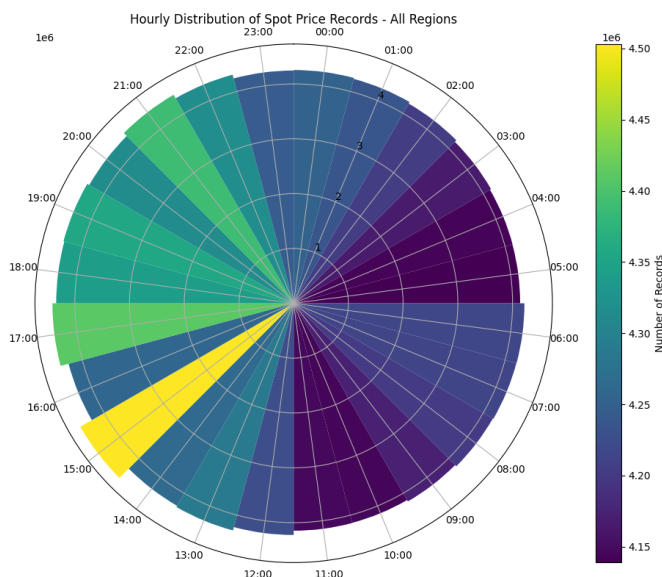


Figura C.1: Distribución horaria (UTC) de la frecuencia de cambios de precio.

Anexo D

Métricas de negocio

Este anexo detalla los indicadores clave de negocio (KPIs) diseñados específicamente para este trabajo con el fin de cuantificar el valor práctico y financiero de las predicciones del modelo, complementando así las métricas de error estándar.

D.1. Precisión de tendencia (*Significant Trend Accuracy*)

Esta métrica mide la capacidad del modelo para predecir correctamente la dirección (subida o bajada) de los movimientos de precio que son suficientemente grandes para ser relevantes. Anticipar la dirección de cambios importantes es crítico para la toma de decisiones proactivas. La métrica representa el porcentaje de aciertos en la dirección del cambio, considerando únicamente los movimientos de precio que superan un umbral predefinido.

La fórmula es la siguiente:

$$\text{significant_trend_accuracy} = \frac{1}{N_{\text{sig}}} \sum_{\substack{t=1 \\ |\Delta y_t| \geq \delta}}^{N_{\text{eval}}} \mathbb{I}(\text{sign}(\Delta \hat{y}_t) = \text{sign}(\Delta y_t)) \times 100\%$$

Donde:

- $\text{sign}(\Delta \hat{y}_t)$ es el signo de la variación porcentual del precio **predicho** por el modelo.
- $\text{sign}(\Delta y_t)$ es el signo de la variación porcentual del precio **real**.
- La condición en el sumatorio, $|\Delta y_t| \geq \delta$, asegura que solo se evalúen los *timesteps* donde el cambio de precio real supera un **umbral de significancia** δ (ej., un 2 %).
- $\mathbb{I}(\cdot)$ es la **función indicadora**, que vale 1 si la predicción de la dirección es correcta (signos iguales) y 0 en caso contrario.
- N_{sig} es el número total de movimientos de precio significativos encontrados en el conjunto de evaluación.
- N_{eval} es el número total de *timesteps* evaluados.

D.2. Ahorro de coste (*Cost Savings*)

Cuantifica el beneficio financiero directo que se obtendría al usar el modelo para decidir el momento óptimo de aprovisionamiento de una instancia *spot*. En cada ventana de decisión, se simula la elección entre ejecutar una tarea de inmediato o posponerla. El ahorro porcentual mide la reducción de coste lograda al seguir la recomendación del modelo frente a la ejecución inmediata.

La fórmula para una única ventana de decisión es:

$$\text{spot_price_savings}_{\text{individual}} = \frac{y_{t_0} - y_{\text{óptimo predicho}}}{y_{t_0}} \times 100 \%$$

Donde:

- y_{t_0} es el precio real en el momento inicial de la decisión (ejecutar ahora).
- $y_{\text{óptimo predicho}}$ es el precio real en el *timestep* futuro que el modelo había identificado como el más barato dentro de la ventana.

D.3. Ahorro con información perfecta (*Perfect Information Savings*)

Esta métrica establece el máximo ahorro teórico posible. Funciona como un punto de referencia o límite superior, calculando el ahorro que se habría conseguido si se conocieran de antemano todos los precios futuros. Para ello, compara el coste de ejecución inmediata con el coste en el punto de mínimo precio real dentro de la ventana.

La fórmula para una única ventana de decisión es:

$$\text{perfect_information_savings}_{\text{individual}} = \frac{y_{t_0} - y_{\text{mínimo real}}}{y_{t_0}} \times 100 \%$$

Donde:

- y_{t_0} es el precio real en el momento inicial de la decisión.
- $y_{\text{mínimo real}}$ es el precio más bajo que se observó realmente dentro de la ventana de decisión.

D.4. Eficiencia del ahorro (*Savings Efficiency*)

Mide la efectividad del modelo para capturar el potencial de ahorro disponible. Se calcula como la relación entre el ahorro medio que logra el modelo y el ahorro máximo que se podría haber conseguido con información perfecta. Un valor del 100 % indicaría que el modelo es tan bueno como un oráculo perfecto para tomar decisiones de coste.

La fórmula es:

$$\text{savings_efficiency} = \left(\frac{\text{spot_price_savings}_{\text{promedio}}}{\text{perfect_information_savings}_{\text{promedio}}} \right) \times 100 \%$$

Donde:

- $\text{spot_price_savings}_{\text{promedio}}$ es el ahorro de coste promedio logrado por el modelo en todas las ventanas de evaluación.
- $\text{perfect_information_savings}_{\text{promedio}}$ es el ahorro promedio obtenido con información perfecta.

Anexo E

Artefactos del experimento y enfoque dirigido por configuración

Este anexo detalla el enfoque de diseño dirigido por configuración y los archivos generados en el entrenamiento de modelos. Se explicará cómo un único archivo de configuración centraliza todos los parámetros de un experimento y, a continuación, se presentará una traza de los directorios y archivos (artefactos) generados en cada fase del *pipeline*, desde la carga de datos hasta la evaluación final.

E.1. Diseño basado en configuración

La reproducibilidad y flexibilidad del sistema residen en el archivo `config.yaml`, que actúa como la única **fuentes de verdad** para cada ejecución. Este enfoque permite una experimentación ágil y una trazabilidad completa, ya que todos los parámetros del modelo, los datos y el entrenamiento se definen en este archivo sin necesidad de modificar el código fuente.

Para cada experimento, se crea un directorio único que almacena una copia del `config.yaml` utilizado junto con todos los artefactos generados, garantizando la total reproducibilidad. La estructura del archivo YAML se divide en secciones lógicas que controlan cada fase del pipeline: `sequence_config`, `dataset_features`, `dataset_config`, `model_config`, `loss_config`, `training_hyperparams` y `evaluate_config`.

E.2. Estructura de Artefactos Generados

El pipeline, orquestado por `load_train_evaluate.py`, genera una estructura de directorios estandarizada para la trazabilidad y el análisis.

E.2.1. Directorio de datos: data/

La clase `LoadSpotDataset` procesa los datos brutos según la configuración y los divide cronológicamente. Los DataFrames resultantes se guardan en formato `pickle` para una carga eficiente.

- `instance_info.df.pkl`: Metadatos de las instancias filtradas.
- `prices.df.pkl`: Serie temporal de precios completa y preprocesada.

- `train_df.pkl`, `val_df.pkl`, `test_df.pkl`: Subconjuntos cronológicos para el entrenamiento, validación y prueba del modelo.

E.2.2. Directorio de entrenamiento: `training/`

Este directorio contiene los artefactos generados durante la fase de entrenamiento, gestionada por la clase `Training`.

- `training_history.json`: Almacena un registro detallado de la evolución de las métricas (pérdida, MAPE, etc.) en cada época, tanto para el conjunto de entrenamiento como para el de validación. Es la fuente de datos para la visualización del progreso del entrenamiento.
- `metrics.json`: Guarda un resumen con las métricas finales y clave del proceso de entrenamiento, como la mejor pérdida de validación obtenida, el MAPE final y el tiempo total de entrenamiento.

E.2.3. Directorio de evaluación: `evaluation/`

Una vez entrenado el modelo, la clase `Evaluate` lo ejecuta contra el conjunto de prueba y genera los siguientes archivos con los resultados de rendimiento.

- `instance_metrics.json`: Contiene las métricas de rendimiento agregadas para cada `id_instance` individual evaluada.
- `dashboard_metrics.json`: Un archivo JSON estructurado que combina la configuración del modelo, los metadatos de cada instancia y sus métricas de evaluación detalladas. Este archivo está diseñado para ser consumido directamente por un panel de visualización interactivo, facilitando el análisis granular del rendimiento.

E.2.4. Artefactos raíz

En el directorio principal del experimento se encuentran los siguientes archivos:

- `config.yaml`: La copia exacta del archivo de configuración utilizado para el experimento.
- `model.pth`: El modelo de PyTorch entrenado y serializado. Contiene los pesos aprendidos y está listo para ser cargado para inferencia o reentrenamiento.
- `job_info.txt`: Un archivo de texto con metadatos sobre la ejecución, como la fecha de inicio, el ID del proceso y la GPU utilizada.
- `output.log`: Un registro detallado (log) de toda la ejecución del pipeline.
- `training_history.png`: Una visualización gráfica que muestra las curvas de pérdida y otras métricas a lo largo de las épocas.

Anexo F

Resultados de los modelos entrenados

Este anexo recoge los resultados detallados de todos los experimentos formales realizados para la validación de los modelos predictivos. La Tabla F.1 consolida dichas ejecuciones, presentando los hiperparámetros de configuración junto a las métricas de rendimiento, tanto de error estadístico como de negocio. El propósito de esta compilación de datos es ofrecer una referencia exhaustiva que sirva como fundamento empírico para las conclusiones extraídas en el Capítulo 6.

Tabla F.1: Tabla comparativa de los resultados de todos los modelos evaluados en la fase de experimentación.

Arquitectura	Config. Temporal			Config. Arquitectura			Config. Datos			Métricas Error (MAPE %)				Métricas Negocio		
	Épocas	Timestep (h)	Pred. Length (d)	Hidden Dim	T. Entr. (h)	Overfitting	Región	Loss Weight	+Datos ARM	MAPE 0-4d	MAPE 5-20d	MAPE Total	Precisión Tend.	Ahorro Coste	Eficiencia Ahorro	Ahorro Perfecto
featureseq2seq	20	12	2	256.0	0.95	2.33	eu-north-1	0.7	False	22.27	32.00	29.57	54.70	0.54	-16.03	5.84
featureseq2seq	20	4	4	128.0	5.65	3.23	eu-north-1	0.7	False	29.42	38.13	35.95	83.11	0.11	-598.42	4.29

Continúa en la siguiente página

Tabla F.1 – continuación de la página anterior

Arquitectura	Config. Temporal			Config. Arquitectura			Config. Datos			Métricas Error (MAPE %)				Métricas Negocio		
	Épocas	Timestep (h)	Pred. Length (d)	Hidden Dim	T. Entr. (h)	Overfitting	Región	Loss Weight	+Datos ARM	MAPE 0-4d	MAPE 5-20d	MAPE Total	Precisión Tend.	Ahorro Coste	Eficiencia Ahorro	Ahorro Perfecto
featureseq2seq	20	4	2	128.0	1.25	2.79	eu-north-1	0.5	False	28.13	36.50	34.40	81.40	0.30	-331.48	4.29
seq2seq	20	12	4	64.0	0.93	4.66	eu-north-1	1.0	False	17.97	25.19	23.39	47.42	0.94	-80.36	5.84
featureseq2seq	38	12	4	256	2.19	2.21	eu-north-1	1.0	False	13.03	22.44	20.08	52.93	1.09	-49.12	7.32
seq2seq	20	12	2	64.0	1.60	4.17	eu-north-1	1.0	False	20.42	27.98	26.09	48.21	0.51	-27.15	5.84
seq2seq	20	12	3	64.0	0.89	4.56	eu-north-1	1.0	False	22.32	28.06	26.62	52.25	0.73	-52.73	5.84
seq2seq	13	6	4	64.0	1.70	5.75	eu-north-1	1.0	False	27.23	32.59	31.25	53.13	-1.08	-1260.01	4.73
featureseq2seq	20	12	4	256.0	0.70	2.52	eu-north-1	0.5	True	20.62	25.21	24.06	62.12	1.29	-55.42	6.22
featureseq2seq	20	4	2	256.0	1.27	3.25	eu-north-1	0.5	False	23.05	29.72	28.05	81.75	0.21	-308.57	4.29
featureseq2seq	20	12	4	256.0	1.03	2.69	eu-north-1	0.5	False	16.41	21.55	20.27	55.57	0.91	-40.00	5.84
seq2seq	14	6	3	64.0	0.71	5.74	eu-north-1	1.0	False	26.52	29.54	28.78	52.76	-1.56	-1222.54	4.73
lstm	20	4	2	64.0	1.20	1.08	eu-north-1	1.0	False	37.21	39.35	38.81	65.02	0.31	-885.93	4.29
featureseq2seq	20	4	4	256.0	5.67	3.33	eu-north-1	0.5	False	25.90	35.12	32.82	82.34	0.48	-323.76	4.29
seq2seq	20	12	5	64.0	0.79	4.78	eu-north-1	1.0	False	16.85	24.99	22.96	47.61	1.06	-79.32	5.84
seq2seq	16	4	5	64.0	5.79	5.71	eu-north-1	1.0	False	26.63	37.10	34.48	67.65	-0.06	-501.91	4.29
featureseq2seq	20	12	4	128.0	1.03	2.47	eu-north-1	0.5	False	19.52	25.24	23.81	55.51	0.70	-46.19	5.84
featureseq2seq	20	12	2	256.0	0.96	2.29	eu-north-1	0.5	False	22.18	31.48	29.15	53.73	0.52	-16.95	5.84
featureseq2seq	20	12	4	512.0	2.27	2.45	eu-north-1	0.5	False	19.54	36.33	32.13	62.30	1.48	-90.92	6.22
featureseq2seq	20	12	4	256.0	1.04	2.81	eu-north-1	0.7	False	16.05	21.56	20.18	53.97	0.95	-42.59	5.84
featureseq2seq	20	12	2	128.0	0.93	2.26	eu-north-1	0.7	False	19.95	26.61	24.95	54.65	0.49	-23.60	5.84
featureseq2seq	20	12	2	256	1.94	1.75	eu-north-1	1.0	False	22.27	32.00	29.57	54.70	0.54	-16.03	5.84
featureseq2seq	20	12	2	128.0	0.94	1.95	eu-north-1	0.5	False	19.74	26.85	25.07	54.75	0.46	-23.78	5.84
featureseq2seq	40	4	2	256	1.07	2.96	eu-north-1	1.0	False	17.85	22.60	21.41	77.83	0.64	-1113.23	4.29
gru	40	12	4	256	0.54	3.22	eu-north-1	1.0	False	12.62	22.90	20.33	50.53	1.17	-49.48	7.32
featureseq2seq	22	12	4	256	2.53	3.00	eu-north-1	1.0	False	10.08	26.92	22.71	70.12	1.32	-27.61	5.20
seq2seq	20	6	2	64.0	0.87	6.19	eu-north-1	1.0	False	24.48	28.94	27.82	52.46	-0.49	-898.06	4.73

Continúa en la siguiente página

Tabla F.1 – continuación de la página anterior

Arquitectura	Config. Temporal			Config. Arquitectura			Config. Datos			Métricas Error (MAPE %)				Métricas Negocio		
	Épocas	Timestep (h)	Pred. Length (d)	Hidden Dim	T. Entr. (h)	Overfitting	Región	Loss Weight	+ Datos ARM	MAPE 0-4d	MAPE 5-20d	MAPE Total	Precisión Tend.	Ahorro Coste	Eficiencia Ahorro	Ahorro Perfecto
featureseq2seq	20	12	4	1024.0	3.62	2.62	eu-north-1	0.5	False	18.72	48.88	41.34	61.22	1.69	-88.78	6.22
featureseq2seq	20	4	4	256.0	5.70	3.40	eu-north-1	0.7	False	25.77	34.71	32.48	82.79	0.33	-456.78	4.29
featureseq2seq	20	12	4	128.0	1.02	2.61	eu-north-1	0.7	False	19.63	24.27	23.11	56.00	0.74	-48.67	5.84
seq2seq	40	12	4	256	0.80	1.94	eu-north-1	1.0	False	11.57	26.71	22.92	53.13	1.34	-56.76	7.32
lstm	20	6	2	64.0	0.69	1.90	eu-north-1	1.0	False	31.03	31.26	31.20	58.75	0.25	-992.39	4.73
seq2seq	18	6	5	64.0	3.41	5.96	eu-north-1	1.0	False	30.54	38.57	36.56	63.51	-0.17	-1153.32	4.73
lstm	35	12	4	256	0.54	3.10	eu-north-1	1.0	False	19.47	24.82	23.48	51.22	1.19	-27.69	7.32
seq2seq	17	4	4	64.0	5.34	6.20	eu-north-1	1.0	False	27.14	34.54	32.69	70.37	-0.64	-736.02	4.29
seq2seq	13	4	3	64.0	2.87	5.33	eu-north-1	1.0	False	30.73	33.37	32.71	52.22	-1.77	-989.39	4.29
featureseq2seq	20	4	4	128.0	5.62	3.10	eu-north-1	0.5	False	29.34	38.06	35.88	83.52	0.26	-526.03	4.29
featureseq2seq	20	4	4	256	7.97	3.52	eu-north-1	1.0	False	25.77	34.71	32.48	82.79	0.33	-456.78	4.29
lstm	20	12	2	64.0	1.51	3.94	eu-north-1	1.0	False	32.44	37.75	36.42	51.06	0.72	-33.43	5.84
seq2seq	20	4	2	64.0	1.31	6.09	eu-north-1	1.0	False	26.54	31.73	30.43	52.59	-0.51	-559.41	4.29
featureseq2seq	14	12	4	256	0.43	3.32	ap-northeast-1	1.0	False	5.40	20.87	17.00	58.40	-0.01	-30.10	4.12
lstm	40	12	4	256	0.91	3.97	eu-north-1	1.0	True	29.89	35.62	34.19	53.14	1.20	-84.98	6.22
featureseq2seq	27	12	4	256	1.48	0.89	us-east-1	1.0	False	5.80	13.17	11.33	68.01	0.23	-32.15	3.71
gru	40	12	4	256	0.85	3.99	eu-north-1	1.0	False	19.44	29.79	27.20	55.77	1.22	-38.23	6.22
lstm	40	12	4	256	0.91	3.97	eu-north-1	1.0	False	29.89	35.62	34.19	53.14	1.20	-84.98	6.22
featureseq2seq	40	12	4	256	1.40	2.83	eu-north-1	1.0	True	19.18	27.80	25.64	60.84	1.53	-62.72	6.22
gru	40	12	4	256	0.85	3.99	eu-north-1	1.0	True	19.44	29.79	27.20	55.77	1.22	-38.23	6.22
featureseq2seq	40	12	4	256	1.40	2.83	eu-north-1	1.0	False	19.18	27.80	25.64	60.84	1.53	-62.72	6.22

Anexo G

Herramienta de visualización para el análisis de resultados

En este anexo se describe la herramienta de visualización interactiva¹ desarrollada como soporte para la experimentación, la cual puede ser explorada en su versión desplegada². Se detallan su arquitectura, funcionalidades y el modo en que facilita el análisis, la interpretación y la comparación de los resultados obtenidos por los distintos modelos. Esta herramienta es fundamental para la selección del modelo predictivo óptimo y para la toma de decisiones informadas a lo largo del ciclo de vida del proyecto, transformando métricas complejas en visualizaciones intuitivas.

G.1. Objetivos y funcionalidades

El objetivo principal del *dashboard* es proporcionar una plataforma centralizada para el análisis comparativo y la validación de los modelos entrenados. Para ello, se han implementado las siguientes funcionalidades clave:

Carga y comparación de modelos La herramienta permite cargar múltiples archivos `.json` con los resultados de las evaluaciones. El usuario puede seleccionar un modelo primario y uno secundario para realizar una comparación directa de sus métricas y configuraciones, lo que resulta esencial para identificar el impacto de los distintos hiperparámetros en el rendimiento.

Sistema de filtrado dinámico Se ha integrado un panel de filtros que permite segmentar los resultados por múltiples dimensiones, como el horizonte temporal, la región, la zona de disponibilidad (AZ), el tipo de instancia, la generación o la familia. Todos los componentes visuales se actualizan dinámicamente para reflejar el subconjunto de datos seleccionado, permitiendo un análisis granular y la validación de la robustez del modelo en escenarios específicos. La Figura G.1 muestra su interfaz.

¹El código fuente de la herramienta está disponible en el repositorio: https://github.com/alvdef/model_dashboard.

²La demostración interactiva de la herramienta se encuentra disponible en: https://alvdef.github.io/model_dashboard/.

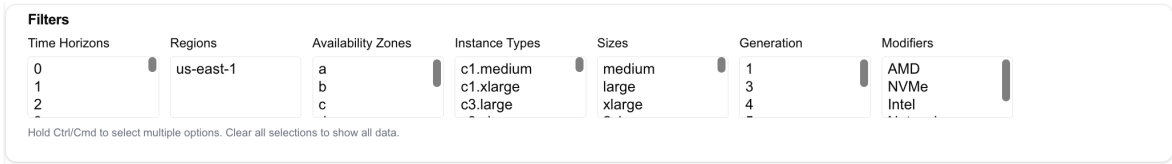


Figura G.1: Sistema de filtrado dinámico de las instancias evaluadas

Análisis de configuraciones Al comparar dos modelos, la interfaz presenta una tabla que resalta las diferencias en sus archivos de configuración (`config.yaml`), como se observa en la Figura G.2. Esta funcionalidad permite correlacionar de manera directa los cambios en los hiperparámetros —como la región de entrenamiento, el tipo de modelo o el tamaño de las capas ocultas— con las variaciones observadas en el rendimiento, agilizando el proceso iterativo de ajuste.

Configuration Property	us-east-1-12h-4d-featureseq2seq_v2	lstm-12h-4d-1.0wt-256hd
Dataset_features > Regions	us-east-1	eu-north-1
Model_config > Instance_feature_size	23	20
Model_config > Model_type	FeatureSeq2Seq_v2	LSTM
Model_name	us-east-1-12h-4d-featureseq2seq_v2	lstm-12h-4d-1.0wt-256hd

Figura G.2: Tabla comparativa de las diferencias de configuración entre dos modelos

G.2. Visualizaciones implementadas

El núcleo de la herramienta es su conjunto de visualizaciones interactivas, diseñadas para ofrecer una perspectiva completa del comportamiento del modelo. La herramienta está desarrollada usando **React** y la librería de visualización **Recharts**.

G.2.1. Comparativa global de modelos

Para facilitar una evaluación inicial rápida, la herramienta presenta una tabla comparativa que consolida las métricas de evaluación más importantes para todos los modelos cargados (Figura G.3). Esta vista permite identificar de un vistazo los modelos con mejor rendimiento en indicadores clave como el **Average MAPE**, **Trend Accuracy** o **Cost Savings**, sirviendo como punto de partida para un análisis más profundo.

G.2.2. Análisis del horizonte temporal y atributos de instancia

La capacidad de un modelo para mantener su precisión a lo largo del tiempo es un factor crítico. La Figura G.4 muestra la evolución del **MAPE** a medida que se extiende el horizonte de predicción, permitiendo identificar el punto en que la fiabilidad del modelo comienza a decaer.

Metric	lstm-12h-4d-1.0wt-256hd	seq2seq-12h-4d-1.0wt-256hd	gru-12h-4d-1.0wt-256hd	featureseq2seq_v2-12h-4d-256hd-reduced	us-east-1-12h-4d-featureseq2seq_v2
Average MAPE (%)	23.48	22.92	20.33	20.08	11.33
Average MSE	0.05	0.05	0.04	0.04	0.03
MAPE Std Dev	22.43	11.15	14.96	13.50	3.71
Min MAPE (%)	1.10	0.66	0.62	0.68	0.00
Max MAPE (%)	102.96	55.72	75.24	64.73	38.74
Trend Accuracy (%)	51.22	53.13	50.53	52.93	68.01
Cost Savings (%)	1.19	1.34	1.17	1.09	0.23
Perfect Savings (%)	7.32	7.32	7.32	7.32	3.71
Savings Efficiency (%)	-27.69	-56.76	-49.48	-49.12	-32.15

Figura G.3: Tabla de comparación de métricas clave para todos los modelos evaluados.

Complementariamente, la Figura G.5 agrupa el rendimiento por características como la generación o el tamaño de la instancia, facilitando la detección de sesgos o nichos de rendimiento.

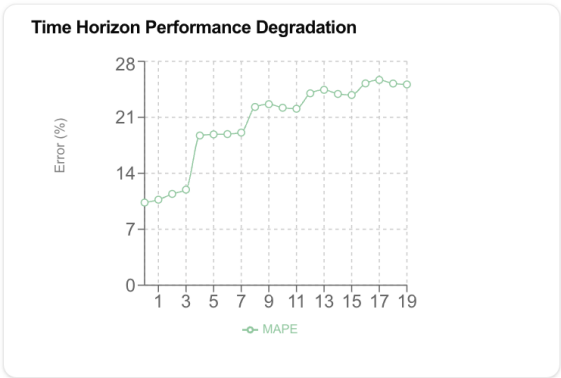


Figura G.4: Evolución del error (MAPE) en función del horizonte de predicción

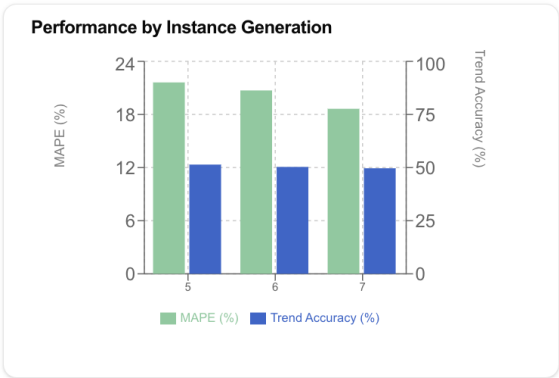


Figura G.5: Rendimiento del modelo (MAPE) agrupado por generación de instancia

Por otro lado, esta vista, similar a una hoja de cálculo, permite un análisis detallado para identificar si la degradación del rendimiento es uniforme o si afecta de manera desproporcionada a ciertos tipos de instancia o momentos específicos del futuro. Las filas representan los tipos de instancia (agrupados por zona de disponibilidad) y las columnas los pasos temporales del horizonte de predicción (de $t+1$ a $t+19$).

G.2.3. Distribución de errores y métricas de negocio

Es crucial cuantificar el valor práctico de las predicciones. La Figura G.8 visualiza la eficiencia del modelo en términos de ahorro. Compara el ahorro real obtenido (*Actual Savings*) con el ahorro máximo teórico que se podría haber alcanzado con información perfecta (*Perfect Savings*). Este gráfico es fundamental para entender el *gap* de rendimiento y subraya una conclusión importante: para que un modelo de predicción sea verdaderamente efectivo en la optimización de costes, es necesario que exista una volatilidad de precios significativa en las instancias objetivo. Si el ahorro perfecto es bajo, el margen de mejora para el modelo es intrínsecamente limitado.

Por otro lado, para una evaluación completa, no basta con analizar el error promedio, sino que

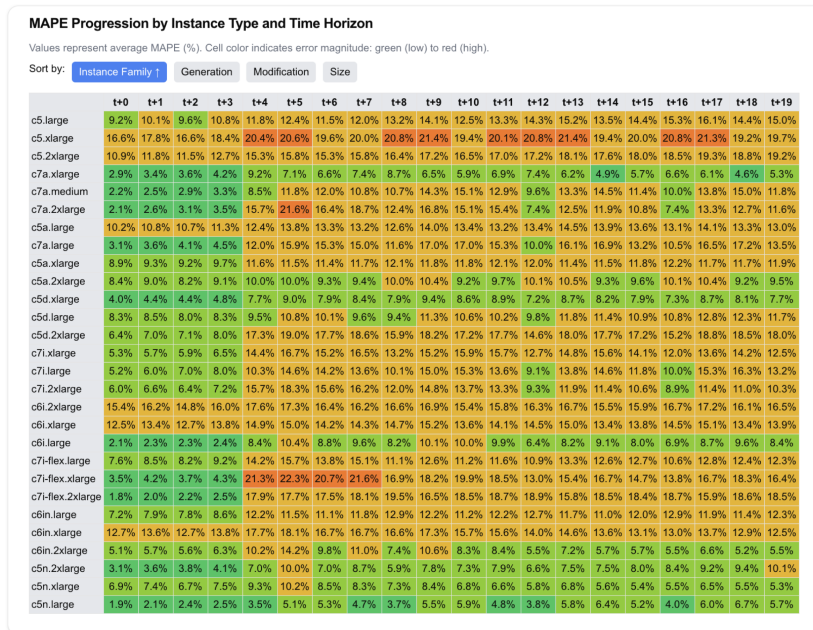


Figura G.6: Matriz de progresión del MAPE por tipo de instancia y horizonte temporal.

es fundamental comprender su distribución. La Figura G.7 presenta un histograma de errores que categoriza las predicciones en rangos de precisión, desde Muy Preciso (<1%) hasta Error >20%.

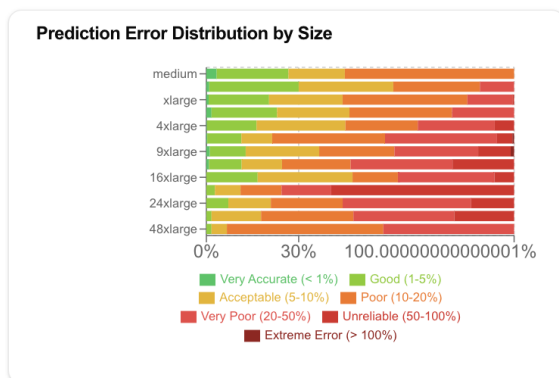


Figura G.7: Distribución de errores de predicción (MAPE) en rangos de precisión

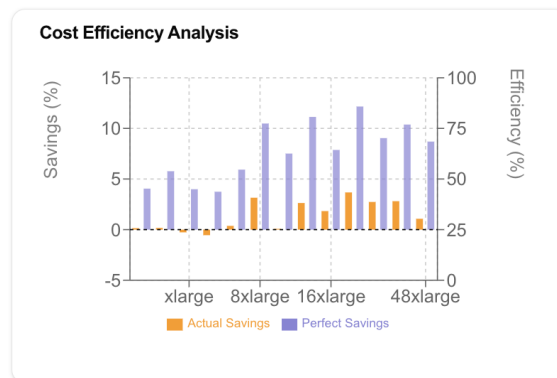


Figura G.8: Análisis de la eficiencia de costes, comparando el ahorro real y el teórico.

Anexo H

Traza de ejecución

El script `load_train_evaluate.py` orquesta el ciclo de vida completo de un modelo predictivo, desde la preparación del entorno y la carga de datos, hasta el entrenamiento y la evaluación. La traza de ejecución que se presenta a continuación detalla los pasos clave del proceso, incluyendo la configuración de GPU, el preprocesamiento de datos, los hiperparámetros de entrenamiento y los resultados de rendimiento.

```
1 Starting spot price prediction pipeline
2 GPU detected: 1 available
3 Using GPU 0: Tesla T4 (14.56 GB)
4 CUDA GPU environment configured for optimal performance
5 CUDA version: 12.6
6 PyTorch CUDA available: True
7 Number of GPUs: 1
8 GPU 0: Tesla T4
9   - Memory: 14.56 GB
10  - Compute capability: 7.5
11  - SM count: 40
12 Working directory: _models/eu-north-1-12h-4d-gru
13 Loading spot price datasets
14 Found credentials from IAM Role: DL-Training-roles
15 Loading info from eu-north-1
16 Filters being applied: {'instance_family': ['c'], 'metal': False,
17 'product_description': ['Linux/UNIX']}
18 Applying instance_family filter: ['c']
19 Instances after instance_family filter: 654
20 Applying product_description filter: ['Linux/UNIX']
21 Instances after product_description filter: 405
22 Applying metal filter: False
23 Instances after metal filter: 365
24 Loaded instance info for eu-north-1
25 Loaded prices for eu-north-1
26 Data saved to _models/eu-north-1-12h-4d-gru
27 Pipeline started on 2025-06-26 12:18:02.260412
28 Train data: 2024-01-01 00:00:00+00:00 to 2025-01-09 00:00:00+00:00, 374 days
29 Validation data: 2025-01-09 00:00:00+00:00 to 2025-03-18 12:00:00+00:00, 68 days
30 Test data: 2025-03-18 12:00:00+00:00 to 2025-05-09 00:00:00+00:00, 51 days
31 Minimum required length for sequences 48. Min group length: 734, Max group length: 1514
32 Created 425469 sequences from 339 instances
33 Dataset created with 425469 sequences
34 Context shape: torch.Size([425469, 40, 1]), Target shape: torch.Size([425469, 8])
35 Instance feature shape: torch.Size([365, 22])
36 Time feature shape: torch.Size([425469, 4, 40])
37 Minimum required length for sequences 48. Min group length: 129, Max group length: 282
38 Created 71126 sequences from 365 instances
39 Dataset created with 71126 sequences
40 Context shape: torch.Size([71126, 40, 1]), Target shape: torch.Size([71126, 8])
41 Instance feature shape: torch.Size([365, 22])
42 Time feature shape: torch.Size([71126, 4, 40])
43 Initializing GRU with parameters:
44 Model parameters:
45   - dropout_rate: 0.2
46   - hidden_size: 256
47   - instance_feature_size: 22
48   - model_type: GRU
49   - num_layers: 2
50   - output_scale: 1.0
51   - shuffle_buffer: 1000
52   - time_feature_size: 4
```

```

53 - timestep_hours: 12
54 - sequence_length: 40
55 - window_step: 1
56 - tr_prediction_length: 8
57 - n_timesteps_metrics: 2
58 - prediction_length: 40
59 Model statistics:
60 - Total parameters: 595,720
61 - Trainable parameters: 595,720
62 Training started at: 2025-06-26 12:18:54.897992
63 No previous training history found at _models/eu-north-1-12h-4d-gru/training/training_history.json
64 Initialized TrendFocusLoss with weights: MSE=1.0, Trend=0.0, Threshold=0.02
65 Training Configuration:
66 - Model: GRU
67 - Input sequence length: 40
68 - Prediction length: 40
69 - Window step: 1
70 - Batch size: 128
71 - Learning rate: 0.0001
72 - Max learning rate: 2e-06
73 - Weight decay: 5e-05
74 - Epochs: 40
75 - Early stopping patience: 5
76 - Optimizer: AdamW
77 - Loss function: TrendFocusLoss
78 Preparing normalizer for the model...
79 Computed global normalization stats: mean=0.4069, std=0.0868 from 339 instances
80 Normalizer prepared for 339 instances
81 Global normalization mean: 0.4069, std: 0.0868
82 Epoch| Train Loss| Val Loss| mse| mape| trnd_acc| cost_sav| sav_eff| LR| Duration
83 -----
84 1| 0.429960| 0.317742| 0.3177| 94.2736| 50.9221| 0.3072| -4.9092| 2.7e-07| 76.9
85 2| 0.358862| 0.227206| 0.2272| 129.8840| 51.9677| 0.3576| -4.7657| 4.6e-07| 75.6
86 3| 0.177927| 0.066032| 0.0660| 240.9491| 52.3526| 0.7136| -10.7091| 7.6e-07| 75.4
87 4| 0.028839| 0.007524| 0.0075| 39.8254| 50.3539| 0.7088| -13.8282| 1.1e-06| 75.2
88 5| 0.004100| 0.004732| 0.0047| 35.2388| 50.2823| 0.5354| -5.2680| 1.4e-06| 74.9
89 6| 0.001925| 0.003240| 0.0032| 22.9130| 50.1495| 0.4317| -10.5298| 1.7e-06| 74.7
90 7| 0.001018| 0.002411| 0.0024| 15.7730| 51.5539| 0.5909| -9.0426| 1.9e-06| 75.4
91 8| 0.000590| 0.001867| 0.0019| 7.9544| 52.0691| 0.3623| -11.8687| 2.0e-06| 74.8
92 9| 0.000396| 0.001509| 0.0015| 5.1874| 53.8218| 0.5398| -10.0215| 2.0e-06| 75.3
93 10| 0.000326| 0.001366| 0.0014| 4.7688| 52.8625| 0.4385| -16.1471| 2.0e-06| 75.6
94 11| 0.000289| 0.001236| 0.0012| 4.6700| 52.7813| 0.1688| -21.4844| 2.0e-06| 75.5
95 12| 0.000263| 0.001140| 0.0011| 4.5455| 51.5539| 0.0428| -15.4964| 1.9e-06| 76.1
96 13| 0.000244| 0.001051| 0.0011| 3.9045| 52.1719| 0.1618| -11.4469| 1.9e-06| 76.4
97 14| 0.000229| 0.000996| 0.0010| 3.8363| 51.6094| 0.1621| -12.5606| 1.8e-06| 76.4
98 15| 0.000217| 0.000956| 0.0010| 4.0946| 50.8704| 0.0802| -11.8029| 1.8e-06| 76.6
99 16| 0.000206| 0.000895| 0.0009| 4.0366| 50.7807| 0.0422| -9.1888| 1.7e-06| 76.4
100 17| 0.000197| 0.000835| 0.0008| 3.5918| 51.7007| 0.1468| -12.4885| 1.6e-06| 76.5
101 18| 0.000190| 0.000786| 0.0008| 3.6764| 53.5268| 0.2193| -11.3306| 1.6e-06| 76.6
102 19| 0.000184| 0.000773| 0.0008| 3.4717| 50.9851| 0.1038| -3.5180| 1.5e-06| 76.6
103 20| 0.000179| 0.000755| 0.0008| 3.4101| 51.6390| 0.1506| -10.8620| 1.4e-06| 76.4
104 21| 0.000174| 0.000733| 0.0007| 3.4871| 52.4356| 0.1425| -9.8956| 1.3e-06| 76.5
105 22| 0.000171| 0.000700| 0.0007| 3.4896| 52.6747| 0.1513| -12.7039| 1.2e-06| 76.5
106 23| 0.000168| 0.000684| 0.0007| 3.4165| 53.6014| 0.1879| -9.6624| 1.1e-06| 76.5
107 24| 0.000165| 0.000685| 0.0007| 3.2612| 53.0004| 0.1797| -9.9796| 1.0e-06| 76.7
108 25| 0.000163| 0.000661| 0.0007| 3.1763| 51.4812| 0.1452| -10.6797| 9.0e-07| 76.7
109 26| 0.000161| 0.000646| 0.0006| 3.2319| 52.2546| 0.1455| -10.6698| 8.0e-07| 76.6
110 27| 0.000159| 0.000635| 0.0006| 3.1691| 54.0913| 0.1708| -9.9052| 7.1e-07| 76.4
111 28| 0.000158| 0.000629| 0.0006| 3.1420| 53.3311| 0.2143| -10.3670| 6.2e-07| 76.4
112 29| 0.000157| 0.000628| 0.0006| 3.2064| 53.4271| 0.1707| -10.9762| 5.3e-07| 76.6
113 30| 0.000156| 0.000621| 0.0006| 3.1285| 53.1735| 0.2252| -6.3539| 4.4e-07| 76.4
114 31| 0.000155| 0.000618| 0.0006| 3.0948| 53.5976| 0.2445| -6.4921| 3.7e-07| 76.4
115 32| 0.000155| 0.000612| 0.0006| 3.2880| 53.6378| 0.2478| -9.4994| 2.9e-07| 76.4
116 33| 0.000154| 0.000615| 0.0006| 3.1271| 53.7272| 0.2504| -9.5201| 2.3e-07| 76.4
117 34| 0.000154| 0.000611| 0.0006| 3.1429| 53.6397| 0.2378| -6.7671| 1.7e-07| 76.5
118 35| 0.000153| 0.000607| 0.0006| 3.1724| 53.8561| 0.2565| -8.9242| 1.2e-07| 76.7
119 36| 0.000153| 0.000611| 0.0006| 3.1237| 53.1263| 0.2192| -7.3398| 7.6e-08| 76.4
120 37| 0.000153| 0.000610| 0.0006| 3.2347| 53.2808| 0.2600| -7.8601| 4.3e-08| 76.5
121 38| 0.000153| 0.000613| 0.0006| 3.2085| 53.1739| 0.2270| -7.8058| 1.9e-08| 76.4
122 39| 0.000153| 0.000610| 0.0006| 3.1464| 53.3769| 0.2278| -7.9510| 4.8e-09| 76.6
123 40| 0.000153| 0.000610| 0.0006| 3.1480| 53.3420| 0.2280| -7.9865| 2.0e-11| 76.5
124 EARLY STOPPING: Patience of 5 epochs without improvement reached at epoch 40.
125 Best eval loss was 0.000607.
126 Early stopping triggered - halting training
127 Saving current model state and metrics...
128 Saved training history to _models/eu-north-1-12h-4d-gru/training/training_history.json
129 Saved metrics summary to _models/eu-north-1-12h-4d-gru/training/metrics.json
130 Training completed at: 2025-06-26 13:16:49.842997
131 Minimum required length for sequences 80. Min group length: 115, Max group length: 182
132 Created 30019 sequences from 365 instances
133 Dataset created with 30019 sequences
134 Context shape: torch.Size([30019, 40, 1]), Target shape: torch.Size([30019, 40])
135 Instance feature shape: torch.Size([365, 22])
136 Time feature shape: torch.Size([30019, 4, 40])
137 Loading model for evaluation
138 Initializing GRU with parameters:
139 Model parameters:

```

```
140 - dropout_rate: 0.2
141 - hidden_size: 256
142 - instance_feature_size: 22
143 - model_type: GRU
144 - num_layers: 2
145 - output_scale: 1.0
146 - shuffle_buffer: 1000
147 - time_feature_size: 4
148 - timestep_hours: 12
149 - sequence_length: 40
150 - window_step: 1
151 - tr_prediction_length: 8
152 - n_timesteps_metrics: 2
153 - prediction_length: 40
154 Model statistics:
155 - Total parameters: 595,720
156 - Trainable parameters: 595,720
157 Starting model evaluation
158 Starting model evaluation
159 - Model type: GRU
160 - Input sequence length: 40
161 - Prediction length: 40
162 - Window step: 1
163 - Batch size: 32
164 - Total instances to evaluate: 365
165 - Total instances in model's normalizer: 339
166 Metrics saved to _models/eu-north-1-12h-4d-gru/evaluation
167 Metrics saved to _models/eu-north-1-12h-4d-gru/evaluation
168 Evaluation completed successfully
```

Anexo I

Estimación del esfuerzo

En este anexo se presenta una estimación detallada del esfuerzo, en horas, dedicado a cada una de las fases y tareas que componen este Trabajo Fin de Grado. La tabla desglosa el proyecto en sus componentes principales, desde la investigación inicial y el diseño de la solución, hasta el desarrollo de los módulos, la experimentación y la redacción de la memoria. Esta estimación cuantitativa complementa la planificación del proyecto y permite dimensionar el trabajo realizado en cada etapa.

Tabla I.1: Tabla de estimación de esfuerzo por fases y tareas.

Fase / Tarea	Horas Estimadas	Descripción
Fase 1: Investigación y fundamentos		
Revisión bibliográfica	30h	Estudio de los fundamentos de Cloud (AWS, EC2), instancias Spot, series temporales y arquitecturas de <i>Deep Learning</i> (RNN, LSTM, etc.).
Análisis del estado del arte	20h	Investigación de trabajos previos sobre predicción de precios, modelos locales y globales, y herramientas comerciales para posicionar el proyecto.
Fase 2: Diseño de la solución		
Definición de requisitos y arquitectura	6h	Especificación de requisitos funcionales y no funcionales. Diseño de la arquitectura modular del sistema, incluyendo los componentes de captura, predicción y API.
Fase 3: Sistema de captura y gestión de datos		
Desarrollo del módulo de captura	25h	Implementación de las funciones AWS Lambda con Boto3 para la recolección de precios Spot, On-Demand y otros metadatos.
Diseño e implementación de la BBDD	25h	Creación del esquema relacional en PostgreSQL (RDS). Configuración de índices (BRIN), vistas y seguridad.
Análisis y caracterización de datos	10h	Análisis exploratorio para entender la dinámica, frecuencia y volatilidad de los precios, guiando el diseño del modelo.
Fase 4: Modelo predictivo de precios		
Preprocesamiento y generación de datos	20h	Creación de secuencias temporales, normalización por instancia y codificación de características para alimentar los modelos.

Continúa en la página siguiente

Tabla I.1 – continuación de la página anterior

Fase / Tarea	Horas Estimadas	Descripción
Implementación del framework de entrenamiento	50h	Desarrollo de la estructura de código para el entrenamiento configurable, la carga de datos, el bucle de entrenamiento/validación y la persistencia de artefactos.
Implementación de arquitecturas de IA	20h	Desarrollo en PyTorch de los modelos <i>baseline</i> (LSTM, GRU) y avanzados (Seq2Seq, FeatureSeq2Seq).
Desarrollo de la API de servicio	2h	Creación de la API RESTful con FastAPI para la servitización del modelo, incluyendo el <i>endpoint</i> de predicción.
Fase 5: Experimentación y resultados		
Fase exploratoria de hiperparámetros	20h	Búsqueda y evaluación inicial de hiperparámetros clave como la longitud de secuencia, el grano horario y la longitud de predicción.
Refinamiento y entrenamiento final	20h	Entrenamiento de los modelos finalistas con los parámetros óptimos, explorando el tamaño de la capa oculta y el impacto de datos diversificados.
Desarrollo del dashboard de visualización	20h	Implementación de la herramienta interactiva con React y Recharts para el análisis comparativo de los resultados de los experimentos.
Análisis final de resultados	10h	Interpretación de las métricas de error y de negocio utilizando el dashboard para la selección del modelo finalista.
Fase 6: Integración en orquestador		
Diseño de la integración con Airflow	5h	Diseño conceptual del ‘CostOptimizationSensor’ y el flujo de decisión lógico para la integración en un DAG de Airflow.
Fase 7: Redacción de la memoria		
Redacción y maquetación	60h	Escritura de todos los capítulos y anexos de la memoria. Creación de figuras, tablas y formato en LaTeX.
Fase 8: Revisión y entrega		
Revisión final y preparación de defensa	5h	Revisión integral del documento, correcciones final.
Total Estimado	370h	